

Алгоритмы в алгебре и теории чисел

Айдагулов Р.Р.

22 ноября 2022 г.

0.1 Введение

В последние 20 лет производительность процессоров выросла незначительно. Их максимальная частота выросла только с 3ГГц до 5ГГц. Потребность в сложных вычислениях продолжает расти. Поэтому, удовлетворять эти потребности возможно только за счет распараллеливания и за счет выбора более эффективных алгоритмов. Первый путь не всегда дает хороший результат. Если в алгоритме 10% операций нельзя вычислить параллельно, даже миллион процессоров не сможет вычислить в десять раз быстрее, чем один процессор. Это обосновывает поднятие эффективности используемых в вычислениях алгоритмов.

Курс начинается с элементарных понятий об эффективности алгоритмов, сравнении сложности разных операций на компьютере. Приводятся эффективные алгоритмы умножения больших чисел и матриц. В курс включены так же традиционные разделы теории чисел, используемых в криптографии – проверка на простоту, дискретное логарифмирование, факторизация больших чисел. Примерно половину материала можно найти в книге:

Р. Крэндал, К.Померанс. «Простые числа. Криптографические и вычислительные аспекты.» Москва, URSS, 2011.

Часть в следующих книгах:

П. Ноден, К. Китте. Алгебраическая алгоритмика», Москва, «Мир», 1999 г.

Р. Грэхем, Д. Кнут, О. Паташник. «Конкретная математика.» Москва, «Мир», 1998.

С. Дасгупта, Х. Пападимитриу, У. Вазирани. «Алгоритмы». Москва, Изд-во МЦНМО, 2014 г.

Глава 1

Основные понятия

1.1 Элементарные операции на компьютере

Наиболее быстрыми операциями, выполняемыми на компьютере являются побитовые операции – сдвиги, переходы к битовому отрицанию (значение каждого бита $x = 0, 1$ меняется на $1 - x$), логические операции И, ИЛИ. Несколько медленнее выполняются операции сложения, вычитания за счет переносов, далее операции сравнения (требуется вычесть одно от другого и сравнить с 0). Операции умножения, деления выполняются на порядок медленнее. Поэтому, в вычислительных задачах примерно 90% времени расчета уходит на выполнение операций умножения, деления. Для ускорения вычислений алгоритмов с умножениями по модулю простого числа лучше заменить операцию умножения на быструю операцию сдвига. Допустим порядок числа 2 в группе Z_p^* (группа обратимых элементов кольца)

$$\text{ord}_p(2) = m.$$

Это означает, что m минимальное натуральное число k , удовлетворяющее сравнению $2^k = 1 \pmod p$. Порядок m является делителем порядка группы, равного $p - 1$. Для подавляющего большинства простых чисел p отношение $k = \frac{p-1}{m}$ равно 1 или мало. Это позволяет перебирать вычеты в два цикла. В первом цикле $g^i, i = 0, 1, \dots, k - 1$, взятые по модулю, во втором $g^i 2^j$, где каждый следующий член получается сдвигом на 1 бит влево и в случае превышения p вычитываем p . Таким образом, $p - 1$ умножений заменяется на малое число k умножений и $p - 1$ сдвигов. Когда нет других умножений, такая замена ускоряет вычисление в 10-12 раз.

1.1.1 Компьютерная арифметика

Целые числа на компьютере представляются последовательностью из d двоичных цифр 0 или 1. Число d характеризует формат чисел и обычно оно равно $d = 16, 32, 64$. Операции сложения, вычитания и произведения выполняются по модулю 2^d . Результатом операции деления является целая часть при делении, когда отношение чисел r неотрицательное. Иначе, результатом деления будет $\text{sign}(r)[|r|]$ целая часть модуля, взятое со знаком отношения. Целые числа разделяются на беззнаковые с меткой **unsigned** и обычные (со знаком). Если числа не отмечены как беззнаковые, то меняется их интерпретация. Числа $x \geq 2^{d-1}$ интерпретируются как отрицательные $x - 2^d$. В частности, число из d единиц

представляет -1. Знаки чисел, получаемые после выполнения арифметических операций, могут не совпасть с ожидаемым. Например, сумма 2^{d-1} чисел, равных 1, станет отрицательным числом. Результатом произведения двух таких положительных чисел x, y , что $1^{d-1} \leq xy < 2^d$ будет отрицательное число. Количество различных целых чисел в одном формате равно 2^d .

В компьютерах используются так же числа с плавающей запятой. В этом случае часть битов отводится на мантиссу и часть на разряд. Пусть l битов относится на мантиссу, l_r на разряд. Для таких чисел мантисса и разряд имеют знаки и им отводятся отдельные биты. Поэтому $d = l + l_r + 2$. Представимые целые числа имеют вид:

$$x = \pm y * 2^{r-l}, \quad -2^{d-2-l} \leq 2^{d-2-l}.$$

Все эти числа рациональные, со знаменателями, являющимися степенями двойки. Если в мантиссе стоит число меньше 2^{l-1} , то можно умножить это число на 2, уменьшая r на 1, не меняя представляемое число. Поэтому, все числа с разрядом $r > -2^{d-2-l}$ можно считать имеющими 1 в старшем бите мантиссы. Только при $r = -2^{d-2-l}$ мантисса принимает произвольное значение. Таким образом, количество различных представимых чисел в формате (d, l) равно

$$2^l(2^{d-l-1}) + (2^{l+1} - 1) = 2^{d-1} + 2^l - 1.$$

Это почти в 2 раза меньше количества представимых целых чисел. Обычно используется формат **float** - (32, 24) и **double** - (64, 53). Длина мантиссы $l = 53$ относится к компилятору **visual studio**. В других компиляторах (**Netbeans**) это число может равняться $l = 52$.

Арифметика чисел с плавающей запятой на компьютере отличается от арифметики целых чисел. При сложении, вычитании эти числа приводятся к разряду числа с большим значением разряда. При этом число, имеющее меньший разряд, может обнулиться. После приведения к общему разряду сумма может превзойти $2^l - 1$, тогда младший бит исчезнет, а разряд увеличится на 1. Возможна и ситуация, когда при сложении (чисел с противоположными знаками) или вычитании часть старших битов обнулятся. Тогда разряд уменьшается на количество обнулившихся разрядов, биты мантиссы сдвигаются влево добавляя нули с конца. При этом происходит существенное понижение точности вычислений. Аналогично, при вычислении произведения таких чисел, мантиссой произведения является l старших битов произведения мантисс и разряд произведения вычисляется с учетом отбрасываемых младших битов. Знаки чисел после операции вычисляются отдельно, не как в случае целых чисел. При умножении целых чисел отбрасываются старшие биты не уместившиеся в формат числа (значением будет произведение по модулю 2^d), а при умножении чисел с плавающей запятой, отбрасываются младшие биты, не уместившиеся в формат мантиссы. Отличие чисел с плавающей запятой (действительных чисел) и целых чисел заключается в использовании архимедовой и неархимедовой 2-адической топологии.

Таким образом, умножение целых чисел и чисел с плавающей запятой существенно отличаются. Если в результате длинных вычислений без деления над числами целого формата ответом является число, представимое в формате целых чисел, то он находится точно, независимо от того, выходили ли промежуточные результаты вычисления из этого формата. При аналогичных вычислениях с числами с плавающей запятой в качестве ответа можно получить все что угодно, даже в случае, когда все промежуточные результаты вычислений оставались в рамках представимых чисел. Поэтому, для уверенной точности вычислений, задачу сложных вычислений с плавающей запятой сводят к целочисленному

или к вычислению с повышенной точностью, увеличивая формат d . При делении в формате целых чисел операция выполняется в архимедовой топологии. Например при делении на нечетное число ответом будет не умножение на обратное по модулю 2^d , а целая часть от деления. А деление на 2 является делением на нильпотентный элемент $2^d = 0 \pmod{2^d}$ и теряется точность, сильно в архимедовой топологии и слабо в 2-адической. Например, результат вычисления на компьютере $\frac{a+b}{2}$ для чисел $a < 2^d, b < 2^d, a+b > 2^d$ даст $\frac{a+b}{2} - 2^{d-1}$. Число 2^{d-1} большое в архимедовой топологии и малое в 2-адической.

В физических приложениях чаще используются переменные с плавающей запятой **float** формата (32, 24) и **double** формата (64, 53). За счет накопления ошибок при расчетах использование формата **float** часто приводит к качественно неверному результату, например не находит все действительные корни многочлена с малым параметром. Поэтому числа с плавающей запятой размера $d = 16$ не предусмотрены. У переменной **double** на мантиссу отводится 53 бита, что более чем в 2 раза больше, чем у переменной **float** и казусы с качественно неверными вычислениями происходит реже.

Другие операции компьютер проводит сводя их к перечисленным выше. Например, операции с комплексными числами или операции вычисления специальных функций сводятся специальными программами к элементарным операциям сложения, вычитания, умножения и деления. Так как последние выполняются на порядок медленнее, сложность вычисления можно свести к оценке количества умножений в алгоритме. Производительность процессоров, компьютеров и измеряются в соответствующих единицах Gflops (сколько миллиардов умножений с плавающей запятой может выполнить компьютер за одну секунду).

Эти вещи надо знать когда используешь числа с плавающей точкой. В нашем случае они появляются при приближенных вычислениях. Например, при приближенном вычислении степени a^b для целых a, b . Точное значение может состоять из тысяч бит при необходимой точности в 1%.

1.2 Основные сведения из алгебры

Перечислим основные сведения из алгебры, часто используемые в алгоритмах.

Понятие группы. Преимущественно используются конечные абелевы группы. Циклические группы. Порядок конечной группы.

Порядок элемента x в группе, обозначаемое через $ord(x)$ в конечной группе является делителем порядка группы.

Структура конечных абелевых групп. Конечная абелева группа есть прямая сумма циклических: $G = Z_{m_1} \oplus Z_{m_2} \oplus \dots \oplus Z_{m_k}, m_1 | m_2 | \dots | m_k$.

Мультипликативные функции из множества натуральных чисел в кольцо, удовлетворяющее требованию

$$g(mn) = g(m)g(n), (m, n) = 1.$$

Здесь $(m, n) = \gcd(m, n)$ - наибольший общий делитель. Если указанное свойство выполняется при всех m, n (не только для взаимно простых), то функция называется строго мультипликативной. Мультипликативные функции полностью определяются со своими значениями на степенях простых чисел. Задав произвольные значения на степенях простых чисел можно определить мультипликативную функцию как произведения значений

из разложения аргумента на простые. Значение в 1 будет 1, как произведение пустого количества элементов.

Кольца вычетов, обозначаемые здесь так же Z_m , как и конечные циклические группы. Группа обратимых элементов кольца Z_m^* (группа Эйлера), имеющая порядок $\phi(m)$, где $\phi(m)$ мультипликативная функция Эйлера, определенная на степенях простых как $\phi(p^k) = p^k - p^{k-1} = p^k(1 - 1/p)$.

Идеалы, главные идеалы. Факторкольца.

Поля. Z_p (p — простое число) является полем F_p из p элементов. Любое конечное поле является алгебраическим расширением такого поля и $q = p^k$ элементов. Простое число p называется характеристикой поля. Под алгебраическим расширением поля F понимается кольцо многочленов над полем F факторизованное по максимальному идеалу. Если многочлен f не приводим над полем F , то главный идеал, порожденный элементом (f) максимален. Фактор по этому идеалу кольца многочленов дает алгебраическое расширение порядка степени многочлена. Хотя в курсе не предполагается знание теории алгебраических чисел, знакомство ими по книге Ленг "Алгебраические числа" желательно.

Другие необходимые сведения из алгебры и теории чисел будут даны по ходу курса по необходимости.

1.3 Эффективность алгоритмов

Эффективность алгоритмов для решения некоторой задачи измеряется тем, сколько потребуется сделать операций данному алгоритму для решения задачи. Под задачей (или проблемой) понимается здесь массовая проблема со входными данными, занимающими n бит (двоичных знаков для представления чисел). Точнее надо учесть и размер выходных данных и оценить количество операций $T(n)$, где n суммарное количество битов входных и выходных данных. Например, при вычислении степени натурального числа a^k , входные данные занимают $\lceil \log k \rceil + 1$ для числа k и $\lceil \log a \rceil + 1$ для числа a , выходные гораздо больше. Здесь и далее всюду под $\log(k)$ (без указания основания логарифма) понимается логарифм по основанию 2. Когда число a является степенью 2 (основанием исчисления в компьютере) ответ получается мгновенно. В других случаях степень можно вычислить не более чем $2 \log(k)$ умножениями. Правда надо учесть, что вычисляемые степени быстро увеличиваются и приходится умножать числа имеющие порядка n битов в записи. Общее количество операций будет расти как количество операций при умножении двух k битовых (больших) чисел, т.е. при использовании быстрого алгоритма умножения $O(k \log k)$ операций, более точная оценка (как будет видно ниже) $O(k \log^2 k)$.

Количество операций, требуемых для решения задачи данным алгоритмом редко удается точно вычислить. Автору известно только то, что доказательно это установлено для вычисления значений многочлена по схеме Горнера:

$$a_0 + a_1x + \dots + a_nx^n = a_0 + x(a_1 + x(\dots + x(a_{n-1} + a_nx)\dots)).$$

Установлено, что если степень трансцендентности поля $K(a_0, \dots, a_n)$ над K $n + 1$, то нельзя вычислить многочлен без n сложений, а так же нельзя вычислить без n умножений. Условие существенно, например, если многочлен окажется вида $(ax + b)^n$ потребуется не более одного сложения и не более $2 \log n$ умножений. Можно вычислить минимальное число команд, необходимых для программы, решающую данную задачу. Это тоже

может потребовать очень большого количества вычислений. В качестве примера рассмотрим экзотическую систему команд, представимых в системе вычисления Конвея. Любая вычислимая строго монотонная последовательность натуральных чисел больше $x_n > 1$ может быть вычислимо с помощью набора рациональных чисел

$$r_1, r_2, \dots, r_k$$

по следующей схеме. На вход подается число $N_0 = 2$, каждое следующее N_{i+1} вычисляется как $N_i r_j$, где j минимальный номер, для которого указанное выражение $N_{i+1} = N_i r_j$ целое. Все числа из последовательности N_i , являющиеся степенями двойки 2^{x_j} дадут x_j , ($x_1 > 1$). Сам набор рациональных чисел

$$r_1, r_2, \dots, r_k$$

представляет набор команд в экзотическом языке программирования, вычисляющую заданную последовательность чисел. Конвей нашел набор из 14 рациональных чисел для вычисления всех простых чисел в порядке роста. Для этого достаточно даже девяти рациональных чисел:

$$r_i = \frac{3}{5}, \frac{67375}{108}, \frac{175}{18}, \frac{55}{39}, \frac{1}{3}, \frac{26}{77}, \frac{6}{7}, \frac{9}{13}, 189.$$

Можно доказать, что восемью или меньшим числом рациональных чисел нельзя вычислить последовательность всех простых чисел, предварительно сводя количество бесконечного числа наборов рациональных чисел к конечному набору кандидатов, могущих вычислять простые числа.

Колмогоров ввел понятие сложности для массовой проблемы как количество операций $f(n)$, необходимых для решения задачи со входом n для некоторой оптимальной программы, такой что для любой другой программы существует такая константа C , что она решит проблему за число операций не меньше чем $f(n) - C$. Доказывается существование такого алгоритма и такой функции. Однако доказательство не конструктивное. Дело в том, что даже множество алгоритмов, решающих данную массовую проблему не является разрешимым множеством. Точнее, задача вначале должна быть формализовано. Формализованная задача имеет хотя бы один (обычно не эффективный, переборный) алгоритм решения. Соответственно решает ли данный алгоритм указанную задачу формализуется как вычислимость одной и той же функции двумя программами. В этом смысле нет единого алгоритма, который работая с двумя номерами программ выдавало бы ответ - да (вычисляют одну функцию) или нет при любых данных. В этом смысле понимается неразрешимость множества алгоритмов, решающих задачу.

Имея одну программу решения задачи, можно улучшить ее. Например, вычислив (пусть и очень долго) результаты для первых миллиард значений входного параметра, мы выдаем значение из вычисленной таблицы, если n мало и ранее по предварительному вычислению определено, иначе приступаем к вычислению по старой схеме. Это показывает, что и сама константа, присутствующая в определении не является ограниченной и вычислимой.

Обычно с ростом n сложность задачи растет. Однако, это не всегда так. В качестве примера рассмотрим задачу вычисления y_n , определяемого рекуррентным соотношением:

$$y_1 = 1, y_{n+1} = \sqrt{\frac{n+1}{3}} \sin(y_n \sqrt{\frac{3}{n}})$$

с точностью до 10^{-12} .

Для многих рекуррентных задач можно получить асимптотическое разложение:

$$y_n = g_0(n) \left(1 + \frac{a_{10} + a_{11} \log n}{n} + \dots + \frac{P_k(\log n)}{n^k} + \dots \right)$$

с вычисляемыми коэффициентами многочленов $P_k(\log n)$ от логарифма. Старшие коэффициенты a_{ii} многочлена $P_i(\log n)$ не зависят от начального значения в рекуррентном соотношении. Для приведенной выше задачи $g_0(n) = 1$, $a_{11} = \frac{1}{2}$ и оставшиеся члены находятся в диапазоне $\frac{-2}{n} < \dots < 0$. Следовательно, при $n > 10^{14}$ мы можем смело считать (с нужной точностью) $y_n = 1$.

Для справки асимптотическим разложением функции $f(x)$ при $x \rightarrow x_0$ (обычно $x_0 = \infty$) называется разложение:

$$f(x) = g_0(x) + g_1(x) + \dots$$

такое что, выполняется $\frac{f(x) - g_0(x) - \dots - g_k(x)}{g_k(x)} \rightarrow 0$ при $x \rightarrow x_0$. В качестве упражнения получите первые члены асимптотического разложения для последовательностей, определенных рекуррентными соотношениями:

- 1) $x_0 = 1, x_{i+1} = x_i + \frac{1}{x_i}$.
- 2) $x_0 = 1, x_{i+1} = x_i + \frac{i}{x_i}$.

В отличие от сходящихся рядов, асимптотическое разложение обычно расходится и для каждого из окрестности x_0 можно пользоваться только несколькими (зависит от степени близости к предельному значению) первыми приближениями.

Для количества числа необходимых операций для решения задачи в лучшем случае удается оценить только нулевым приближением асимптотического разложения, т.е. найти функцию $g(n)$, так, что выполняется:

$$\lim_{n \rightarrow \infty} \frac{T(n)}{g(n)} = 1.$$

В этом случае будем писать $T(n) = \theta(g(n))$.

Чаще удается оценить только порядок роста: $0 < C_1 g(n) < T(n) < C_2 g(n)$. В таком случае используется обозначение: $T(n) = \Omega(g(n))$.

Наличие такой оценки обычно достаточно для сравнения двух различных алгоритмов. Обычно $g(n) \rightarrow \infty$ при n стремящимся к бесконечности, тогда отношение логарифмов стремится к 1:

$$\frac{\log(T(n))}{\log(g(n))} = 1 + O\left(\frac{1}{\log(g(n))}\right) \rightarrow \log(T(n)) = \theta(\log(g(n))).$$

Рост количества операций в зависимости от количество входной информации оценивают через типичные (простого вида) функции, такие как - $\log n, n^a, \exp(an^\beta)$ и т.д.

Использование термина сложность по отношению к алгоритмам не совсем корректно. Мы будем пользоваться термином эффективность вычисления для сравнения различных алгоритмов решения задачи. Обычно более эффективные алгоритмы (решающие задачу за значительно меньшее количество операций по сравнению с простыми алгоритмами) достаточно сложны и требуют существенных математических знаний. В качестве примера можно привести вычисление точного количества простых чисел $\pi(x)$ не превосходящих x ,

при $x = 10^k, k = 1, 2, \dots, 25$. Мы не можем сказать, что в будущем не появятся еще более эффективные алгоритмы, решающие нашу задачу. Сказать, что они будут простыми, только из за того, что они решают задачу более эффективно при больших значениях входного параметра n нельзя. Обычно более эффективные алгоритмы вычисления $f(n)$, считают быстрее простых алгоритмов только при больших $n > n_0$. Например, все эффективные алгоритмы факторизации числа n уступают простейшему алгоритму при малых $n < 10^8$.

1.4 Принцип – разделяй и властвуй

Принцип разделяй и властвуй относится к сведению задачи с большим входным параметром n на несколько (m) задач с параметром в l раз меньше - $\frac{n}{l}$. Далее задачи с уменьшенными значениями параметров делятся на еще более мелкие задачи и т.д. Так задача с размером (параметром) n фильтруется на подзадачи как l дерево. При разделе и сборе (объединении или слиянии) решенных мелких задач требуется выполнять определенное количество операций Cn^α . На общее количество операций получаем рекуррентное соотношение:

$$T(n) \leq mT\left(\frac{n}{l}\right) + Cn^\alpha.$$

Тогда $T(n)$ оценивается как

$$T(n) \leq Cn^\alpha + mC\left(\frac{n}{l}\right)^\alpha + m^2T\left(\frac{n}{l^2}\right) \leq \dots$$

$$Cn^\alpha(1 + x + x^2 + \dots + x^k), \quad x = \frac{m}{l^\alpha}, \quad k = \lceil \log_l n \rceil.$$

Считаем, что при $i > k, (\frac{n}{l^i} < 1)$ $T(n) = 0$ при $n \leq 1$.

Когда $m > l^\alpha$ (или $\alpha < \log_l m = \beta, x > 1$) это дает оценку $T(n) < \frac{Cx}{x-1}n^\alpha x^l \leq C_1 n^\beta, C_1 = \frac{Cx}{x-1}$.

Когда $\alpha > \beta$ получаем оценку $T(n) < C_2 n^\alpha, C_2 = \frac{C}{1-x}$.

Когда $\alpha = \beta, (x = 1)$, получается оценка $T(n) \leq Cn^\alpha \log_l n$.

Обычно используют $l = 2$ и называют это методом бинарного дерева. То, что n не является точной степенью l роли не играет, можно оценить $T(n) \leq T(l^k), k = \lceil \log_l n \rceil + 1$ или доказать, что возможные неравномерности деления дадут вклад $o()$ малое от основной оценки. Если неравенство в рекуррентном соотношении выполняется в обратном направлении с другой константой C , то такое же неравенство верно в другую сторону с другой константой. Соответственно можно писать из

$$T(n) = mT\left(\frac{n}{l}\right) + \Omega(n^a),$$

то

$$T(n) = \begin{cases} \Omega(n^{\max(\alpha, \beta)}), & \alpha \neq \beta = \log_l m \\ \Omega(n^\alpha \log n), & \alpha = \beta. \end{cases}$$

Эту оценку называют мастер теоремой.

В качестве упражнения на деление целого числа примерно на равные целые части (максимальное от минимального отличается не более чем на 1) предлагаю доказать равенство:

$$n = \left\lfloor \frac{n}{l} \right\rfloor + \left\lfloor \frac{n+1}{l} \right\rfloor + \dots + \left\lfloor \frac{n+l-1}{l} \right\rfloor.$$

Это равенство верно при любом целом n и натуральном k .

В качестве простейшего примера разделения на мелкие подзадачи рассмотрим вычисление суммы n величин примерно одного порядка, встречающиеся при вычислении интегралов.

$$S = \Delta x \left[\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + i\Delta x) \right], \quad \Delta x = \frac{b-a}{n}.$$

Если суммировать последовательно, прибавляя вычисленной до этого сумме последующее значение, то ошибка счета растет быстро. Так как вычисленная сумма k величин примерно в k раз больше последующего $k+1$ -го слагаемого, то ошибка на этом шаге будет порядка $k\varepsilon$. Складывая ошибки получаем ошибку порядка $\frac{n^2}{2}\varepsilon$. При суммировании количество операций (можно сказать) не меняется. Оно может меняться при суммировании целых чисел за счет выхода формата суммы от формата представления слагаемых.

Если суммировать по принципу разделяй и властвуй, то эти недостатки почти устраняются. В этом случае на нижнем уровне суммируются пары последовательных чисел, то что остается без пары переносится на следующий уровень. На втором этапе суммируются суммы соседних пар и т.д. При этом ошибка суммирования действительных чисел растет медленнее (не быстрее линейного $n\varepsilon$). Количество увеличения общего количества операций за счет перехода суммы к другому формату при суммировании целых чисел так же будет как правило значительно меньше.

Плюс к этому, вычисления можно произвести параллельно на многоядерном компьютере увеличивая скорость вычисления. При вычислении произведения n величин преимущества последнего увеличиваются. Такой способ вычисления в дальнейшем назовем вычислением методом бинарного дерева.

В качестве примера можно рассмотреть сумму членов массива **float**. Пусть задан массив $s[N]$ длиной $N=100\,000\,000$, и требуется вычислить сумму $S = \sum_i s[i]$. Для простоты рассмотрим случай $s[i] = 1$. Так как для переменных **float** мантиссе отводится только 24 бита, то сумма достигнув до величины $2^{24} = 16777216$ больше не увеличится, так как при приравнивании разрядов для суммирования слагаемые 1 не попадают в разряд мантисы суммы. Таким образом вместо суммы 100 000 000 получим 16777216 представленной в экспоненциальной записи. Ничего этого не произойдет, если бы суммировали объединяя на пары на каждом уровне суммирования (методом бинарного дерева).

В качестве упражнения читателю предлагается разобрать сложение $s[i] = (float)i$. Здесь уже при переводе целых чисел в формат действительных теряется точность. Анализировать суммы в формате действительных чисел $S = \sum_i s[i]$

- 1) в случае суммирования подряд в сторону роста,
- 2) в сторону убывания (в обратном порядке),
- 3) объединяя в пары с начала и конца.

1.5 Сортировка и поиск

Часто данные упорядочены по одному признаку, а для решения некоторой задачи требуется упорядочение по другому признаку. Эта задача сводится к упорядочению массива x_i . Определим функцию

$$\pi(X|x) = \#(x_i \leq x, x_i \in X).$$

Здесь X множество значений нашего массива. Соответственно $\pi(X|x)$ означает количество значений из массива, которые не превосходят число x . Она является монотонно растущей ступенчатой, непрерывной справа функцией. Её значение равно 0, пока x меньше минимального значения из массива, когда $x = x_{min}$ принимает значение 1 и при следующем по росту значении массива функция $\pi(X|x)$ принимает значение 2 и т.д. Между двумя значениями функция принимает постоянное значение равное значению слева (непрерывность справа). Задача сортировки сводится к построению обратной функции к $\pi(X|x)$, а задача поиска по значению x сводится к вычислению этой обратной функции для конкретного значения.

Так как задача поиска проще, рассмотрим вначале эту задачу. Пусть $x_1 < \dots < x_n$. Нам надо найти по заданному x номер k , такой, что $x_k \leq x < x_{k+1}$. Это может быть данные в архиве, которые упорядочены (например по алфавиту) и каждый занимает некоторое количество байт и мы не знаем где найти данные с таким значением. Принцип разделяй и властвуй соответствует разделению области поиска на две части, т.е. смотреть какое значение данных по середине $x_{n/2}$. Если там значение меньше, то нам надо искать справа, иначе слева. Так делением попалам мы найдем нужную информацию за не более чем $\log n$ шагов. Точнее среднее количество шагов при поиске равно

$$1 * \frac{1}{n} + 2 * \frac{1}{n/2} + \dots + k * \frac{1}{n/2^{k-1}} = \frac{1}{n}(1 + 2 * 2^1 + \dots + k * 2^{k-1}) = \frac{(k-1)2^k + 1}{n}, \quad k = \lceil \log n \rceil.$$

Таким образом, среднее количество шагов от максимального отличается не более чем в 2 раза, в среднем по возможным значениям n оно меньше максимального в $\ln 2$ раза, т.е. равно $\frac{\log n}{\ln 2}$.

Если бы $x_k = f(y)$, $y = \frac{k}{n}$ выражалась гладкой функцией, то делая поиск k как решение уравнения $f(y) = x$ с точностью до $\frac{1}{n}$ по методу Ньютона (с помощью касательных) нам бы потребовалось $O(\log \log n)$ шагов.

Метод касательных заключается в поиске аргумента в интервале (a, b) для которого должно быть $f(a) < f(y) = x < f(b)$, последовательно сужая интервалы. До этого мы уже находили (проверили) значения a, b . Находим следующее значение c для y как $a + \frac{x-f(a)}{f(b)-f(a)}(b-a)$. Проверяем значение функции в этой точке и сужаем интервал поиска до (a, c) , если $f(c) > x$, иначе до (c, b) . Здесь отличие от деления интервала пополам заключается в более точном прогнозируемом значении c . Для дифференцируемой функции до точности $\frac{1}{n}$ мы дойдем за указанные $\log \log n$ шагов. Однако, нормальные данные нельзя считать образующими гладкую функцию. Тем не менее можно считать, что положительная разность $x_{k+1} - x_k$ имеет распределение случайной величины, некоторое математическое ожидание и дисперсию. Тогда разность $x_{i+k} - x_i$ как сумма k таких разностей имеет k раз большую дисперсию и математическое ожидание. Соответственно ошибка определения k в следующем шаге будет $\sqrt{L}$. Здесь L длина точности поиска в единицах средне квадратичного отклонения Δx в предшествующий момент, а $\sqrt{L}$ та же длина погрешности, после поиска по методу Ньютона в следующий момент. Таким образом, поиск в случае случайного распределения шага $x_{k+1} - x_k$ приводит к результату примерно за то же количество шагов $O(\log \log n)$, как и в случае гладкой функции. Только коэффициент перед O большим в первом случае оценивается через $|\frac{f''}{f'}|$ (находится сразу, если функция линейная), а во втором случае отношением средне квадратичного отклонения разницы $x_{k+1} - x_k$ к ее математическому ожиданию. Здесь ускорение поиска связано с тем, что данные в некотором

смысле равномерны. Пока под этим будем понимать, что они не являются специально подобранными, тем самым исключительными, т.е. встречающимися в естественных данных с исключительно малой вероятностью, что на подсчет среднего количества шагов они не влияют из-за большой редкости встретить такие исключительные данные.

Рассмотрим теперь задачу сортировки. В этом случае мы должны каждому значению x_i найти место $j(i) = \pi(X, x_i)$, такое, чтобы $x_{I(1)} \leq x_{I(2)} \leq \dots \leq x_{I(n)}$, где $I(j)$ обратная функция $I(j(i)) = i$ к $j(i)$.

Сами данные могут быть большими массивами. Поэтому не желательно их перемещать. Желательно найти такую функцию $I(j)$, располагающую их в нужном порядке, например по алфавитному порядку автора статьи, для быстрого поиска.

Метод разделяй и властвуй сводит задачу к упорядочению двух примерно одинаковых кусков длин $\lfloor \frac{n_1}{2} \rfloor, \lfloor \frac{n_1+1}{2} \rfloor$, а затем упорядочению пары массивов слиянием:

$$x_{I(k)} \leq x_{I(k+1)} \leq \dots \leq x_{I(k+m-1)}, m = \lfloor \frac{n_1}{2} \rfloor$$

$$x_{I(k+m)} \leq x_{I(k+m+1)} \leq \dots \leq x_{I(k+n_1-1)}.$$

Считается, что предварительно создали массив упорядочивания $I(j) = j$, которое в шаге обработки будет обновляться.

Вводим два индекса (для движков) i_1, i_2 , для указания номеров элементов из разных массивов, которых сравниваем на данный момент. Первоначально положим $i_1 = k, i_2 = k + m$. Вводим еще массив для новой нумерации $In[j]$. Вначале положим индекс $j = 0$ (как первый элемент массива). Пока $i_1 < k + m$ или $i_2 < k + n_1$ делаем:

$$If(x_{I[i_1]} \leq x_{I[i_2]}) In[j++] = I[i_1++],$$

$$else In[j++] = I[i_2++].$$

Здесь не корректно оформили случай, когда не выполняется одно из условий пока. В этом случае надо переписать оставшуюся часть другого массива без сравнения в хвост.

Далее нумерацию I заменяем на новую нумерацию In на том же интервале $[k, k + n_1)$. Общее количество сравнений участка длины n_1 есть $n_1 - 1$. Соответственно общее количество операций на k -ом уровне есть $n - M_k, M_k = 2^{k-1}$. Суммируя все сравнения по уровням получаем, что количество операций примерно $n \log n$.

Рассмотрим теперь другой алгоритм, основанный не на разделении пополам, а сразу на множество из m ячеек. Когда данные распределены близко к равномерному (не исключительны) число корзин m можно взять в несколько раз (2-3) больше чем n . Когда данные дикие, в одну ячейку может попасть $n - 1$ членов. Тогда потребуется большое число отведенной памяти $m(n - 1)$ для массива обновления упорядочивания $In[i, j]$ и для хранения номеров членов попавших в каждую ячейку. Соответственно брать $m = 2n$ для такого случая может оказаться не рациональной. Относительно не большое $m \approx \sqrt{n}$ примерно так же быстро работает.

Предварительно создаем нумерацию как и раньше $I(j) = j$, количество участков упорядочивания вначале равно $k = 1$, интервал упорядочивания номеров $[min[0], max[0]) = [0, n)$. Если $n < 2$ то ничего не надо делать. Сперва в каждом интервале упорядочивания находим минимальное и максимальное значение для нанесения координат ячейкам. Они находятся по программе:

$$x_{\min} = x_{I[0]}, x_{\max} = x_{I[0]}, j_{\min} = 0, j_{\max} = 0,$$

```

for(j = 1; j < n; j++) {j1 = I[j]; if(xj1 < xmin) {jmin = j1, xmin = xj1}
    else {if(xj1 > xmax) {jmax = j1, xmax = xj1}}.

```

Сейчас мы можем поставить на место минимальный и максимальный элемент

$$j_1 = I[0], I[0] = I[j_{min}], I[j_{min}] = j_1,$$

$$j_1 = I[n-1], I[n-1] = I[j_{max}], I[j_{max}] = j_1.$$

Если n не больше 3, то уже массив упорядочен. Иначе разделим интервал на m частей вводя границы ячеек $[y[j], y[j+1])$, длиной $\Delta y = \frac{x_{max}-x_{min}}{m}$, где $y[0] = x_{min}$, $y[j+1] = y[j] + \Delta y$. Пройдя по номерам нашего массива определим номера ячеек, в которые они попадают

```

for(j = 1; j < n-1; j++) {i =  $\frac{x_{I[j]} - x_{min}}{\Delta y}$ , In[i, n[i]++] = j}
    k = 0, i0 = 1, i1 = 0,
    while(i0 < n-1) {
        if(n[i1] > 1) {m[k] = i0, n[k++] = n[i1],
        for(j1 = 0; j1 < n[i1]; j1++) I[i0++] = In[i1, j1]; }
        if(n[i1] == 1) I[i0++] = In[i1, 0];
        i1++; }

```

Здесь выделяем ячейки, где более одного элемента для их дальнейшего изучения и заносим их номера в соответствующий массив без внутреннего упорядочивания. Пустые ячейки пропускаются, где всего один элемент их место в упорядочивании уже определяется однозначно. Остальные ячейки подвергаются дальнейшему анализу (находятся там минимальный и максимальный элементы, если больше 4 элементов то делятся опять на части. В случае 3 элементов в корзине, её элементы будут упорядочены при нахождении (без дополнительного сравнения) минимума и максимального элементов. В случае 4 элементов устанавливаем порядок сравнивая 2 оставшихся элемента, не являющимися ни минимумом ни максимумом.

Количество уровней прохождения не превзойдет $n/2$, так как минимумы и максимумы уже установлены на свои места и не участвуют в дальнейшем упорядочении. Допустим, что на k -ом уровне упорядочивания делим не упорядоченные участки на $\frac{2n}{k}$ частей. Тогда общее число сравнений порядка $2n(1 + \frac{1}{2} + \dots + \frac{1}{k}) = O(n \log n)$, $k \leq \frac{n}{2}$. Однако, через 2-3 итерации не понадобится дальнейшее уровни деления при нормальных данных. Даже если после этого останутся не упорядоченные корзины, в одну корзину попадут малое количество элементов массива, которых можно упорядочить даже квадратичным алгоритмом. Точнее, если мы в первый раз уже разделили на $2n$ частей при нормальных данных в одну ячейку попадет не более $O(\log n)$ элементов и число ячеек с 2-мя и более элементами в ячейке будет $O(\sqrt{n})$. Соответственно, для нормальных данных уже после первой итерации можно вторично обработать квадратичным методом за $O(\sqrt{n} \log^2 n) < O(n)$ операций. Даже для дикого случая через 2-3 итераций длины не разделенных по упорядочиванию ячеек станет меньше точности задания данных и дальнейшее разделение для их точного упорядочивания станет бессмысленным. Поэтому, этот алгоритм требует всего $O(n)$

операций для упорядочивания их с точностью задания данных. В случае целых данных точность задания данных 1.

Общее необходимое количество сравнений для упорядочивания массива $O(n \log n)$. Секрет второго подхода заключается в том, что при рассеивании по интервалам, вместо сравнения между собой, мы получаем сразу $\log n$ побитовых сравнений в случае, когда n не велико $n < 2^{32}$. Если n велико, то требуемое количество операций порядка $O(n \frac{\log n}{32})$. Например, при упорядочении квадратичных вычетов по модулю p изначально номеру i соответствует число $x_i = i^2 \bmod p, i = 1, 2, \dots, \frac{p-1}{2}$. При сортировке по значениям каждому вычету j по модулю p сопоставим корзину. Корзины вначале пустые. Это означает, что количество элементов массива в корзине j равна $l[j] = 0$ нулю. Пробегая по массиву x_i номера $i > 0$ кладем в корзину x_i . Далее упорядочиваем массив исключая пустые корзины по программе:

```
j1=0;j=0;
```

```
while( j<p ){if (l[j]){l[+j1]=l[j]; pi[i]=j1; }j++; }
```

В случае сильно не равномерного распределения мы не можем разложить номера массивов по корзинам так, чтобы в каждую корзину попал не более одного номера. В этом случае можно предварительно сортировать по хранилищам корзин не определяя точное место для каждого номера. Рассмотрим это в случае, когда значения (по которым сортируем) являются числа вида **double**. В качестве первого витка сортировки примем знак мантиссы. Вначале в отдельное хранилище поместим отрицательные, в другое не отрицательные. Во втором витке сортируем по разряду. Эту часть так же можно проделать в несколько ступеней, отсортируя вначале по знаку разряда, потом по значению старшего бита разряда и так по количеству битов, отведенных на разряд. Так сортируются в хранилища - папки, где каждый элемент имеет одинаковый знак и одинаковый разряд. Далее разносим по подпапкам в зависимости от значения первых (старших) битов, пока в каждой корзине не станет единственный элемент.

Количество сравнений в алгоритме определяется суммарным количеством битов для разделения значения x_i от соседей слева и справа. Если x_i, x_j имеют $l - 1$ одинаковых битов (в порядке, указанным выше), а на l -ом бите отличаются, то требуется l битовых сравнений для их различения. При l -ом витке сравнений мы различаем (кладем в разные подпапки) те элементы, у которых в первых $l - 1$ битах одинаковые значения. Обозначим через $l[i]$ число битов, необходимых для различия i -го элемента от других слева и через $r[i]$ для различия справа. У самого младшего $l[i] = 0$. у самого старшего $r[i] = 0$. Для соседних чисел $x[i1] < x[i2]$ выполняется $r[i1] = l[i2]$. Отсюда получается следующая теорема:

Теорема 1. *Сортировка за минимальное количество сравнений выполняется за $S = \sum_i \frac{1}{2}(l[i] + r[i])$ сравнений с побочными операциями уточнения местоположения (занесение номера из большой папки в номер подпапки) в количестве $R = \sum_i \max(l[i], r[i])$ и на вывод упорядочения (массива $i[j1]$) $O(n)$ операций.*

Данные могут быть неравномерно распределенными как в случае, когда все значения имеют большое количество L одинаковых старших битов. Тогда чтение (и сравнение по ним) Ln битов не дадут никакой информации об расположении. Можно определить плотность информации об расположении как $\frac{n \log n}{S} \leq 1$ и степень неравномерности как $\frac{S}{n \log n} \geq 1$. Так как число R примерно такого же порядка как и $S \geq n \log n$ количество побитовых операций при сортировке оценивается как $O(S)$.

1.6 Алгоритм Евклида

Алгоритм Евклида предназначен для вычисления наибольшего общего делителя двух натуральных чисел $a = a_0, b = a_1$. Он очень простой, последовательно вычисляются отношения и остатки от деления:

$$q_i = \left[\frac{a_i}{a_{i+1}} \right], a_{i+2} = a_i - q_i a_{i+1}.$$

Числа a_i убывают начиная с первого (a_0 может быть и меньше a_1 , если мы заранее их не поменяем) $a_1 > a_2 > \dots > a_k > a_{k+1} = 0$. Тогда число $a_k = \gcd(a, b)$ будет искомым наибольшим общим делителем. Действительно, с одной стороны

$$a_k | a_{k-1} \rightarrow a_k | a_{k-2} \rightarrow \dots \rightarrow a_k | \gcd(a_0, a_1).$$

С другой стороны для любого общего делителя $d | a_0, d | a_1$:

$$d | a_2 \rightarrow d | a_3 \rightarrow \dots \rightarrow d | a_k.$$

Это означает, что a_k наибольший общий делитель.

Этот алгоритм позволяет определить одновременно еще несколько вещей. Представим отношение

$$\frac{a_0}{a_1} = \frac{c * \gcd(a_0, a_1)}{d * \gcd(a_0, a_1)}$$

в виде:

$$\frac{a_0}{a_1} = q_0 + \frac{1}{\frac{a_1}{a_2}} = q_0 + \frac{1}{q_1 + \frac{1}{\frac{a_2}{a_3}}} = \dots = q_0 + \frac{1}{q_1 + \frac{1}{q_2 + \dots + \frac{1}{q_{k-1}}}}$$

разложения в цепную дробь. Определим рациональные числа

$$\frac{P_l}{Q_l} = q_0 + \frac{1}{q_1 + \frac{1}{q_2 + \dots + \frac{1}{q_l}}}$$

Определяя начальные $P_{-1} = 1, Q_{-1} = 0, P_0 = q_0, Q_0 = 1$ получаем, что

$$P_i = q_i P_{i-1} + P_{i-2}, Q_i = q_i Q_{i-1} + Q_{i-2}.$$

Это соотношение легче получить из того, что

$$a_2 = Q_0 a_0 - P_0 a_1,$$

$$a_3 = -(Q_1 a_0 - P_1 a_1),$$

и по индукции

$$a_{k+1} = a_{k-1} - q_{k-1} a_k = (-1)^{k-1} [(Q_{k-3} a_0 - P_{k-3} a_1) + q_k (Q_{k-2} a_0 - P_{k-2} a_1)].$$

При этом отношения $\frac{P_i}{Q_i}$ оказываются наилучшими приближениями исходного отношения $\alpha = \frac{a}{b}$ (даже если оно иррационально). Когда исходное отношение иррационально,

разложение в цепную дробь не заканчивается и поэтому их часто называют непрерывными дробями.

Приближение наилучшее в том смысле, что

$$|\alpha - \frac{P_i}{Q_i}| \leq \frac{1}{Q_i Q_{i+1}}.$$

Не существует приближения лучше, чем эти при меньших знаменателях. При этом дроби с четными номерами приближаются к нашему отношению снизу, нечетные сверху:

$$\frac{P_0}{Q_0} < \frac{P_2}{Q_2} < \dots < \alpha < \dots < \frac{P_3}{Q_3} < \frac{P_1}{Q_1}.$$

Расстояние между двумя соседними номерами

$$|\frac{P_i}{Q_i} - \frac{P_{i+1}}{Q_{i+1}}| = \frac{1}{Q_i Q_{i+1}}.$$

Доказательство этих фактов не сложные, имеются во многих учебниках и оставляется в качестве упражнения.

Дроби $\frac{P_i}{Q_i}, \frac{P_{i+1}}{Q_{i+1}}$ оказываются действительно соседями, в том смысле, что между ними нет ни одной дроби со знаменателем, не превосходящем максимума их знаменателей.

У любого рационального числа $\frac{P}{Q}, (P, Q) = 1$ (несократимая запись дроби) имеется только два рациональных соседа (одна слева, другая справа) знаменатели которых не превосходят Q . Чтобы их найти надо разложить в непрерывную дробь $\frac{P}{Q} = \frac{P_k}{Q_k}$. Тогда если $Q_k = 1$ (число целое), то соседи есть соседние целые числа. При $Q_k > 1$ одно соседнее число есть $\frac{P_{k-1}}{Q_{k-1}}$, другое $\frac{P_k - P_{k-1}}{Q_k - Q_{k-1}}$. Если k четное, то первое число сосед сверху, второе снизу. Когда k нечетное, все наоборот. Если рациональные числа $\frac{a}{b}, \frac{c}{d}$ соседние (знаменатели всегда считаются положительными), то

$$|\frac{a}{b} - \frac{c}{d}| = \frac{1}{bd}.$$

Это соотношение позволяет найти обратные по модулю. Пусть

$$ad - bc = 1.$$

Тогда

$$d = a^{-1} \mod b, a - c = b^{-1} \mod a.$$

Например, в нашем случае, если $\gcd(a, b) = 1, \frac{a}{b} = \frac{P_k}{Q_k}$, то при четном k

$$P_{k-1} = b^{-1} \mod a, a - Q_{k-1} = a^{-1} \mod b,$$

при нечетном

$$b - P_{k-1} = b^{-1} \mod a, Q_{k-1} = a^{-1} \mod b.$$

В дальнейшем часто понадобится найти обратное по модулю другого взаимно простого числа. Покажем, как это вычисляется алгоритмом Евклида при обратном ходе. Пусть $\gcd(a_0, a_1) = 1 = a_{k+1}$. Придем к выражению a_{k+1} через a_0, a_1 :

$$a_{k+1} = a_{k-1} - q_{k-1}a_k = a_{k-1} - q_{k-1}(a_{k-2} - q_{k-2}a_{k-1}) = -q_{k-1}a_{k-2} + (1 + q_{k-2}q_{k-1})a_{k-1}.$$

Пусть получена формула:

$$1 = a_{k+1} = (-1)^i (c_{k+1-i} a_{k-1-i} - c_{k-i} a_{k-i}).$$

Тогда получаем, что

$$c_{k-1-i} = c_{k+1-i} + c_{k-i} q_{k-1-i}, \quad c_{k+1} = 1, c_k = q_{k-1}.$$

В итоге получим

$$1 = (-1)^{k+1} (c_2 a_0 - c_1 a_1).$$

На самом деле, таким образом мы вычисляем соседнее рациональное число с меньшим знаменателем $\frac{P_{k-1}}{Q_{k-1}}$ для рационального числа $\frac{a_0}{a_1} = \frac{P_k}{Q_k}$. При этом

$$\frac{P_k}{Q_k} - \frac{P_{k-1}}{Q_{k-1}} = \frac{(-1)^{k-1}}{Q_k Q_{k-1}} \rightarrow a_0 Q_{k-1} - a_1 P_{k-1} = (-1)^{k-1}.$$

Как сам алгоритм так и нахождение обратного (можно их проделать одновременно) осуществляются не более чем за

$$\frac{\log(\min(a, b))}{\log(\frac{\sqrt{5}+1}{2})} + 1$$

шагов и следовательно имеют сложность $O(\log a)$.

Обратную, можно вычислить и возведением в степень $m^{-1} \bmod n = m^{T-1} \bmod p$ для взаимно простых чисел. Здесь T период по модулю n , например $\phi(n)$ или $c(n)$ - функция Кармайкла. Однако при больших n здесь требуется $O(\log n)$ тяжелых операций - умножений больших чисел, в то время как в алгоритме Евклида используется $O(\log n)$ менее тяжелых операций. Поэтому, вычисление обратного по алгоритму Евклида практически всегда более эффективно.

Алгоритм Евклида можно несколько улучшить. Один из способов заключается в разложении такую цепную дробь, что все частные q_i по модулю не меньше 2. Вычисляя каждое частное по формуле:

$$q_i = \left\lfloor \frac{a_i}{a_{i+1}} + \frac{1}{2} \right\rfloor, \text{ ранее использовали } q_i = \left\lfloor \frac{a_i}{a_{i+1}} \right\rfloor.$$

При этом q_i могут быть и отрицательными. Например, для золотого сечения $\frac{1+\sqrt{5}}{2}$ рациональными приближения являются все отношения чисел Фибоначчи $\frac{F_{i+1}}{F_i}$, а в модифицированном алгоритме только четные $\frac{F_{2i+1}}{F_{2i}}$ и результат вычислится примерно в два раза быстрее.

Другой способ заключается в выделении степеней числа 2. Вначале делим числа m на 2, пока они оба делятся на 2. Эта степень числа 2 и есть $\nu_2(gcd)$. При этом одно из них нечетное. Если и второе нечетное, добавив первое приведем к вычислению наибольшего общего делителя двух натуральных чисел, одно из которых (обозначим x_i) нечетное, а другое $y_i = 2^{\nu_2(y_i)} x_{i+1}$ четное. Имеет место:

$$gcd(x_i, y_i) = gcd(x_{i+1}, y_{i+1}), \quad y_{i+1} = x_i + x_{i+1}, \quad \nu_2(y_{i+1}) \geq 1.$$

Продолжая эту процедуру вычисления, придем к числам $gcd(x_k, y_k = 2x_k) = x_k$. Этот алгоритм так же чуть быстрее стандартного и по сложности примерно соответствует приведенному выше.

1.6.1 Евклидовы кольца и алгоритм Евклида

Алгоритм Евклида работает и в кольцах, где определено деление с остатком. Под этим понимается кольцо R , где для любых двух элементов $a, b \neq 0$ существует представление

$$a = cb + r, \quad f(r) < f(b), \quad f : R \rightarrow Z_+, \quad f(x) = 0 \iff x = 0.$$

Здесь r остаток при делении a справа на b такой, что функция $f(r)$ (норма) Евклида меньше, чем функция от b . Такие кольца называются Евклидовыми.

Области целостности (кольца без делителей нуля), являющиеся Евклидовыми факториальными, т.е. любой элемент может быть разложен в простые, определенные однозначно с точностью до обратимых (делителей 1). Если кольцо K факториально, то и кольцо многочленов $K[x]$ факториально. Имеет место аналог этой теоремы для Евклидовых колец. Кольцо многочленов над полем Евклидова. За функцию Евклида для многочленов $f : K[x] \rightarrow Z_+$ можно взять степень с увеличением на 1 $f(P(x)) = \deg(P(x)) + 1$. Нулевой многочлен считается имеющим степень -1. Таким образом, алгоритм Евклида работает в кольце многочленов. Факториальные и Евклидовы области целостности являются областью главных идеалов.

1.7 Модулярная арифметика

Одним из применений принципа разделяй и властвуй является модулярная арифметика. Модулярная арифметика используется для вычислений с большими числами, заменяя большие числа на несколько вычетов по степеням различных простых чисел. Этот факт обосновывается китайской теоремой об остатках.

Теорема 2. *Прямые суммы колец*

$$Z_m \oplus Z_n \cong Z_{\text{lcm}(m,n)} \oplus Z_{\text{gcd}(m,n)}$$

изоморфны. В частности, когда $(m, n) \equiv \text{gcd}(m, n) = 1$ имеет место

$$Z_{mn} \cong Z_m \oplus Z_n.$$

Доказательство. Докажем это вначале для случая взаимной простоты чисел $(m, n) \equiv \text{gcd}(m, n) = 1$. С помощью алгоритма Евклида находим представление $m't - n'n = 1$. Пусть $(x, y) \in Z_m \oplus Z_n$. Определим число $z = ym't - xn'n \pmod{mn}$. Ясно, что $z = x \pmod{m}$, так как $n'n = -1 \pmod{m}$. А так же $z = y \pmod{n}$. Если взять другой элемент $(x', y') \in Z_m \oplus Z_n$ и определить число $z' = y'm't - xn'n \pmod{mn}$, то легко проверяется, что числу $(x, y) + (x', y') = (x + x' \pmod{m}, y + y' \pmod{n})$ соответствует число $z + z' \pmod{mn}$ и числу $(x, y) * (x', y') = (xx' \pmod{m}, yy' \pmod{n})$ соответствует $zz' \pmod{mn}$. Это отображение является обратным к отображению $x = z \pmod{m}, y = z \pmod{n}$. Это значит имеется кольцевой изоморфизм

$$Z_{mn} = Z_m \oplus Z_n, \quad (m, n) = 1.$$

Пусть $m = \prod_p p^{m_p}, n = \prod_p p^{n_p}$ разложение на степени различных простых чисел. Мы доказали, что

$$Z_m = Z_{p_1^{m_{p_1}}} \oplus Z_{m/p_1^{m_{p_1}}} = \dots = \bigoplus_p Z_{p^{m_p}}.$$

Аналогично для Z_n . Следовательно

$$Z_m \oplus Z_n \cong \oplus_p Z_{p^{m_p}} \oplus Z_{p^{n_p}} \cong \oplus_p Z_{p^{\max(m_p, n_p)}} \oplus_p Z_{p^{\min(m_p, n_p)}} \cong Z_{lcm(m, n)} \oplus Z_{gcd(m, n)}.$$

□

Обычно эта теорема применяется в случае $(m, n) = 1$. Если разложение $n = \prod_i p_i^{k_i}$ на простые делители известно, то вычисления по модулю n сводятся вычислениям по модулям $p_i^{k_i}$ и восстановлению z компоненты по известным вычетам по модулям $p_i^{k_i}$. Когда таких модулей много, восстановление надо произвести по методу бинарного дерева разбивая на пары, потом объединяя вычеты по двум парам и т.д. При этом используется алгоритмы быстрого умножения больших чисел. Она используется и при решении диофантовых уравнений. Решение по модулю n сводится к решениям по модулям $p_i^{k_i}$. Если количество решений по модулю $p_i^{k_i}$ равно n_i , то количество решений по модулю $n = \prod_i p_i^{k_i}$ равно $\prod_i n_i$. В частности, если хотя бы по одному из простых p_i нет решений ($n_i = 0$), то нет решений и по модулю n .

Одним из простых применений модулярного метода является их использование в доказательстве первого случая теоремы Ферма для многих простых p :

$$a^p + b^p = c^p$$

не имеет решений с условием $p \nmid abc$, $p > 2$. Простейшее применение заключается в проверке отсутствия таких решений по модулю p^2 . Так как $(a + mp)^p = a^p \pmod{p^2} = a^p \pmod{p} = a \pmod{p}$ и $(-a)^p = -a^p$, это сравнение можно рассмотреть только в случае

$$1 + x^p = (1 + x)^p, \quad 1 \leq x < (p - 1)/2.$$

Случай $x = (p - 1)/2$ сводится к случаю $x = 1$. Отсюда сразу получается, что первый случай верен при $p = 3, 5$. Однако, в случае $p = 6k + 1$ имеется примитивный кубический корень по модулю p . Пусть x такой корень, тогда $x^2 = -(1 + x)$ и $1 + x^p = (1 + x)^p$. Т.е. такой примитивный способ не работает в случае $p = 1 \pmod{6}$. Куммер доказал, что такие решения сравнений не дают реальных решений. Далее, такое доказательство проходит и для многих других $p = 5 \pmod{6}$ начиная с $p = 11$.

Для операций сложения, вычитания по модулю n такая редукция не оправдывается. Она не оправдывается и для одной операции умножения. Когда используется множество операций, как указано ниже при вычислении чисел Бернулли, алгоритм становится достаточно эффективной.

В качестве применения китайской теоремы об остатках рассмотрим решение задачи:

$$x = x_1 \pmod{a}, \quad x = x_2 \pmod{b}.$$

Она имеет решение тогда и только тогда, когда $x_1 = x_2 \pmod{gcd(a, b)}$. Пусть $x_1 = 5, x_2 = 6, a = 268, b = 113$.

Находим вначале $gcd(a_0 = a, a_1 = b)$ по алгоритму Евклида.

$$q_0 = \left[\frac{a_0}{a_1} \right] = 2, a_2 = a_0 - q_0 a_1 = 42, q_1 = \left[\frac{113}{42} \right] = 2, a_3 = a_1 - q_1 a_2 = 29,$$

$$q_2 = 1, a_4 = 13, q_3 = 2, a_5 = 3, q_4 = 4, a_6 = 1 = gcd(a_0, a_1), q_5 = 3.$$

Так как числа взаимно просты условие на совместимость выполняется. Далее вычисляем представление $gcd(a, b)$ через линейную комбинацию a, b .

$$\begin{aligned} a_6 &= a_4 - q_4 a_5 = a_4 - 4(a_3 - q_3 a_4) = -4 * a_3 + 9 * a_4 = -4 * a_3 + 9(a_2 - q_2 * a_3) = 9 * a_2 - 13 * a_3 = \\ &= 9 * a_2 - 13 * (a_1 - q_1 a_2) = -13 a_1 + 35 a_2 = -13 a_1 + 35(a_0 - q_0 a_1) = 35 a_0 - 83 a_1. \end{aligned}$$

Следовательно $35 = a_0^{-1} \mod a_1$, $-83 = a_1^{-1} \mod a_0$ и

$$35 * 6 a_0 - 83 * 5 * a_1 = 6 \mod a_1 = 5 \mod a_0$$

или

$$x = 9385 \mod 268 * 113 = 9385 \mod 30284.$$

Эта теорема позволяет определить так же структуру группы Эйлера Z_n^* - группы взаимно простых с n вычетов по умножению по модулю n . Группа Эйлера имеет порядок $\phi(n) = n \prod_{p|n} (1 - 1/p)$ и

$$Z_n^* = \oplus_i Z_{p_i}^{*k_i}.$$

Теорема 3. *Группа Z_{p^k} циклическая и имеет порядок $p^k - p^{k-1}$, $k > 0$, когда $p > 2$ или $p = 2, k < 3$.*

$$Z_{2^k}^* = Z_2 \oplus Z_{2^{k-2}}, k > 2.$$

Доказательство. Цикличность группы Z_p^* следует стандартным образом из того, что Z_p есть поле. Пусть a образующая этого поля. При $p > 2$ она останется образующей и для Z_{p^k} , если $a^{p-1} \neq 1 \mod p^2, p > 2$. Если это не выполняется, то заменим эту образующую на $a + p$. Для z_2^* образующая 1, для Z_4^* образующая 3. Для $Z_{2^k}^*, k > 2$ квадрат нечетного числа дает остаток 1 при делении на 8 и степени одного числа не заполняют все нечетные остатки. Однако, для любого $a, a^2 = 9 \mod 16$ числа $\pm a^i, i = 0, \dots, 2^{k-2} - 1$ дают все нечетные вычеты. Соответственно,

$$Z_{2^k}^* = Z_2 \oplus Z_{2^{k-2}}, k > 2.$$

□

Максимальный порядок элемента группы Эйлера равна:

$$c(n) = lcm(c(p_i^{k_i}), c(p^k) = p^k(1 - 1/p), p > 2 \text{ or } p = 2, k < 3, c(2) = 1, c(4) = 2.$$

Функция $c(n)$ называется функцией Кармайкла. Она не мультипликативна. Мультипликативными называют функции от натурального аргумента $g(n)$, когда выполняется $g(mn) = g(m)g(n)$, если $(m, n) = 1$ (взаимно просты).

Часто применяют обозначение $(m, n) \equiv gcd(m, n)$, $[m, n] = lcm(m, n)$. Для любого натурального c выполняется $c(m, n) = (cm, cn)$, $c[m, n] = [cm, cn]$.

1.7.1 Модулярная арифметика в криптографии

Модулярная арифметика широко используется в криптографии, где открытый ключ n длины l (в RSA сейчас используют ключ длиной 1024 бита) является большим модулем для вычислений. Для публики в открытую дается функция шифровки $f_e : X \rightarrow X$ из множества вычетов по модулю n в себя. Функция f_e вычисляется просто, обычно не более чем за $O(l^{2+\epsilon})$ операций. Обратная функция $f_e^{(-1)}$ почти не вычислима по открытым данным (n, e) (требуется годы вычислений на суперкомпьютере).

В алгоритме *RSA* роль такой функции играет:

$$f_e(x) = x^e \mod n, \quad n = pq.$$

Здесь n является произведением двух больших простых чисел. Обратная функция имеет такой же вид:

$$f_e^{(-1)}(y) = f_d(y) = y^d \mod n.$$

Само число d известно только хозяину. Чтобы вычислить d по открытым данным n, e надо решить систему сравнений:

$$de = 1 \mod p - 1, \quad de = 1 \mod q - 1 \text{ или } de = 1 \mod c(n).$$

Решение этой задачи просто только если известно $c(n) = lcm(p - 1, q - 1)$ или разложение числа n на простые множители. Факторизовать 1024 битные числа n , используемые в RSA пока не смогли даже на суперкомпьютерах.

В других алгоритмах так же используются трудно вычисляемые функции по большому модулю типа дискретное логарифмирование, обращение автоморфизма на эллиптической кривой.

1.8 Вычисление степени и квадратного корня по модулю простого числа p

Вычисление степени целого числа одна из самых наиболее используемых операций. Для вычисления $b = a^m$ записывают число m в двоичной системе исчисления $m = m_0 + m_1 * 2 + \dots + m_k * 2^k$. Тогда искомое число можно представить в виде:

$$b = (a^{2^0})^{m_0} (a^{2^1})^{m_1} \dots (a^{2^k})^{m_k}.$$

Соответственно вычисляем последовательные квадраты числа и умножаем число b на это число, если соответствующая цифра $m_i = 1$. Программа выглядит так:

```
c=a; if(m &1) b=a; else b=1; m > >=1;
while(m){c=c*c; if(m & 1)b*=c; m > >=1; }
```

Количество операций умножения не меньше чем $\log m$ и не больше, чем $\log m + l - 1$, где l - количество не нулевых двоичных цифр в записи m . Этот метод не всегда оптимален по количеству используемых умножений. Например для вычисления a^{15} потребуется 6 ($\log 15 = 3, l = 4, 6 = 3 + 4 - 1$) операций умножения, в то время как при вычислении $a^2 = a * a, a^4 = a^2 * a^2, a^5 = a^4 * a, a^{10} = a^5 * a^5, a^{15} = a^{10} * a^5$ требуется только 5 операций.

Результат быстро растет в зависимости от степени. Обычно возведение степени вычисляется по модулю некоторого числа n , проверяемого на простоту в тестах на простоту.

Тогда операции умножения заменяются на операции умножения по модулю n (умножение, затем деление результата на n с выводом остатка в качестве результата операции).

Такие операции можно использовать и для вычисления обратного по модулю $b = a^{-1} \bmod n = a^{c(n)-1}$, когда известно $c(n)$. Здесь $c(n)$ максимальный порядок в мультипликативной группе Z_n^* (Группа Эйлера), называемой функцией Кармайкла.

Можно вычислить и дробные степени, т.е. найти решение уравнения

$$x = a^{1/k} \bmod n \iff x^k = a \bmod n.$$

Такая задача решается по модулю $p_i^{k_i}$ для каждого простого делителя $p_i | n$, а затем используя китайскую теорему об остатках находится решение для n . Задача для основания $n = p^l$ решается вначале для простого случая $n = p$, а далее по лемме Гензеля решение для $n = p$ поднимается до решения $n = p^l$. Метод Гензеля является по сути методом Ньютона последовательного приближения к корню уравнения в случае p -адической нормы. Здесь могут возникнуть трудности в случае, когда $p | k$. Мы рассмотрим только случай квадратного корня $k = 2$ по модулю нечетного простого числа $p \nmid k$, т.е. решим уравнение

$$x^2 = a \bmod p.$$

Решение существует тогда и только тогда, когда a квадратичный вычет по модулю p . Последнее можно проверить при помощи квадратичных законов взаимности:

$$\left(\frac{2}{a}\right) = (-1)^{(a^2-1)/8}, \quad \left(\frac{-1}{a}\right) = (-1)^{(a-1)/2},$$

$$\left(\frac{a}{b}\right) = \left(\frac{b}{a}\right)(-1)^{(a-1)(b-1)/4}, \quad \left(\frac{b_1 b_2}{a}\right) = \left(\frac{b_1}{a}\right)\left(\frac{b_2}{a}\right), \quad \left(\frac{b}{a_1 a_2}\right) = \left(\frac{b}{a_1}\right)\left(\frac{b}{a_2}\right).$$

Здесь a, b, a_1, a_2 нечетные натуральные числа. Соответствующий символ $\left(\frac{a}{b}\right)$ называется символом Якоби. В соответствии с законами взаимности числа a, b можно считать бесквадратными, сокращая квадраты от значений символов, равных ± 1 . В случае нечетного простого p из-за цикличности группы Эйлера порядка $p - 1$ следует:

$$\left(\frac{a}{p}\right) = a^{(p-1)/2} \bmod p.$$

Длее мы не будем писать, что равенство выполняется по модулю фиксированного числа p , когда это ясно из контекста.

Пусть p нечетное простое число вида $p - 1 = 2^l m$, $m - odd$, a — квадратичный вычет. Решение уравнения $x^2 = a$ будем искать в виде:

$$x = \epsilon a^{(m+1)/2}.$$

Это выражение даст решение тогда и только тогда, когда

$$\epsilon^2 a^m = 1.$$

Обозначим вычисленное число $a^m = b$ и через s обозначим $s = g^m$, где g произвольный квадратичный вычет. Тогда степени s образуют циклическую группу порядка 2^l . Таким образом, решение представляется в виде:

$$x = c^r a^{(m+1)/2}, \quad r = r_0 + r_1 * 2 + \dots + r_{l-2} * 2^{l-2},$$

при условии

$$c^{2r}b = 1.$$

Здесь в разложении r взяли степени 2 до $l-1$, так как $c^{2^{l-1}} = -1$ и из того, что x решение задачи (о квадратном корне), следует что $-x$ тоже решение. При $l = 1$ поправочный коэффициент не нужен, $x = a^{(m+1)/2}$ является решением. Пусть $l \geq 2$. Очевидно при любом $i \geq 0$ выполняется

$$(c^{2r}b)^{2^i} = c^{r^{2^{i+1}}}b^{2^i} = 1.$$

Подставив сюда $i = l-2$ получаем

$$(-1)^{r_0} = b^{2^i} \Rightarrow r_0 = \begin{cases} 1, & b^{2^{l-2}} = -1 \\ 0, & b^{2^{l-2}} = 1. \end{cases}$$

Далее, если $l \geq 3$ уменьшим i на 1 и определим r_1 из условия:

$$(-1)^{r_1} = c^{r_0 2^{i+1}} b^{2^i}.$$

Продолжая этот процесс пока $i \geq 0$ вычислим все коэффициенты разложения r_i , а значит найдем решение уравнения $x^2 = a$.

В качестве примера рассмотрим вычисление квадратного корня из 3 по модулю 97. Убеждаемся, что $(\frac{3}{97}) = (\frac{97}{3}) = (\frac{1}{3}) = 1$, т.е. число 3 квадратичный вычет по модулю 97. Число 97 представляется как $97 = 1 + 2^5 * 3 \rightarrow l = 5, m = 3$. Убеждаемся, что число 5 не является квадратичным вычетом $(\frac{5}{97}) = (\frac{97}{5}) = (\frac{2}{5}) = -1$. Поэтому $5^3 = 28 \mod 97$ является корнем степени 32 - $28^{16} = -1 \mod 97$. Соответственно корень $\sqrt{3} \mod 97$ ищем в виде $28^k 3^{(m+1)/2} = 28^k * 9$. $a^m = 3^3 = 27 = b, b^2 = 50 \mod 97, b^4 = 75 \mod 97, b^8 = -1 \mod 97$. Это значит в поправочном коэффициенте $\epsilon = 28^k$, число k нечетное. Для $28^2 * 27 = 22 \mod 97$ получаем $22^2 = -1 \mod 97$, т.е. $\epsilon = 28^k, k = 1 + 4 \mod 8$. При $k = 5$ получаем $\epsilon^2 a^m = -1$, соответственно надо брать $\epsilon = 28^{13} = 42 \mod 97$ и $\sqrt{3} \mod 97 = \pm 42 * 9 = \pm 10$. Действительно $10^2 = 3 \mod 97$.

Для вычисления квадратного корня по модулю p^k при наличии корня по модулю $p > 2$ или по модулю 8 можно улучшать p - адическую точность корня, вводя улучшающие добавки $x_{i+1} = x_i + a_i * p^i$ и находя a_i из линейного соотношения $a - x_{i+1}^2 = 0 \mod p^{i+1} \rightarrow a_i = \frac{x_i^2 - a}{2x_i} \mod p, p > 2$. При $p = 2$ улучшение начинаем со степени 2^3 . Квадратный корень по составному модулю существует тогда и только тогда, когда существуют по модулю всех простых делителей и по модулю 8, если степень вхождения 2 не меньше 3. Корень вычисляется по всем модулям и по китайской теореме об остатках находится корень по составному модулю n . Количество решений (когда существует) $2^{\omega(n)}$, где $\omega(n)$ количество простых делителей. Когда n делится на 8 это количество решений меньше $n/2$ и столько же решений с добавлением $n/2$.

1.9 Замена сложной операции умножения

Операция умножения является более сложной операцией, чем сложение вычитание. Это относится как операции над небольшими числами до 2^{32} , так и для операции с большими числами или операции в конечномерных алгебрах.

Имеются разные методы ускорения вычисления произведения, которые будут рассмотрены в следующей главе. Здесь рассмотрим замену умножения более простыми операциями. Один из способов заключается в логарифмировании, т.е. в вычислении

$$a * b = \exp(\log a + \log b).$$

Здесь $\exp$ и $\log$ вычисляются по одному и тому же основанию, для операции над целыми числами обычно рассматривают по модулю степени простого числа. Согласно китайской теореме об остатках кольцо вычетов по модулю mn изоморфно прямой сумме колец:

$$Z_{mn} = Z_m \oplus Z_n, \quad (m, n) = 1.$$

Это позволяет ограничиться рассмотрением случаев, когда $n = q = p^k$ является степенью простого числа p . Легко показать, что группа обратимых элементов Z_q^* является циклической, когда $p > 2$, Z_2^* единичная группа, Z_4^* группа из двух элементов (циклическая), а группы $Z_{2^k}^*, k > 2$ не являются циклическими и имеют структуру $Z_2 \oplus Z_{2^{k-2}}$.

Метод логарифмирования не эффективен из-за сложности операций перехода, как логарифмирования, так и экспонирования. Тем не менее в некоторых задачах, когда необходимо использовать почти всю систему вычетов, такой метод становится эффективным. В ниже рассматриваемой задаче замена умножения по модулю простого числа сложением индексов привело к 12 кратному увеличению производительности.

Мах Алексеев проверил, что при $n \leq 45000$ выполняется соотношение:

$$\gcd(2^n + 1, n! - 1) = 1 \text{ or } 2n + 1.$$

Соответственно он предложил в форуме *dxdu* доказать, что это соотношение выполняется при любых n . В дальнейшем это предположение было отвергнуто и показано, что единственным исключением при $n \leq 225000$ является $n = 221799$, когда $\gcd$ равно простому числу $p = 521881$, отличному от $2n + 1$.

Назовем простое число p специальным по основанию a , если p делит одновременно $(\frac{p-1}{2})! - 1$ (2 - не является специальным простым при любом основании) и $a^{(p-1)/2} + 1$. Соответственно утверждение (гипотеза) сводилось к тому, что делителями $\gcd(a^n + 1, n! - 1)$ могут быть только специальные простые для основания $a = 2$. Точнее она дополнительно говорит о несуществовании специальных простых, для которых одновременно выполняются условия $p^2 | 2^{p-1} - 1$ и $p^2 | n! - 1$. Вначале охарактеризуем специальные простые числа.

Для целого числа a не делящегося на простое число p , обозначим через $\text{ord}_p(a)$ мультипликативный порядок числа a по модулю p или, что то же самое, порядок подгруппы, образованной элементом a в мультипликативной группе Z_p^* кольца вычетов Z_p . Здесь мы подробно изложили обозначение $\text{ord}_p(a)$ в связи с тем, что в некоторых книгах (см. [3]) таким образом обозначено то, что мы обозначили $v_p(a)$.

Условие $p | a^n + 1$ означает, что число $\frac{2n}{\text{ord}_p(a)}$ нечётное целое число. В частности, это означает, что число a квадратичный невычет для специальных простых по основанию a .

Для проверки того, что простое число p делит $\gcd(a^n + 1, n! - 1)$ при некотором n , необходимо и достаточно вычислить полупорядок $m = \frac{\text{ord}_p(a)}{2}$ и проверить делимость чисел $(km)! - 1$ на p для нечетных k не превосходящих $\frac{p-1}{m}$. Учитывая, что

$$(p - 1 - n)! = \frac{(p - 1)!}{(p - 1) \cdots (p - n)} \equiv (-1)^{n-1} (n!)^{-1} \pmod{p},$$

можно проверку чисел $(km)!$ по модулю p делать только для нечетных $k < \frac{p-1}{2m}$. При нечетном m из

$$p \mid \gcd(a^n + 1, n! - 1), n = km \text{ if and only if } p \mid \gcd(a^{p-1-n} + 1, (p-1-n)! - 1).$$

Это означает, что решения появляются парами (n, p) , $(p-1-n, p)$.

При четном m

$$p \mid \gcd(a^n + 1, n! + 1) \text{ if and only if } p \mid \gcd(a^{p-1-n} + 1, (p-1-n)! - 1).$$

Обозначим далее $c = \frac{p-1}{2}$ и $b \equiv c! \pmod{p}$. Тогда $(-1)^c b^2 + 1 \equiv 0 \pmod{p}$. Соответственно для специальности простого числа p необходимо, чтобы $p \equiv 3 \pmod{4}$ и a – квадратичный невычет. Однако, эти условия гарантируют только то, что p является общим делителем чисел $\gcd(a^n + 1, (n!)^2 - 1)$ при $n = c$. Так как $p \equiv 3 \pmod{4}$ означает, что -1 квадратичный невычет достаточно требовать, чтобы среди первых c чисел квадратичных невычетов было четное количество, соответственно квадратичных вычетов среди первых c натуральных чисел должно быть нечетное количество. Это характеризует специальные простые числа. В случае $a = 2$ примерно половина простых чисел вида $p \equiv 3 \pmod{8}$ являются специальными.

Простые делители $p \mid \gcd(a^n + 1, n! - 1)$ не являющиеся специальными простыми назовем исключительными. Примерный алгоритм их поиска описали выше. Однако, вычисление произведений чисел по модулю p довольно медленная операция. Поэтому, алгоритм был усовершенствован описанным ниже способом и вычисление ускорилось более 10 раз (при 32 разрядном коде). Для этого вначале вычисляется порядок $\text{ord}_p(a)$ за полиномиальное (от $\log(p)$) время. Полиномиальность получается из того, что при произведении простых чисел по решетку можно одновременно получать и простые делители числа $p-1$. Если порядок нечетное число или больше или равно $c = \frac{p-1}{2}$, то такое простое не может быть исключительным.

Вычисляем полупорядок m и $n_1 = \frac{p-1}{m}$. Заведем массив $X[k]$ для вычисления произведения чисел не превосходящих $(2k-1)m$, $2k-1 < n_1$. Находим порядок числа 2 (если a отлично от 2) и находим образующую g . Удобнее работать с таким образующим, что $g^{(p-1)/\text{ord}_p(2)} \equiv 2 \pmod{p}$, хотя это и не обязательно. Пробегаем половину всех вычетов по формуле $g^l 2^i$ для $l < d = \frac{p-1}{\text{ord}_p(2)}$. При этом каждый следующий вычет вычисляется умножением на 2, т.е. быстрой операцией сдвига. При этом, если предыдущий вычет был больше c , то вычитываем от полученного p . Если полученный вычет x меньше c , то тем $X[k]$, что $(2k-1)m \geq x$ прибавляем порядок $l + id$ в случае выбора соответствующего удобного образующего. Если $x > c = \frac{p-1}{2}$, то $X[k]$ сравниваем с $y = p - x$ и добавляем им порядок и вычитываем $(p-1)/2$. Иначе порядки представляем двумя числами. порядком по g и по 2 и суммируем по отдельности и приводим к одному основанию. Если m нечетное, то те k , для которых $p-1 \mid X[k]$ дают два исключительных значения $n = (2k-1)m$ and $n = p-1 - (2k-1)m$ для данного простого p . Если m четное, то те k , что $\frac{2X[k]}{p-1}$ нечетное целое дают решение $n = p-1 - (2k-1)m$, те, что дают четное целое приводят к решению $n = (2k-1)m$. Таким образом, получено 10 решений при $p < 225000000$ для основания $a = 2$. Четыре случая соответствуют тому же исключительному простому, только со значением $p-1-n$ (n - нечетное) и их не приводим в таблице. Вот они:

При этом обнаружилось любопытное свойство исключительных простых чисел. Когда исключительное значение индекса n нечетное выполняется $v_2(p-1) = 3$, когда исключительное значение индекса n четное, то $v_2(p-1) = 6$.

p	n
521881	221799
774593	338884
4327489	3652374
21764537	8161701
28807001	10802625
213833929	80187723

Таблица 1.1: p is prime, n is index

Здесь выигрыш достигнут за счет замены операции умножения по модулю простого числа. При умножении двух 32 разрядных чисел приходится прибегать к 64-разрядной форме, а затем вычислять остаток при делении на простое число. Последнее так же не быстрая операция. После замены операции приходится складывать только индексы и в случае превышения суммы числа $p - 1$ вычтем $p - 1$. Экспонировать не надо, проверяется равенство суммы индексов нулю или $(p - 1)/2$.

1.10 Числа Бернулли и их вычисление

Числа Бернулли первоначально были введены для выражения сумм степеней:

$$S_m(q) = \sum_{j=0}^{q-1} j^m = \frac{1}{m+1} (B_{m+1}(q) - B_{m+1}), \quad (0^0 = 1).$$

Здесь $B_{m+1}(x)$ многочлен Бернулли: $B_n(x) = \sum_{k=0}^n \binom{n}{k} B_k x^{n-k}$ записываемой часто в удобном (формальном) виде $B_n(x) = (B + x)^n$.

Числа Бернулли появляются во многих асимптотических формулах типа формул Эйлера-маклорена. Для выражения $n!$ эти формулы имеют вид:

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \exp\left(\sum_{i=1}^m \frac{B_{2i}}{2i(2i-1)n^{2i-1}} + O(n^{-2m-1})\right).$$

Значения дзета функции при четных натуральных значениях аргумента так же выражаются через числа Бернулли:

$$\zeta(2k) = \sum_{n=1}^{\infty} \frac{1}{n^{2k}} = (-1)^{k-1} \frac{(2\pi)^{2k}}{2(2k)!} B_{2k}.$$

Эти числа появились также у Куммера в доказательстве теоремы Ферма для регулярных простых чисел [1]. Числа Бернулли имеют многочисленные приложения.

Все алгоритмы для вычисления чисел Бернулли являются квадратичными. Могут отличаться на множители типа $O(\ln(m))$, но для вычисляемых на компьютерах значения $m \leq 10^9$ не так велики и такие множители могут быть компенсированы за счет уменьшения множителя константы или за счет преимущественного использования более быстрых операций как сложения или сдвига мантииссы вместо умножения. Здесь делается попытка

уменьшить константный множитель для ускорения вычислений. Рассмотрим вначале на соотношения (??) как на систему уравнений относительно чисел Бернулли в $q = p^n$ - ичной системе исчисления. Так как все числа Бернулли с нечётными номерами равны нулю, система выглядит переопределенной:

$$\sum_{i=0}^m \binom{m+1}{i} B_i q^{m+1-i} = (m+1)S_m(q) = (m+1) \sum_{j=0}^{q-1} j^m, \quad 0^0 = 1. \quad (1.1)$$

Учитывая, что

$$\log_q |B_k| = (k + \frac{1}{2}) \log_q(k) - (k - \frac{1}{2}) \log_q(2\pi) - \frac{k}{\ln q} + O(\frac{1}{k}) \quad (1.2)$$

для вычисления B_k в $q(q < k)$ ичной системе потребуется вычислять все B_i с некоторой точностью (в p -адической топологии), получаем, что количество необходимых операций кубична от k при попытке использования только одного p -адического основания. Поэтому, для эффективности вычислений потребуется использовать мультимодулярный алгоритм и для каждого $q = p^n$ (используемые простые числа p и степени для них $n = n(p)$ выберем позднее) вычисления с помощью (1.1) проведем с точностью до $q^l, l = l(p)$. При этом из (1.2) получаем ограничение на множество используемых оснований в мультимодулярном вычислении

$$\sum_p n(p) l(p) \ln p > \ln Q_k + (k + \frac{1}{2}) \ln(k) - (k - \frac{1}{2}) \ln(2\pi) - k. \quad (1.3)$$

При $l > 1$ приходится считать $l/2$ значений чисел Бернулли с разной точностью. Это приводит к $O(q l^3)$ операций для вычисления с точностью до q^l . При вычислении с $l = 1$ нам приходится вычислять l раз большим количеством оснований, которые преимущественно в l раз превосходят основания, при котором использовалась точность q^l для каждого основания. Т.е. количество операций меньше, чем при использовании l кратной точности с большим l . Эти соображения говорят, что использование точности $l > 2$ не выгодно. Вообще то наши соображения не совсем точны при малых q . Рассматривая только нечетные простые $p > 2$ легко видеть, что для прямых вычислений лучше обходиться с $n = 1$, т.е. $q = p$ и увеличить $l(p)$ в n раз. Однако, можно объединять элементы x в суммировании по вычетах по модулю $q = p^n$ дающие одинаковый остаток при делении на p . Соответственно, это приводит к экономии вычисления по большим степеням $q = p^n$. К тому же получаемые формулы при $q = p^n$ не отличаются особой сложностью от простого случая $q = p$. Соответственно, в дальнейшем $l(p) \leq 2$ и будут использоваться формулы второго порядка точности. От них при необходимости легко перейти к формулам одинарной точности. При этом не потребуется никакого вычисления B_{k-2} . Соответственно в основе нашего вычисления будет лежать формула:

$$B_k = \frac{1}{q} S'_k(q) - \frac{k(k-1)}{6} B_{k-2} q^2 + O(q^3). \quad (1.4)$$

Здесь под $S'_k(q)$ понимается соответствующая сумма k ых степеней с пропуском членов, делящихся на p , которые не дают вклада в нашу p -адическую аппроксимацию при $k > 1$. Соответственно, это приводит к задаче быстрого вычисления $S'_k(q)$ по модулю q^3 .

Пусть $(a, q) = 1$. Умножая $S'_k(q)$ на a^k и обозначая $r_x = ax - q[\frac{ax}{q}]$ получаем:

$$a^k S'_k(q) = \sum_{(x,p)=1}^{q-1} (r_x + [\frac{ax}{q}]q)^k = S'_k(q) + \sum_{j=1}^k \binom{k}{j} q^j \sum_{(x,p)=1}^{q-1} r_x^{k-j} [\frac{ax}{q}]^j.$$

Таким образом

$$(a^k - 1) \frac{S'_k(q)}{kq} = a^{k-1} \sum_{(x,p)=1}^{q-1} x^{k-1} [\frac{ax}{q}] - \frac{(k-1)qa^{k-2}}{2} \sum_{(x,p)=1}^{q-1} x^{k-2} [\frac{ax}{q}]^2 + O(q^2).$$

Точнее остаток $O(q^2)$ при $p > 3$, случаи $p = 2, 3$ требуют более точного вычисления. Обозначим через

$$C_k = \frac{B_k}{k}, \quad X_{a,j} = \sum_{\frac{ja}{a} < x < \frac{(j+1)q}{a}, (x,p)=1} x^{k-1}, \quad Y_{a,j} = \sum_{\frac{ja}{a} < x < \frac{(j+1)q}{a}, (x,p)=1} x^{k-2}.$$

Известно [1], что C_k является p целым для каждого p , $p-1 \nmid k$. Полученные соотношения можно записать в виде:

$$(e_a - a)C_k = - \sum_{j=1}^{a-1} jX_{a,j} + \frac{k-1}{2a}q \sum_{j=1}^{a-1} j^2Y_{a,j} + R. \quad (1.5)$$

Где $e_a = a^{1-k} \mod q^2$, $R = O(q^2)$, $p-1 \nmid k-2$, $R = (a^k - 1)\frac{(k-1)q^2}{6p} + O(q^2)$, $p \nmid a$, $p > 3$, Рассматривая отдельно случаи $p = 2, 3$ привлекая при этом дополнительные члены в разложении получаем: $R = \frac{k-1}{6}\frac{q^2}{p} + O(\frac{q^2}{p}) = O(\frac{q^2}{p^2})$.

Расчёт по формулам (1.5) (с $l = 2$) вполне эффективен даже при $p-1 \nmid k$, когда $q \leq 2^{16}$ для 32-разрядных машин и всегда ($k < 10^9$) для 64-разрядных машин. Для 32-разрядных компьютеров в случае $p > 2^{16}$ лучше брать $q = p$, ($k = 1$). Все эти ухищрения уже связаны не с количеством операций как таковой, а с реализованной скоростью выполнения операций. Однако, простых чисел с условием $p-1 \nmid k$ при вычислении чисел Бернулли с большими номерами k встречается ещё более незначительное количество. Соответственно лучше для таких чисел использовать только тот факт, что

$$B_k = \frac{V_k}{Q_k}, \quad Q_k = \prod_{p-1 \nmid k} p, \quad V_k = \frac{-Q_k}{p} \mod p : p-1 \nmid k.$$

Предварительно сделаем алгоритмы вычисления C_k более эффективным за счет разрежения сумм, используя разные значения a в (??) и комбинируя их между собой. Предварительно выводим необходимые соотношения для сумм

$$S_{k,a,j} = \sum_{\frac{ja}{a} < x < \frac{(j+1)q}{a}, (x,q)=1} x^k.$$

Переходя к дополнениям $q - x$ получаем соотношения симметрии:

$$S_{k,a,a-1-j} = \sum_x (q-x)^k = (-1)^k S_{k,a,j} + (-1)^{k-1} kq S_{k-1,a,j} + O(q^2).$$

С учетом четности k и точностью вычисления X, Y эти соотношения принимают вид:

$$X_{a,a-i-1} = -X_{a,i} + q(k-1)Y_{a,i}, \quad Y_{a,a-i-1} = Y_{a,i}, \quad \sum_{i < [a/2]} Y_{a,i} + (a \bmod 2)Y_{2a,a-1} = \delta, \quad (1.6)$$

где $\delta = \frac{p-1}{2}p^{n-1}, p-1 | k-2$, *else* $\delta = 0$. Используя эти соотношения, формулы можно привести к виду, когда используются только $X_{a,i}, Y_{a,i}$ с $i < [a/2]$.

$$(e_a - a)C_k = \sum_{j < a/2} (a-1-2j)X_{a,j} - \frac{q(k-1)}{2a} \sum_{j < [a/2]} (a^2-1-2j^2-2j)Y_{a,j} - \frac{q(k-1)(a^2-1)(a \bmod 2)}{4a} Y_{2a,a-1}. \quad (1.7)$$

Последний член появляется для нечетных значений a . Обычно пользуются упрощенным вариантом этой формулы, взятых по модулю p и в качестве a берут или примитивный корень по модулю p или $a = 2$. Однако, взяв комбинации этих формул при разных значениях a можно существенно сократить общую длину интервалов суммирования.

Взяв $c = ab$ выводим соотношения деления для сумм по интервалу с точностью до $O(q^2)$:

$$\begin{aligned} S_{k,a,i} &= \sum_{j=0}^{b-1} \sum_{q \frac{i+aj}{c} < x < q \frac{i+1+aj}{c}} (bx - jq)^k = b^k \sum_{j=0}^{b-1} S_{k,c,i+aj} - kqb^{k-1} \sum_{j=0}^{b-1} j S_{k-1,c,i+aj} = \\ &= b^k \sum_{j < \frac{b}{2}} S_{k,c,i+aj} + (-b)^k \sum_{1 \leq j \leq \frac{b}{2}} S_{k,c,aj-i-1} - kqb^{k-1} \sum_{j < \frac{b}{2}} S_{k-1,c,i+aj} - kq(-b)^{k-1} \sum_{1 \leq j \leq \frac{b}{2}} (b-j-1) S_{k-1,c,aj-i-1}. \end{aligned}$$

Учитывая, что $S_{k,a,i} = S_{k,c,bi} + S_{k,c,bi+1} + \dots + S_{k,c,bi+b-1}$ получаются соотношения на суммы $S_{k,c,j}$. Для переменных $X_{a,i}, Y_{a,i}$ соотношения деления запишутся в виде:

$$e_b X_{a,i} = \sum_{j < b/2} X_{c,i+jb} - \sum_{1 \leq j \leq \frac{b}{2}} X_{c,aj-i-1} - \frac{q(k-1)}{b} \left[\sum_{j < b/2} j Y_{c,i+ja} + \sum_{1 \leq j \leq \frac{b}{2}} (b-j-1) Y_{c,ja-i-1} \right], \quad (1.8)$$

$$be_b Y_{a,i} = \sum_{j < b/2} Y_{c,i+ja} + \sum_{1 \leq j \leq \frac{b}{2}} Y_{c,aj-i-1}, \quad e_b = b^{1-k}. \quad (1.9)$$

С учетом этих соотношений получаются несколько укороченные формулы:

Запишем отдельно формулу для случая $a = c = 2$, когда $Y_{2,0} = Y_{2,1} = \delta$ и формула приобретает простой вид:

$$(*2) \quad (e_2 - 2)C_k = X_{2,0} - \frac{3}{8}q(k-1)\delta.$$

Здесь и далее под $(*c)$ будем обозначать формулы, использующие части деления на c . При определении на регулярность (p, k) число $\delta = 0$, так как $k-2 < p$. В наших выражениях многие комбинации из $Y_{a,i}$ выражаются через δ так же как комбинации от $X_{a,i}$ вычисленные по модулю p выражаются через C_k .

Всего имеется $\sum_{1 < d < c, d|c} d$ соотношений деления. Однако, среди них могут быть зависимые. Например, для $c = 4$ соотношения деления не дают новых соотношений, кроме известных из симметрии. Соответственно, проще получать укороченные суммы взяв в

(1.7) линейные комбинации с делителями некоторого числа c . В дальнейшем мы несколько уменьшим точность вычислений до $R = O(\frac{q}{p^m})$, $m = 0$ если $p - 1 \nmid k - 2$, $m = 1$, если $p - 1 \mid k - 2$, $p \neq 3$ и $m = 2$, если $p = 3 \mid k - 2$ и в дальнейшем не будем писать члены указывающие точность формул. Вычитывая из формулы (1.7) для $a = 4$ случай $a = 2$ умноженное на 1 или 3 получаем:

$$(*4) \quad \frac{(e_2 + 1)(e_2 - 2)}{2} C_k = X_{4,0} - \frac{q(k-1)}{4} Y_{4,0},$$

$$\frac{(1 - e_2)(e_2 - 2)}{2} C_k = X_{4,1} - \frac{q(k-1)}{4} Y_{4,1}.$$

При $e_2 \not\equiv 2 \pmod{p}$, т.е. $2^k \not\equiv 1 \pmod{p}$, хотя бы одна из последних формул имеет коэффициент перед C_k не делящийся на p .

Между $Y_{6,i}$ имеются следующие соотношения: $Y_{6,1} = 2e_2 Y_{6,2}$, $Y_{6,0} = -(2e_2 + 1)Y_{6,2}$. Соответственно удобнее пользоваться формулами суммирования с одинаковыми интервалами суммирования для X, Y переменных. С учетом этих соотношений получаются формулы при $c = 6$:

$$\frac{e_6 - e_2 - e_3 - 1}{2} C_k = X_{6,0} - \frac{q(k-1)}{6} Y_{6,0},$$

$$(*6) \quad \frac{e_2 + 2e_3 - 2 - e_6}{2} C_k = X_{6,1} - \frac{q(k-1)(2e_2 - 1)}{12e_2} Y_{6,1},$$

$$\frac{2e_2 - 1 - e_3}{2} C_k = X_{6,2} - \frac{q(k-1)}{3} Y_{6,2}.$$

Учитывая, что $X_{12,2} = X_{4,0} - X_{6,0}$, $X_{12,3} = X_{3,0} - X_{4,0}$ получаем формулы с короткими суммами при $c = 12$:

$$X_{12,2} = \frac{(e_2 - 1)(e_2 + 1 - e_3)}{2} C_k + \frac{q(k-1)}{24e_2} [(4e_2 - 1)Y_{12,2} - (2e_2 + 1)Y_{12,3}],$$

$$(*12) \quad X_{12,3} = \frac{e_3 + e_2 - 1 - e_4}{2} C_k + \frac{q(k-1)}{24e_2} [(6e_2 - 1)Y_{12,2} - Y_{12,3}].$$

При $c = 30$ получается наиболее короткая формула:

$$(*30) \quad X_{30,9} - (e_2 + 1)X_{30,5} = e_2 \frac{e_6 - e_5 + 1 - e_2}{2} C_k + \frac{q(k-1)}{180e_3} [(4e_4 + 6e_2)Y_{30,4} +$$

$$+ (4e_4 + 6e_2 - 30e_6 - 27e_3)Y_{30,5} + (2e_3 + 3)Y_{30,7} + (6e_6 + 63e_3)Y_{30,9}].$$

В последних формулах интервалы суммирования для Y в два раза длиннее, чем для X . Возможно существуют более короткие выражения для сумм Y . Здесь не сделан полный анализ такой возможности. Заметим, что во всех таких формулах коэффициент перед C_k обращается в 0, если все используемые e_i приравнять к i . Это непосредственно получается из (1.5). Однако, в приведенных формулах коэффициент обращается в 0 и в случае, когда все e_i приравнены единице. Не трудно заметить, что в формулах (*12) оба коэффициента перед C_k обращаются в 0 по модулю p , тогда и только тогда, когда $e_2 = e_3 = 1 \pmod{p}$ или $e_2 = 2 \pmod{p}$, $e_3 = 3 \pmod{p}$. Если это имеет место для всех формул (*12), (*30), то или $e_2 = e_3 = e_5 = 1 \pmod{p}$ или $e_2 = 2 \pmod{p}$, $e_3 = 3 \pmod{p}$, $e_5 = 5 \pmod{p}$. При заданном k

такие простые встречаются достаточно редко. Поэтому эти формулы очень удобны при вычислении чисел Бернулли по модулю p^2 . Приведём ещё пару аналогичных формул.

$$(*42) \frac{e_7 - e_6 - e_2 + 1}{2} C_m = -X_{42,6} \left(1 + \frac{1}{e_2} + \frac{1}{e_3}\right) + \frac{1}{e_3} X_{42,7} + \frac{1}{e_2} X_{42,14} - \frac{1}{e_3} X_{42,20} + O(q).$$

Здесь (разреженность 10.5) и в следующей формуле (разреженность 12) пропущены громоздкие члены с $Y_{c,i}$.

$$(*132) \frac{e_{12}(e_{11} - e_6 - e_4 - 1)}{2} C_m = X_{132,0} + (1 + e_2 + e_3 + e_4 + e_6) X_{132,11} + X_{132,22} + (1 + e_2) X_{132,33} + \\ + (1 + e_3) X_{132,44} + (1 + e_4 + e_6) X_{132,55} - (1 + e_2 + e_3 + e_6) X_{132,54} - X_{132,43} - \\ - (1 + e_3 + e_4) X_{132,32} - (1 + e_3) X_{132,21} - (1 + e_2) X_{132,10} + O(q).$$

Взяв $c = 2 \prod_i p_i$, где p_i различные простые числа (то, что в разложении c простое число 2 может войти во второй степени связано с симметрией относительно дополнения), можно уменьшить количество членов в суммировании и далее. Однако, при этом быстро растёт количество возможных интервалов суммирования и эти более разреженные формулы становятся не эффективными, из-за необходимости проверять принадлежность тому или иному интервалу, количество которых может стать порядка самих вычетов и из-за усложнения вычисления коэффициентов перед ними. Отметим, что вычисления по формуле (*12) занимает всего на 4–5 % времени больше, чем по формуле (*30) а не на 25 %. Это связано с тем, что кроме умножений производятся и другие операции и тем что в формулах с большим количеством интервалов побочных операций больше. По всей видимости (*30), (*12) являются самыми экономными, т.е. дальнейшее уменьшение количества операций умножения не приводит сокращению времени счета из-за увеличения количества проверок на принадлежность нужному интервалу. На практике достаточно использовать приведенные формулы сокращающие длину интервала суммирования не менее 10 раз. Приведенные выше формулы имеют второй порядок точности, когда $p - 1 \nmid k$, $p - 1 \nmid k - 2$. Использование высших степеней в модулярных формулах фактический не дает выигрыша. Дело в том, что вычисление величин под суммой примерно квадратично от степени простого числа, по модулю которого рассматриваются суммы. Выигрыш при вычислении с точностью второго порядка точности достигается только при использовании 64-битной арифметики в 64-битных процессорах в компьютере. При этом этот выигрыш достигается только если процессор выполняет операцию умножения 64-битных чисел по модулю 64-битного числа примерно за то же время, что и 32-битных чисел. Даже в этом случае выигрыш оказывается не больше, чем в $\frac{4}{3}$ раза. На практике эти операции выполняются медленнее, соответственно использование двойной точности не дает (ощутимого) преимущества. Тем не менее, в других задачах, как вычисление B_m для иррегулярной пары (m, p) возможно использование этих формул для проверки на делимость на дополнительную степень простого числа p .

1.10.1 Алгоритм, использующий разреженные суммы для вычисления чисел Бернулли

Анализ показал, что использование формул высоких порядков точности по простым числам не являются эффективными. Поэтому здесь ограничимся формулами первой степени

точности. При модулярном вычислении чисел Бернулли B_k достаточно использовать только удобные простые числа, для которых работает хотя бы одна из формул:

$$B_k = \frac{2k}{(e_2 - 1)(e_2 + 1 - e_3)} \sum_{\frac{p}{6} < x < \frac{p}{4}} x^{k-1}, \quad (1.10)$$

$$B_k = \frac{2k}{e_3 + e_2 - 1 - e_4} \sum_{\frac{p}{4} < x < \frac{p}{3}} x^{k-1}. \quad (1.11)$$

$$B_k = \frac{2k}{e_2(e_6 - e_5 + 1 - e_2)} \left[\sum_{\frac{3p}{10} < x < \frac{p}{3}} x^{k-1} - (e_2 + 1) \sum_{\frac{p}{6} < x < \frac{p}{5}} x^{k-1} \right]. \quad (1.12)$$

При этом проводим вычисления по простым числам, пока их произведение (оцениваем по их логарифмам) не превзойдет числитель числа Бернулли. Исключать из-за неудобства приходится только небольшое (в пределах 10 для заданного индекса числа Бернулли) количество простых чисел, когда или $e_2 = e_3 = e_5 = 1$ или $e_2 = 2, e_3 = 3, e_5 = 5$.

При получении простых чисел p через решето, можно одновременно получить простые делители чисел $p - 1$. Последнее дают преимущество в нахождении мультипликативной образующей g и вычислении e_2, e_3, e_5 по модулю p . Числа $e_4 = e_2^2, e_6 = e_2 * e_3 \mod p$. По этим числам определяем, можем ли пользоваться самой экономной формулой ($e_5 + e_2 \neq e_6 + 1$) или одной из формул (1.10), (1.11) или пропускаем данное простое значение.

Вычислять для каждого вычета x степень x^{k-1} не эффективно. Более эффективно пробежать по вычетам $x = g^i 2^j, i = 0, 1, \dots, r - 1, j = 0, 1, \dots, \text{ord}_p(2) - 1, r = \frac{p-1}{\text{ord}_p(2)}$. Вычисление произведения двух 32-ух разрядных чисел по модулю 32-ух разрядного простого числа требует использование 64-х разрядной арифметики для умножения и более чем в 10 раз медленнее двух операций - сдвига $x \ll 1$ (умножения на 2) и в случае превышения p вычета $\text{if}(x \geq p) \ x = x - p$.

На самом деле достаточно пробежать по половине всех вычетов. Если $\text{ord}_p(2)$ нечетно вычеты искомого вычета получается из двойного цикла, внешний по всем $i = 0, 1, \dots, r/2 - 1$, внутренний по $j = 0, 1, \dots, \text{ord}_p(2) - 1$. Иначе внешний по всем $i = 0, 1, \dots, r - 1$, а внутренний по $j = 0, 1, \dots, \text{ord}_p(2)/2 - 1$. Тогда для каждого вычета или x или $p - x = x * g^{(p-1)/2}$ появится среди образовавшихся в цикле чисел $g^i 2^j \mod p$. Обозначим через $y = g^{i(k-1)} \mod p$ и через $z = 2^{k-1}$. Во внутреннем цикле образовываем значения проверяемых аргументов по команде $x \ll 1; \text{if}(x > p) x = x - p$; (p - нечетное). Если $x \leq p/2$, и попадает в интервал вычисления, то вычисляя $t = x^{k-1}$ прибавляем полученное значение переменной, обозначающей сумму степеней по модулю p . Если $x > p/2$, и $x_1 = p - x$ попадает в интервал вычисления, то вычисляя $t = x^{k-1}$ вычитываем из суммы степеней. Значение t в алгоритме Харвея [10] вычисляется каждый раз по команде $t * = z; t \% = p$ во внутреннем цикле, а во внешнем задается $t = y$. За счет этого уменьшение длины цикла вдвое приводит примерно вдвое большей производительности у Харвея. В нашем случае вычисление t производится только при попадании x или $p - x$ в нужный интервал, что происходит примерно в 7 раз реже. Попадание в интервал проверяется по известным (вычисленным заранее) значениям концов интервалов или по формуле $k = [ax/p]$ $a = 30$ или $a = 12$ в зависимости от используемой формулы. Так как бывают большое количество пропусков при вычислении t , предварительно вычислим значения $z[1], \dots, z[29], z1[1], z1[2], \dots, z1[30]$, определенные как $z[i] = z^i \mod p, z1[i] = z^{30i}$. Во внутреннем цикле считаем сколько

пропущено, добавляя каждый раз $j1+ = 1$. Если вычет попал в нужный интервал этот счетчик обнуляется $j1 = 0$, он обнуляется и во внешнем цикле. Новое значение t считается по формуле: $t* = z[j1] \bmod p$, если $j1 < 30$ и $t* = z[j1 \% 30] \bmod p$; $t* = z1[j1/30] \bmod p$, если $30 \leq j1 < 900$. В случае, если $j1 \geq 900$ то предварительно умножаем на $z1[30]$ в $\lceil \frac{j1}{900} \rceil$ раз и сводим к случаю $j1 < 900, j1 \% = 900$. На самом деле последний случай вряд ли встретится. Хотя разреженность пробега по вычетам для половины вычетов не превосходит 7.5 встречаются случаи пропусков даже $j1 > 200$. При этом значение t находится только одним умножением в подавляющем большинстве случаев. За счет этого вычисление B_k по модулю простого числа получается быстрее в 3 – 4 раза, чем у Харвея. Так как в этом алгоритме именно эта часть занимает не менее 99% процессорного времени, общее вычисление так же в 3-4 раза быстрее, соответственно по сравнению с пакетом Математика более чем в 30 раз для $k > 10^6$.

Можно несколько увеличить скорость вычисления в случае, когда $\gcd(k-1, p-1)$ большое число. Однако, оценка показывает, что увеличение производительности незначительное. Попытка увеличить производительность вычисления за счет выбора только таких простых, когда $\gcd(k-1, p-1)$ большое число, так же не приводит к успеху. Во первых таких простых мало, что приводит к увеличению используемых простых. С ростом самих используемых простых количество используемых операций пропорционально увеличиваются.

Этот алгоритм вычисления B_k по модулю простого числа можно использовать для поиска чисел Вольстенхолма.

По полученным (p, b_p) , $b_p = V_k \bmod p$, $V_k = Q_k(-1)^{1+k/2} B_k$ вычисляем числитель V_k числа Бернулли B_k по методу бинарного дерева. Вначале объединяем на пары $(a_1, b_1), (a_2, b_2)$ модули и находим пару (A, B) , где $A = a_1 * a_2, B = (a_1 * (a_1^{-1} \bmod a_2) * b_2 + a_2(a_2^{-1} \bmod a_1) * b_1) \bmod A$. Оставшаяся без пары модуль переносится на следующий уровень. На нижнем уровне порядка k простых модулей, в следующем примерно в два раза меньше модулей, однако они занимают примерно в два раза больше бит в своей двоичной записи и т.д. Используя алгоритмы быстрого умножения больших чисел получаем, что на вычисление V_k по их вычетам по множеству простых чисел уходит примерно $\log k$ раз больше операций, чем на последнем шаге, т.е. всего $O(k \log^2 k \log \log k)$ операций. При больших k это значительно меньше, чем количество операций при вычислении по модулю всех простых до $p_{max} \sim k \log k$.

ЛИТЕРАТУРА

1. Борович Шафаревич. Теория чисел.
2. Kenneth Ireland and Michael Rosen. A classical introduction to modern number theory, 2 ed., Graduate Texts in Mathematics, vol. 84, Springer-Verlag, 1990.
3. S. Chowla and P. Hartung. An "exact" formula for the m-th Bernoulli number, Acta Arith, 22 (1972), 113-115.
4. Sandra Fillebrown. Faster computation of Bernoulli numbers. J. Algorithms 13 (1992), no. 3, 431-445.
5. Greg Fee and Simon Plouffe. An efficient algorithm for the computation of Bernoulli numbers, 2007, preprint arXiv: math. NT/0702300v2.
6. Oleksandr Pavlyk. Today We Broke the Bernoulli Record: From the Analytical Engine to Mathematica, 2008, posted on Wolfram Blog (<http://blog.wolfram.com/>) on 29th April 2008, retrieved 15th June 2008.
7. Wolfram Research. Inc., Mathematica 6.0, 2007, <http://www.wolfram.com/>

8. William Stein. Modular forms, a computational approach. Graduate Studies in Mathematics, vol. 79, American Mathematical Society, Providence, RI, 2007, With an appendix by Paul E. Gunnells.
9. PARI/GP, version 2.7.5, 2014, available from <http://pari.math.u-bordeaux.fr/download/html>
- 10 David Harvey. A Multimodular algorithm for computing Bernoulli numbers. arXiv:0807.1347v2 [math.NT] 13 Oct. 2008.

Глава 2

Умножение больших чисел и матриц

2.1 Методы фильтрации и градуировки

Метод фильтрации использует разбиение размерности задачи n на l частей, далее еще на l частей и т.д. Доходя до единичного уровня и производя необходимые вычисления, начинаем использовать эти вычисления для уровня на ступень выше. При этом вычисления на нижнем уровне, не попадающие в подмножество нижнего уровня именно для этой фильтрованной размерности никак не используются на уровне чуть выше. Например, при бинарном дереве вычислений ($l = 2$). Из высшего уровня попадаем вначале на 2 задачи уровня ниже 0, 1 размерности $n/2$. Каждая из этих задач разбиваются на задачи еще нижнего уровня 00, 01, 10, 11 размерности $n/4$. При этом вычисления на уровнях 10, 11 не используются на уровне выше в разделе задачи 0 размерности $n/2$. Поэтому, вычисления на единицу размерности всегда выходят экспоненциально зависящими от $\log(n)$ (обычно в формуле мастер теоремы $m > l$).

При вычислении методом градуировки мы используем все уровни $(i_1, \dots, i_k)$ как градуировки и промежуточные вычисления в нижних уровнях сказывается (используется) на всех уровнях выше. Это приводит к количеству операций на один размер задачи как линейно или полиномиально зависящей величины от количества уровней $k = \log(n)$. Продемонстрируем это на примерах. В задаче сортировки мы сортируем вначале по биту, соответствующему знаку мантиссы отделяя положительные числа от отрицательных чисел. В следующей стадии разделяем числа по знаку порядка. Далее идем по битам порядка от старшего к младшим. Часть чисел уже на этом этапе получать номера по числу чисел из массива, которые меньше заданного числа. Далее продолжаем по битам мантиссы, осуществляя проверки только у тех множеств чисел, которые дали одинаковые значения по предыдущим проверкам, т.е. так и не отделились друг от друга. Так получим алгоритм, с минимально возможным числом побитовых операций. Эти операции можно выполнять параллельно, так как идет сравнение каждого числа только по одному биту, пока число не отделилось от других в своем ящике.

В задаче сортировки отличия методов фильтрации и градуирования не существенны, так как достаточно отличать числа по высшему разряду и только в случае одинаковости перейти к низшему. В задачах умножения больших чисел и больших матриц отличие более существенно.

Большие числа на компьютере представляются в M – ичной системе исчисления как

значения многочлена:

$$X = x_0 + x_1M + \dots + x_{k-1}M^{k-1}.$$

M есть степень двойки, обычно $M = 2^d$, $d = 32$. Считаем, что $0 \leq x_i < M$. Соответственно $X < M^k = 2^n$, $n = kd$

Допустим имеется еще другое большое число $Y = y_0 + \dots + y_{l-1}M^{l-1}$ и требуется их умножить. Произведение не превышает M^m , $m = k + l$, и оно представляется в виде:

$$Z = XY = \sum_{i=0}^{m-1} z_i M^i, \quad z_i = \sum_{j \leq i} x_j y_{i-j}.$$

Правда, здесь после вычисления надо осуществить переносы, когда цифры становятся не меньше M . Для этого складываем перенос из нижнего уровня и делим на M , остаток от деления будет настоящей цифрой этого уровня, а целая часть от деления пойдет в следующий уровень как перенос.

Количество операций при таком вычислении равно $O(m^2)$, если не учесть увеличение вычисляемых цифр произведения до переносов. На самом деле из-за роста длины представления цифр примерно в $\log m$ раз количество операций будет как минимум $O(n^2 \log n)$, $n = md$.

Более эффективный метод умножения больших чисел впервые использовал А.А. Карацуба в 1962 г. Он использовал метод фильтрации (разделяй и властвуй), разделяя умножаемые числа на композицию двух поменьше:

$$X = X_0 + X_1 * N, \quad N = 2^R, \quad Y = Y_0 + Y_1 * N.$$

Здесь X_0, Y_0 числа, состоящие из младших разрядов, X_1, Y_1 из старших разрядов. Результат умножения представляется в виде:

$$XY = X_0 Y_0 + N(X_0 Y_1 + X_1 Y_0) + X_1 Y_1 N^2.$$

Среднее число вычисляется одним дополнительным умножением, используя результаты умножений крайних:

$$X_0 Y_1 + X_1 Y_0 = (X_0 + X_1)(Y_0 + Y_1) - X_0 Y_0 - X_1 Y_1.$$

Оказывается, еще Гаусс заметил, что при вычислении умножения комплексных чисел можно обойтись тремя умножениями, используя два умножения, полученные при вычислении действительной части:

$$(a + bi)(c + di) = ac - bd + (bc + ad)i,$$

$$bc + ad = (a + b)(c + d) - ac - bd.$$

На этом основании некоторые зарубежные авторы не ссылаются на Карацуба, упоминая на приведенное замечание Гаусса.

Таким образом, принцип разделяй и властвуй для умножения двух n битных чисел, требующих $T(n)$ операций приводит к рекуррентному соотношению:

$$T(n) = 3T\left(\frac{n}{2}\right) + Cn.$$

Решая эту рекурсию получаем:

$$T(n) = C(2^k + 3 * 2^{k-1} + \dots + 3^k) = C(3^{k+1} - 2^{k+1}) = O(n^{\log 3}). \quad (2.1)$$

В соответствии с приведенной мастер теоремой принципа разделяй и властвуй, число операций при умножении методом Карацуба оценивается величиной $O(n^{\log 3})$. Мы условились, что логарифм без указания основания означает логарифм по основанию 2. В зарубежной литературе часто такое употребление означает натуральный логарифм, для которого у нас есть более короткое обозначение $\ln$.

Впоследствии в 1969 г. Штрассеном найден аналог такого умножения и для матриц. Для этого разобьем матрицы X, Y размером $n \times n$ на подматрицы половинной размерности:

$$X = \begin{pmatrix} A & B \\ C & D \end{pmatrix}, \quad Y = \begin{pmatrix} E & F \\ G & H \end{pmatrix}, \quad XY = \begin{pmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{pmatrix}.$$

Вводя 7 произведений:

$$P_1 = A(F - H), \quad P_2 = (A + B)H, \quad P_3 = (C + D)E,$$

$$P_4 = D(G - E), \quad P_5 = (A + D)(E + H),$$

$$P_6 = (B - D)(G + H), \quad P_7 = (A - C)(E + F),$$

произведение матриц можно представить в виде:

$$XY = \begin{pmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{pmatrix}.$$

Это приводит к рекуррентному соотношению относительно необходимого количества операций для умножения матриц размера $n \times n$:

$$T(n) = 7T\left(\frac{n}{2}\right) + \frac{9}{2}n^2.$$

Последний член складывается из 10 сложений вычитаний матриц порядка $\frac{n}{2}$ и 8 сложений вычитаний после умножений матриц такого порядка. Здесь не учли операции с индексами массивов, необходимых для выбора элементов матриц, используемых для сложения вычитания при вычислении линейных функционалов. Эти операции более быстры по сравнению с операциями сложения, вычитания, в особенности в случае $n = 2^k$.

Решая рекуррентное соотношение получаем

$$T(n) = O(n^{\log 7}), \quad \log 7 \approx 2.81.$$

Точнее, без учета операций с адресами элементов при $n = 2^k$:

$$T(2^k) = \frac{9}{2}4^k + 7T(2^{k-1}) = \frac{9}{2}(4^k + 4^{k-1} * 7) + 7^2T(2^{k-2}) = \dots = \frac{9}{2}(4^k + 4^{k-1} * 7 + \dots + 4 * 7^{k-1}) + 7^k.$$

Вычисляя сумму геометрической прогрессии получаем

$$T(2^k) = 6(7^k - 4^k) + 7^k. \quad (2.2)$$

Здесь 7^k - количество умножений в формате чисел, $6(7^k - 4^k)$ количество сложений и вычитаний в формате чисел. Количество операций над адресами массивов чуть больше и примерно соответствует количеству операций сложения, вычитания. Эти операции выполняются быстрее их, поэтому пренебрежение ими не существенно влияет на время счета. Учитывая, что операции сложения, вычитания выполняются быстрее, чем умножение прием сложность вычисления по алгоритму Штрассена за $2n^{\log 7}$, а сложность при стандартном умножении n^3 . Тогда выигрыш алгоритма при $n = 2^k$ по отношению к стандартному умножению будет

$$f(k) = 2^{3k - k \log 7 - 1} \approx 2^{0.19k - 1}, \quad f(10) \approx 1.866, \quad f(15) \approx 3.6, \quad f(20) \approx 6.96$$

Когда n немногим больше степени 2 выигрыша может не оказаться.

Отметим, что как в случае фильтрации, так и в случае градуировки мы размерность (степень многочлена) делим на малые порядки. Пусть произведение многочленов имеет степень меньше $N = p_1 p_2 \dots p_k$. Удобнее вычислять в случае разбиения $p_i = 2$. Два числа представляются в виде многочлена степени меньше N -

$$f(x) = \sum_{i=0}^{N-1} a_i x^i, \quad \phi(x) = \sum_{i=0}^{N-1} b_i x^i.$$

Сами многочлены на высшем уровне представляются многочленами степени меньше p_k от переменной $n_k = x^{N/p_k}$ с коэффициентами, являющимися многочленами степени меньше n_k (точнее степень произведения не превосходит это число). В дальнейшем, на ступень ниже эти коэффициенты рассматриваются сами как многочлены степени меньше p_{k-1} с коэффициентами в виде многочленов степени меньше $n_{k-1} = n_k/p_{k-1}$. Произведение многочленов на каждой ступени вычисляются через значения. Отличие проявляются только в том, что вычисления на нижних уровнях используется только на одном отцовском уровне в методе фильтрации, в то время как в случае градуировки все промежуточные вычисленные значения используются на всех уровнях.

Оценим количество операций при вычислении произведения многочленов степени меньше p через значения в точках $x = x_i, i = 1, 2, \dots, 2p-1$ (произведение имеет степень меньше $2p-1$). Пусть

$$P(x) = (x - x_1) \dots (x - x_{2p-1}), \quad P_i(x) = \frac{P(x)}{(x - x_i)P'(x_i)} = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

Тогда произведение, являющееся многочленом степени меньше p представляется через значения:

$$\psi(x) = \sum_{i=1}^{2p-1} f(x_i) \phi(x_i) P_i(x).$$

Тут существенную роль играет то обстоятельство, что умножение вычисляется в факторкольце кольца многочленов по идеалу порожденному многочленом $P(x)$, а многочлены $P_i(x)$ являются ортогональными проекторами:

$$P_i(x) P_j(x) = \delta_i^j P_i(x).$$

Вычисление значений в p точках имеет сложность порядка $O(p^2)$ за исключением специальных случаев, когда x_i являются корнями из 1. Поэтому, не выгодно использовать большие p для промежуточных вычислений. В то же время общее количество умножений и операций при методе фильтрации оценивается величиной $O(pn^{\log_p(2p-1)})$, которая уменьшается при росте p вплоть до значений порядка $\log(n)$.

В методе Карацуба все $p_i = p = 2, 2p - 1 = 3$ и значения вычисляются в целых точках $0, 1, 1/0$. В последнем случае, под значением подразумевается старший коэффициент произведения, аналогично нулевому значению. Можно улучшить проводя вычисления для случаев $p = 3, 4, 5$ используя значения в точках $0, 1/0, \pm 1, \pm 2, \pm \frac{1}{2}, \pm 3$.

Градуированный метод вычислений произведения больших чисел рассмотрим в следующем параграфе.

2.2 Проекторы и значения при $n = 2$

Метод Штрассена обоснован на разложении сомножителей по базису проекторов. Прежде чем перейти к более эффективному градуированному методу умножения матриц рассмотрим аналог умножения по Карацуба для матриц. Суть фильтрационного метода умножения матриц A, B заключается в нахождении такой формулы:

$$A * B = \sum_{i=1}^r C_i f_i(A) \phi_i(B) \quad (2.3)$$

для матриц порядка $m \times m$ с как можно меньшим значением $r = r(m)$. Здесь C_i матрицы-константы, указывающие места, куда с некоторым коэффициентом входит произведения линейных функционалов $f_i(A) \phi_i(B)$. Минимально возможное значение $r = r(m)$ называется рангом умножения для матриц порядка $m \times m$. Мастер теорема дает, что умножение для матриц порядка $n \times n$, $n = m^k$ можно вычислить за $n^{\log_m r(m)}$ умножений значений функционалов и не более чем $km^2 n^{\log_m r(m)}$ сложений, вычитаний. Отметим, что такой алгоритм умножения требует еще эффективное вычисление функционалов. Без этого алгоритм менее эффективен даже алгоритма стандартного умножения при $k \leq 3$. Поэтому, даже найденные минимальные экспоненты умножения $\alpha = \log_m r(m)$, $m = 256$ бесполезны для практического применения в расчетах. Далее рассмотрим проекторы и функционалы, приводящие к формулам Штрассена.

Под проекторами понимаются операторы p , удовлетворяющие условию $p^2 = p$. В алгебре с 1 это означает $p(1 - p) = 0$. Считаем, что алгебра с 1 задана над некоторым полем. Условию $p(1 - p) = 0$ удовлетворяют так же 1 (единица алгебры) и 0 и сопряженный проектор $1 - p = \bar{p}$. Проекторы 1 и 0 называем тривиальными операторами - проекторами. Несколько фактов о проекторах:

Лемма 1. *Если p проектор, то $1 - p = \bar{p}$ проектор, $1 - 2p = 2\bar{p} - 1$ квадратный корень из 1. Если $p, ar + b$ различные проекторы, то или один из них тривиальный проектор, или они сопряженные проекторы.*

Доказательство. Первое утверждение доказывается непосредственной проверкой:

$$(1 - p)^2 = 1 - 2p + p^2 = 1 - p, \quad (1 - 2p)^2 = 1 - 4p + 4p^2 = 1.$$

Пусть p не тривиальный проектор и $(ap + b)^2 = (ap + b)$. Тогда

$$(a^2 + 2ab)p + b^2 = ap + b \Rightarrow a^2 + 2ab = a, \quad b^2 = b \Rightarrow b = 0 \text{ or } b = 1.$$

Если $b = 0$ число $a = 0$ ($ap + b = 0$ - тривиальный проектор) или $a = 1$ $ap + b = p$. Это противоречит условию леммы. Если $b = 1$, то $a^2 = -a$ или $a = 0$ ($ap + b = 1$ - тривиальный проектор) или $a = -1$ ($ap + b = 1 - p = \bar{p}$). Это доказывает первую часть леммы. $\square$

Два проектора p, q назовем антикоммутирующими, если $(1 - 2p)(1 - 2q) = -(1 - 2q)(1 - 2p)$.

Лемма 2. Если p, q антикоммутирующие проекторы, то $(\bar{q}, p), (\bar{p}, \bar{q}), (q, \bar{p})$ так же антикоммутирующие пары проекторов. Если p, q различные, не тривиальные, не сопряженные антикоммутирующие проекторы, то элементы $p, q, \bar{p}, \bar{q}$ порождают 4-х мерную подалгебру. В качестве базиса можно принять 1 и проекторы $p, q, 2\bar{q}p$.

Доказательство.

$$(1 - 2\bar{q})(1 - 2p) = -(1 - 2q)(1 - 2p) = (1 - 2p)(1 - 2q) = -(1 - 2p)(1 - 2\bar{q}).$$

Переходы $(p, q) \Rightarrow (\bar{q}, p) \Rightarrow (\bar{p}, \bar{q}) \Rightarrow (q, \bar{p}) \Rightarrow (p, q)$ доказывают первую часть. Из условия леммы $1, p, q$ линейно независимы. Пусть $r = 2pq$. Тогда, из антикоммутируемости p, q получаем, что

$$(1 - 2p)(1 - 2q) = -(1 - 2q)(1 - 2p) \Rightarrow 1 - 2p - 2q + 2qp = -2pq \Rightarrow 2(1 - q)(1 - p) = 1 - 2pq \Rightarrow 2\bar{q}\bar{p} = \bar{r}.$$

Очевидно, что $r(1 - r) = r\bar{r} = 2pq2\bar{q}\bar{p} = 0$. Аналогично, $2p\bar{q}, 2q\bar{p}$ являются сопряженными проекторами. Обозначив $2pq$ через r , получаем

$$2p\bar{q} = 2p - r, \quad 2q\bar{p} = 1 - 2p + r, \quad 2\bar{q}\bar{p} = 1 - r = \bar{r}.$$

Очевидно $pr = r, rp = p(2q\bar{p}) = p(1 - 2p\bar{q}) = p, rp = rpp = pp = p$. и если $r = a + bp + 2cq$, то $a + bp + 2cq = r = pr = (a + b)p + cr$. Если $c \neq 1$, то будучи проектором и линейной комбинацией $1, p$ должен или сопряженным или тривиальным, или совпадать с p . Так как это не выполняется $c = 1, a = b = 0$. В этом случае $r = 2q = 2pq \Rightarrow q = 0$ - противоречие с условием. Значит r не является линейной комбинацией $1, p, q$. В произведениях r с другими не появляются выражения в виде трех сомножителей:

$$r = pr, \quad rp = p, \quad \bar{p}r = 0, \quad r\bar{q} = 0, \quad r\bar{p} = r - p, \quad \bar{q}r = r - q.$$

Это доказывает лемму. $\square$

Лемма 3. Если p, q антикоммутирующие проекторы ($x = 1 - 2p, y = 1 - 2q, xy = -yx$) и $\varepsilon_1, \varepsilon_2, \varepsilon_3$ такие числа, что $\varepsilon_1^2 + \varepsilon_2^2 = 1 + \varepsilon_3^2$, то $f = \frac{1 + \varepsilon_1 x + \varepsilon_2 y + \varepsilon_3 xy}{2}$ проектор. Все нетривиальные проекторы имеют такой вид.

Доказательство. Так как $x^2 = (1 - 2p)^2 = 1 = (1 - 2q)^2 = y^2$ и $xy = -yx$, переменная $z = xy$ антикоммутирует с переменными x, y . Здесь переменным x, y, z соответствуют матрицы:

$$x = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad y = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad z = xy = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}.$$

Действительно

$$y = x(xy) = -x(yx) = -(xy)x, \quad x = x(yu) = (xy)y = -(yx)y = -y(xy).$$

Вычисляем $z^2 = xyxy = x(yx)y = -x(xy)y = -x^2y^2 = -1$. Вычисляем f^2 :

$$f^2 = \frac{1 + 2\varepsilon_1x + 2\varepsilon_2y + \varepsilon_3z + \varepsilon_1^2 + \varepsilon_2^2 - \varepsilon_3^2}{4} + \frac{\varepsilon_1\varepsilon_2(xy + yx) + \varepsilon_1\varepsilon_3(xz + zx) + \varepsilon_2\varepsilon_3(yz + zy)}{4} = f.$$

Это доказывает лемму. $\square$

В бигрупповой алгебре группы Z_2 , являющейся алгеброй квадратных матриц второго порядка два различных не тривиальных и не сопряженных проектора p, q и сопряженные к ним $\bar{p}, \bar{q}$ образовали бы самосопряженную систему. Однако, они не образуют базис, так как $p + \bar{p} = 1 = q + \bar{q}$. Среди всех проекторов несколько особым случаем являются проекторы, соответствующие диагональным матрицам. Соответственно, на диагонали 1 или 0. Можно выбрать n проекторов p_i у которых 1 стоит на пересечении i -ой строки с i -м столбцом, а все остальные элементы равны 0. Любая матрица строка или матрица столбец q диагональный элемент которой равен 1, так же является проектором. Так приходим к выбору проекторов из семейства:

$$p = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}, \bar{p} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, e = \begin{pmatrix} 1 & \theta_1 \\ 0 & 0 \end{pmatrix}, t = \begin{pmatrix} 0 & 0 \\ \theta_2 & 1 \end{pmatrix}, \bar{e} = \begin{pmatrix} 0 & -\theta_1 \\ 0 & 1 \end{pmatrix}, \bar{t} = \begin{pmatrix} 1 & 0 \\ -\theta_2 & 0 \end{pmatrix}.$$

Естественное требование $e\bar{t} = 0 = t\bar{e}$ приводит к соотношению $\theta_1\theta_2 = 1$. Соответственно, удобное (целочисленное) решение приводит к выбору $\theta_1 = \theta_2 = \theta = \pm 1$. За один базис берем систему из $1, e, t$ и p или $\bar{p}$, в сопряженном базисе будут 1 и сопряженные факторы. Все эти базисы приводят к вычислению одного и того же набора функционалов с точностью до перестановок номеров и изменений знаков.

В качестве базиса для первого множителя берем

$$1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, p = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}, e = p2q = \begin{pmatrix} 0 & 0 \\ -1 & 1 \end{pmatrix}, t = \bar{p}2q = \begin{pmatrix} 1 & -1 \\ 0 & 0 \end{pmatrix}, 2q = 1 - p.$$

Легко проверить, что p, e, t являются проекторами и их дополнениями являются следующие проекторы:

$$\bar{p} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \bar{e} = 2\bar{q}\bar{p} = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}, \bar{t} = 2\bar{q}p = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}.$$

Выразим первый множитель через $1, p, e, t$, второй через $1, \bar{p}, \bar{e}, \bar{t}$:

$$\begin{aligned} A &= \begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} = \frac{a_{00} + a_{11}}{2} + \frac{a_{00} - a_{11}}{2}x + \frac{a_{01} + a_{10}}{2}y + \frac{a_{01} - a_{10}}{2}xy = \\ &= a_{00} + a_{01} + (a_{11} + a_{10} - a_{00} - a_{01})p - a_{10}e - a_{01}t. \\ B &= \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{pmatrix} = \frac{b_{00} + b_{11}}{2} + \frac{b_{00} - b_{11}}{2}x + \frac{b_{01} + b_{10}}{2}y + \frac{b_{01} - b_{10}}{2}xy = \\ &= b_{11} - b_{01} + (b_{00} + b_{01} - b_{11} - b_{10})\bar{p} + b_{10}\bar{e} + b_{01}\bar{t}. \end{aligned}$$

Ненулевыми произведениями перемножаемых проекторов являются умножения на 1 и:

$$p\bar{e} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} = -e\bar{p} = \bar{e} - \bar{p} = p - e, \quad p\bar{t} = p, \quad t\bar{p} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} = \bar{p}. \quad (2.4)$$

Таким образом,

$$t\bar{p} + p\bar{t} = 1, \quad e\bar{p} + p\bar{e} = 0.$$

Запишем перемножаемые матрицы в виде:

$$A = (a_{00} + a_{01})\bar{p} + (a_{11} + a_{10})p - a_{10}e - a_{01}t,$$

$$B = (b_{00} - b_{10})\bar{p} + (b_{11} - b_{01})p + b_{10}\bar{e} + b_{01}\bar{t}.$$

Каждый коэффициент в разложении A является значением $A(i_1, i_2, i_3) = A|_{p=i_1, e=i_2, t=i_3}$, где i_1, i_2, i_3 равны нулю или единице. Заметим, что

$$A(i_1, i_2, i_3) + A(1 - i_1, 1 - i_2, 1 - i_3) = \text{tr}(A). \quad (2.5)$$

Аналогично для значений матрицы B .

Вычислим произведение, учитывая только не нулевые произведения:

$$\begin{aligned} C = AB &= -a_{10}(b_{00} - b_{10})(e - p) - a_{01}(b_{11} - b_{01})(t - \bar{p}) + (a_{11} + a_{10})b_{10}(\bar{e} - \bar{p}) + (a_{00} + a_{01})b_{01}(\bar{t} - p) + \\ &\quad a_{10}(b_{01} - b_{11})p + a_{01}(b_{10} - b_{00})\bar{p} + (a_{00} + a_{01})b_{00}\bar{p} + (a_{11} + a_{10})b_{11}p = \\ &\quad -a_{10}b_{00}(e - p) + a_{11}b_{10}(\bar{e} - \bar{p}) - a_{01}b_{11}(t - \bar{p}) + a_{00}b_{01}(\bar{t} - p) + \\ &\quad (a_{10}b_{01} + a_{11}b_{11})p + (a_{00}b_{00} + a_{01}b_{10})\bar{p} = \\ &= (a_{10}b_{00} + a_{11}b_{10})(\bar{e} - \bar{p}) + (a_{01}b_{11} + a_{00}b_{01})(\bar{t} - p) + (a_{10}b_{01} + a_{11}b_{11})p + (a_{00}b_{00} + a_{01}b_{10})\bar{p} = \\ &\quad -(a_{11} + a_{10})b_{00}e + a_{11}(b_{10} - b_{00})\bar{e} + a_{11}b_{00} - (a_{00} + a_{01})b_{11}t + a_{00}(b_{01} - b_{11})\bar{t} + a_{00}b_{11} + \\ &\quad (a_{10}b_{01} + a_{11}b_{11} + a_{10}b_{00} - a_{00}b_{01})p + (a_{00}b_{00} + a_{01}b_{10} + a_{01}b_{11} - a_{11}b_{00})\bar{p} = \\ &= (\text{tr}(A)\text{tr}(B) - (a_{11} + a_{10})b_{00}e - a_{11}(b_{00} - b_{10})\bar{e} - (a_{00} + a_{01})b_{11}t - a_{00}(b_{11} - b_{01})\bar{t} + \\ &\quad + (a_{10} - a_{00})(b_{01} + b_{00})p + (a_{01} - a_{11})(b_{10} + b_{11})\bar{p} = \\ &= f^7\phi^7 - f^5\phi^6p - f^1\phi^2\bar{p} - f^2\phi^4e - f^3\phi^5\bar{e} - f^6\phi^3t - f^4\phi^1\bar{t}. \end{aligned}$$

Здесь из 8 пар произведений получили 7 произведений. Таким образом, произведение матриц:

$$A = f^2p + f^6\bar{p} + (f^3 - f^2)e + (f^2 - f^1)t, \quad B = \phi^1p + \phi^5\bar{p} + (\phi^2 - \phi^3)\bar{e} + (\phi^1 - \phi^2)\bar{t}$$

равно

$$AB = f^7\phi^7 - f^5\phi^6p - f^1\phi^2\bar{p} - f^2\phi^4e - f^3\phi^5\bar{e} - f^6\phi^3t - f^4\phi^1\bar{t}.$$

Вычислим значения для произведения матриц:

$$C(0, 0, 0) = \text{tr}(A)\text{tr}(B) - f(-1, -1, -1)\phi(-1, 1) - f(-1, 0)\phi(1, -1, 1) - f(1, 0)\phi(-1, -1, -1).$$

Здесь третий аргумент указан как значение xy , когда оно не совпадает с произведением значений x, y . Аналогично получаются другие значения:

$$C(0, 1, 1) = \text{tr}(A)\text{tr}(B) - A(1, 1, 1)B(1, 0, 0) - A(1, 0, 0)B(0, 0, 1) - A(0, 0, 0)B(1, 1, 0),$$

$$C(1, 1, 0) = \text{tr}(A)\text{tr}(B) - A(0, 1, 1)B(0, 0, 0) - A(1, 0, 0)B(0, 0, 1) - A(0, 0, 1)B(1, 1, 1),$$

$$C(1, 1, 1) = \text{tr}(A)\text{tr}(B) - A(0, 1, 1)B(0, 0, 0) - A(1, 0, 0)B(0, 0, 1) - A(0, 0, 0)B(1, 1, 0).$$

Выразим функционалы как значения многочленов $f(x, y, z)$ от x, y, z .

$$f^7 = 2f_0 = f^i + f^{7-i}, f_0 = f(0, 0, 0), f^1(-1, -1, -1), f^6 = f(1, 1, 1),$$

$$f^2 = f(-1, 1, -1), f^5 = f(1, -1, 1), f^3 = f(-1, 0, 0), f^4 = f(1, 0, 0).$$

Значения произведения через значения сомножителей выражаются по формулам:

$$\psi^1 = f^7\phi^7 - f^2\phi^4 - f^6\phi^3 - f^5\phi^6,$$

$$\psi^2 = f^7\phi^7 - f^3\phi^5 - f^4\phi^1 - f^5\phi^6,$$

$$\psi^3 = f^7\phi^7 - f^2\phi^4 - f^4\phi^1 - f^5\phi^6,$$

$$\psi^4 = f^7\phi^7 - f^3\phi^5 - f^6\phi^3 - f^1\phi^2.$$

Эти формулы

$$f^i + f^{7-i} = f^7, \quad i = 1, 2, 3, 4, 5, 6. \quad (2.6)$$

Когда номер i у f^i степень двойки (1,2,4) он входит в произведение $f^i\phi^{2i}$ (номера вычисляются по модулю 7), иначе входит в произведение $f^{2i}\phi^i$. Кроме того для значения ψ^i в одном из произведений встречается f^{7-i} , сумма других двух индексов равно 7. Аналогично с индексами для ϕ . Точнее:

$$\psi^i = f^7\phi^7 - f^{-i}\phi^{-4i} - f^{2i}\phi^{4i} - f^{-2i}\phi^{-i}, \quad \text{if } \left(\frac{i}{7}\right) = 1, \quad (2.7)$$

и

$$\psi^i = f^7\phi^7 - f^{-i}\phi^{-2i} - f^{4i}\phi^{2i} - f^{-4i}\phi^{-i}, \quad \text{if } \left(\frac{i}{7}\right) = -1. \quad (2.8)$$

Используя (2.6) можно избавиться от переменных с индексом 7 в этих формулах несколькими способами заменив f^7 на $f^j + f^{-j}$:

$$\psi^i = (f^{2i} + f^{-2i})\phi^7 - f^{2i}\phi^{4i} - f^{-2i}\phi^{-i} - f^{-i}\phi^{-4i} = (f^{2i} - f^{-i})\phi^{-4i} + f^{-2i}\phi^i \quad \left(\frac{i}{7}\right) = 1.$$

$$\psi^i = (f^{4i} + f^{-4i})\phi^7 - f^{4i}\phi^{2i} - f^{-4i}\phi^{-i} - f^{-i}\phi^{-2i} = (f^{4i} - f^{-i})\phi^{-2i} + f^{-4i}\phi^i \quad \left(\frac{i}{7}\right) = -1.$$

$$\psi^i = f^7(\phi^{-4i} + \phi^{4i}) - f^{-i}\phi^{-4i} - f^{2i}\phi^{4i} - f^{-2i}\phi^{-i} = f^i\phi^{-4i} + f^{-2i}(\phi^{4i} - \phi^{-i}) \quad \left(\frac{i}{7}\right) = 1.$$

$$\psi^i = f^7(\phi^{-2i} + \phi^{2i}) - f^{-i}\phi^{-2i} - f^{4i}\phi^{2i} - f^{-4i}\phi^{-i} = f^i\phi^{-2i} + f^{-4i}(\phi^{2i} - \phi^{-i}) \quad \left(\frac{i}{7}\right) = -1.$$

$$\begin{aligned}\psi^i &= (f^i + f^{-i})\phi^7 - f^{-i}\phi^{-4i} - f^{2i}\phi^{4i} - f^{-2i}\phi^{-i} = f^i\phi^7 + f^{-i}\phi^{4i} - f^{2i}\phi^{4i} - f^{-2i}\phi^{-i} = \\ &= f^i\phi^i + (f^i - f^{-2i})\phi^{-i} + (f^{-i} - f^{2i})\phi^{4i} = f^i\phi^i - (f^i - f^{-2i})(\phi^i - \phi^{-4i}), \quad \left(\frac{i}{7}\right) = 1.\end{aligned}$$

Аналогично

$$\psi^i = f^i\phi^i - (f^i - f^{-4i})(\phi^i - \phi^{-2i}), \quad \left(\frac{i}{7}\right) = -1.$$

Запишем последние (более симметричные) формулы для индексов 1,2,3,4,5,6.

$$\psi^1 = f^1\phi^1 - f_3\phi_2, \quad \psi^6 = f^6\phi^6 - f_2\phi_3, \quad (2.9)$$

$$\psi^2 = f^2\phi^2 - f_1\phi_3, \quad \psi^5 = f^5\phi^5 - f_3\phi_1, \quad (2.10)$$

$$\psi^3 = f^3\phi^3 - f_1\phi_2, \quad \psi^4 = f^4\phi^4 - f_2\phi_1. \quad (2.11)$$

Здесь ввели новые 3 переменных (индексов), где

$$f_1 = f^2 - f^3, \quad f_2 = f^4 - f^6, \quad f_3 = f^1 - f^5.$$

Хотя в этих формулах 8 произведений (для 4-х достаточных формул), они удобны как поправки к первым произведениям, соответствующим коммутативному умножению с одинаковыми индексами.

Симметрий несколько больше, если ввести новые функционалы

$$f'^1 = f^1 - f_0 = \frac{1}{2}(f^1 - f^6), \quad f'^2 = f^2 - f_0 = \frac{1}{2}(f^2 - f^5), \quad f'^3 = f^4 - f_0 = \frac{1}{2}(f^4 - f^3).$$

При этом симметрия, связанная умножением индексов на 2 по модулю 7 обратиться в симметрию циклической перестановки индексов $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$. Индексам 3, 5, 6 соответствуют функционалы:

$$f^3 = f_0 - f'^3, \quad f^5 = f_0 - f'^2, \quad f^6 = f_0 - f'^1.$$

Вычислим вначале ψ_0 :

$$\begin{aligned}\psi_0 &= \frac{1}{2}(\psi^3 + \psi^4) = 4f_0\phi_0 - \frac{1}{2}(f^3\phi^5 + f^1\phi^2 + f^6\phi^3 + f^4\phi^1 + f^5\phi^6 + f^2\phi^4) = \\ &= f_0\phi_0 - \frac{1}{2}[f'^1(\phi'^2 + \phi'^3) + f'^2(\phi'^3 + \phi'^1) + f'^3(\phi'^1 + \phi'^2)] = f_0\phi_0 - \frac{1}{2}(f'^1\phi_1 + f'^2\phi_2 + f'^3\phi_3).\end{aligned}$$

Это выражение симметрично относительно замены $f \rightarrow \phi \rightarrow f$. Найдём соотношения между переменными с нижними и верхними индексами:

$$\begin{aligned}f_1 &= f'^2 + f'^3, \quad f_2 = f'^3 + f'^1, \quad f_3 = f'^1 + f'^2, \\ f_1 + f_2 + f_3 &= 2(f'^1 + f'^2 + f'^3), \\ f'^1 &= \frac{f_2 + f_3 - f_1}{2}, \quad f'^2 = \frac{f_3 + f_1 - f_2}{2}, \quad f'^3 = \frac{f_1 + f_2 - f_3}{2}.\end{aligned}$$

Эти формулы проверены непосредственно вычисляя значения для многочленов:

$$\begin{aligned} f &= a_0 + a_1x + a_2y + a_3xy, \quad \phi = b_0 + b_1x + b_2y + b_3xy, \\ \psi &= a_0b_0 + a_1b_1 + a_2b_2 - a_3b_3 + (a_0b_1 + a_1b_0 - a_2b_3 + a_3b_2)x + \\ &\quad (a_0b_2 + a_1b_3 + a_2b_0 - a_3b_1)y + (a_0b_3 + a_1b_2 - a_2b_1 + a_3b_0)xy. \end{aligned}$$

Коэффициенты a_i через коэффициенты матриц выражаются по формулам:

$$a_0 = \frac{a_{00} + a_{11}}{2}, a_1 = \frac{a_{00} - a_{11}}{2}, a_2 = \frac{a_{01} + a_{10}}{2}, a_3 = \frac{a_{01} - a_{10}}{2}.$$

Значения (f_0, f^1, f^2, f^4) , (f_0, f_1, f_2, f_3) и (f'_0, f'_1, f'_2, f'_4) представляют полную систему значений. Для доказательства их полноты вычислим их через коэффициенты a_0, a_1, a_2, a_3 :

$$f_0 = a_0 = \frac{a_{00} + a_{11}}{2}, f^1 = a_0 - a_1 - a_2 - a_3 = a_{11} - a_{01}, f^2 = a_0 - a_1 + a_2 - a_3 = a_{11} + a_{10}, f^4 = a_0 + a_1 = a_{00}.$$

следовательно $a_1 = f^4 - f_0, a_2 = \frac{1}{2}(f^2 - f^1), a_3 = 2f_0 - f^4 - \frac{1}{2}(f^1 + f^2)$. Другие наборы значений так же полные. Другие функционалы:

$$\begin{aligned} f^7 &= a_{00} + a_{11}, f^3 = f^7 - f^4 = a_{11}, f^5 = a_{00} - a_{10}, f^6 = a_{00} + a_{01}, \\ f_1 &= f^2 - f^3 = a_{10}, f_2 = f^4 - f^6 = -a_{01}, f_3 = f^1 - f^5 = a_{11} + a_{10} - a_{00} - a_{01}. \end{aligned}$$

Отметим, что матрица A^* , у которой действительная часть $\frac{1}{2}f^7(A^*)$ совпадает с действительной частью исходной матрицы

$$\frac{1}{2}f^7(A), A = \begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix},$$

а мнимые части имеют противоположный знак $f^i(A^*) = f^{7-i}(A)$ называются сопряженными. Из определения получаем: $A^* = tr(A)E - A = \begin{pmatrix} a_{11} & -a_{01} \\ -a_{10} & a_{00} \end{pmatrix}$. Произведение матрицы со своим сопряженным дает определитель

$$A^*A = AA^* = (a_{00}a_{11} - a_{01}a_{10})E.$$

Замена матриц на сопряженные является нечетным автоморфизмом второго порядка (симметрией):

$$C = AB \rightarrow C^* = B^*A^*.$$

Алгоритм умножения при $n = 2$ будет выглядеть так. Вначале вычисляем значения перемножаемых матриц как с верхними, так и нижними индексами. Далее, воспользовавшись формулами (2.18), (2.19), (2.20) вычисляем $\psi^1, \psi^2, \psi^3, \psi^4$ и получим коэффициенты произведения:

$$c_{00} = \psi^4, c_{11} = \psi^3, c_{01} = \psi^3 - \psi^1, c_{10} = \psi^2 - \psi^3.$$

Заметим, что количество попарных произведений в вычислении одного значения можно уменьшить до двух так же за счет линейной комбинации значений произведения:

$$\psi^1 - \psi^3 = f^4\phi^1 - f^6\phi^3, \quad \psi^2 - \psi^3 = f^2\phi^4 - f^3\phi^5.$$

Для полноты исследований случая $n = 2$ покажем, что меньше чем с семью умножениями не удастся вычислить произведение двух матриц порядка 2×2 . Пусть $j(i) = i * 2^{(i)} \bmod 7$. Покажем, что использованные билинейные функционалы $f^i \phi^{j(i)}$ линейно независимы. Пусть $f^i, i = 7, 1, 2, 4$ линейно независимые функционалы (можно было выбрать любые 4 за исключением двух пар $f^i, f^{7-i}, f^j, f^{7-j}$) и X_i такие операторы, что $f^i(X_j) = \delta_j^i$. Аналогично, пусть Y_l такие операторы, что $\phi^{j(i)}(Y_l) = \delta_l^i$. Семь билинейных функционалов $f^i \phi^{j(i)}, i = 1, 2, \dots, 7$ полностью определяются со своими 16 ю значениями $f^i(X_j) \phi^{j(i)}(Y_l)$ как векторы в 16-мерном пространстве. Несложно проверяется, что эти вектора не являются линейно зависимыми. Произведение любых двух операторов $A * B$ определяется через значения, разлагаемые в линейную комбинацию указанных билинейных функционалов $f^i(A) \phi^{j(i)}(B)$. Рассмотрим теперь разложение 16 произведений $X_i * X_j$, определяемых 64 значениями на значения семи указанных выше билинейных функционалов. Если существует умножение меньше чем с семью умножениями, то семь векторов, состоящих из 64 компонент, будут линейно зависимыми и это приводит к противоречию с отсутствием линейной зависимости между нашими билинейными функционалами.

2.3 Преобразование Фурье и умножение больших чисел

Более эффективный (градуированный) метод умножения основывается на быстром преобразовании Фурье. Большие числа можно представить как значения многочлена

$$P(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}, \quad Q(x) = b_0 + b_1x + \dots + b_{n-1}x^{n-1}.$$

Для удобства можем считать, что многочлены имеют одинаковую степень, заполняя, при необходимости, нулями старшие коэффициенты. На самом деле мы нулями заполним до той поры, чтобы и их произведение имело не более n значащих цифр:

$$R(x) = c_0 + c_1x + \dots + c_{n-1}x^{n-1}, \quad c_k = \sum_{i+j=k} a_i b_j.$$

Так приходим к умножению в конечномерном пространстве

$$R(x) = P(x) * Q(x) = P(x) * Q(x) \bmod \prod_i (x - x_i), \quad i = 1, \dots, n.$$

При стандартном вычислении произведение многочленов вычисляется через $(n-1)^2$ сложений и n^2 умножений. Здесь не учитываем переносы и рост цифр (коэффициентов многочлена) и рост сложности каждой операции над коэффициентами как в $\log \log n$ раз за счет увеличения разрядности на $\log n$ битов.

Многочлены степени меньше n над полем нулевой характеристики полностью определяются значениями в n точках по формуле:

$$f(x) = \sum_i f(x_i) P_i(x), \quad P_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}, \quad P_i(x) P_j(x) = \delta_i^j P_i(x). \quad (2.12)$$

Поэтому произведение двух многочленов, степень произведения которых не выше $n-1$, можно вычислить через произведения их значений в n точках и последующим восстановлением по данным значениям многочлена по указанной формуле.

Обозначим через F линейное отображение, сопоставляющее многочлену степени ниже n элемент (вектор) $(P(x_0), P(x_1), \dots, P(x_{n-1}))$ из K^n , называемое дискретным преобразованием Фурье. Тогда, если произведение многочленов P, Q имеет степень меньше n , то

$$P(x) * Q(x) = F^{-1}(F(P(x))F(Q(x))). \quad (2.13)$$

Произведение справа является произведением функций и вычисляется просто, как и сумма логарифмов. Для эффективного вычисления необходимо подобрать x_i , когда можно быстро вычислить преобразование Фурье $F(P), F(Q)$ и обратное преобразование. При не больших значениях степеней умножаемых многочленов k можно считать $n = 2k - 1$ (не учитывая переносы) и вычислять значения многочленов при не больших целых точках или обратных к целым. Всего в наборе должно быть $2k - 1$ точек.

Оказывается вычисление по формулам (2.12), (2.13) можно сделать эффективно, когда x_i образуют группу корней из 1 степени n , а n раскладывается на малые множители как степень двойки. В таком случае, многочлены можно рассматривать как элементы градуированной алгебры, представляемой групповой алгеброй $K(G)$ группы корней из 1. Так как умножение разных степеней коммутативно и образует циклическую группу, группа G так же циклическая порядка n . Вычисление значений соответствует распространению характера $\mu : G \rightarrow K^*$, $g \rightarrow \mu(g)$, $g \in G$ как линейного оператора на групповой алгебре. При этом значение характера на произведении элементов групповой алгебры соответствует произведению значений. В дальнейшем считаем, что $n = |G| = |G^*|$ обратим в кольце коэффициентов. Для каждого из n характеров определим элементы

$$t_\mu = \frac{1}{n} \sum_{g \in G} \mu(g)g.$$

Характеры можно применять и к этим n различным элементам элементам:

$$\mu_1(t_\mu) = \frac{1}{n} \sum_{g \in G} \mu(g)\mu_1(g)g = t_{\mu\mu_1}.$$

Отметим, что если $g \in G$ не единичный элемент, то существует такой характер μ , что $\mu(g) \neq 1$. Следовательно,

$$S = \sum_{\mu \in G^*} \mu(g) = \sum_{\mu\mu_1 \in G} \mu(g)\mu_1(g) = \mu(g) \sum_{\mu_1 \in G^*} \mu_1(g) = \mu(g)S \rightarrow S = 0.$$

Так как элементы группы g выступают как характеры для группы характеров G^* , мы доказали довольно часто встречающуюся лемму:

Лемма 4. Пусть G конечная коммутативная группа порядка n , изоморфная своей группе характеров. Тогда между характерами и элементами группы имеется соотношение:

$$\sum_{\mu \in G^*} \mu(g) = \begin{cases} |G|, & g = e, \\ 0, & g \neq e. \end{cases}$$

$$\sum_{g \in G} \mu(g) = \begin{cases} |G|, & \mu = e, \\ 0, & \mu \neq e. \end{cases}$$

Как следствие получаем, что если в кольце имеются примитивные корни степени n и $n = |G|$ обратимый элемент, то $t_\mu, \mu \in G^*$ образуют базис групповой алгебры. При этом

$$\sum_{\mu \in G^*} \mu(g^{-1})t_\mu = \frac{1}{n} \sum_{\mu \in G^*} \sum_{g_1 \in G} \mu(g^{-1}g_1)g_1 = g. \quad (2.14)$$

Отметим, что как следствие второго соотношения леммы получается ортогональность базиса t_μ :

$$t_\mu t_{\mu_1} = \frac{1}{n^2} \sum_{g, g_1} \mu(g)\mu_1(g_1)gg_1 = \frac{1}{n} \sum_g \mu(g)\mu_1(g^{-1})t_{\mu_1} = \begin{cases} t_\mu, \mu = \mu_1, \\ 0, \mu \neq \mu_1. \end{cases}$$

Пусть многочлен представляется (задан) как элемент групповой алгебры $f = \sum_g a(g)g$. Используя (2.5) получаем представление в базисе t_μ

$$f = \sum_g a(g)g = \sum_\mu t_\mu \sum_g a(g)\mu(g^{-1}) = \sum_\mu \bar{a}(\mu)t_\mu.$$

Таким образом, коэффициенты многочлена в базисе t_{μ_i} получаются

$$\bar{a}_\mu = \sum_g \mu(g^{-1})a(g). \quad (2.15)$$

Аналогично, обратный переход

$$f = \sum_\mu \bar{a}(\mu)t_\mu = \frac{1}{n} \sum_{\mu, g} \bar{a}(\mu)\mu(g)g = \sum_g a(g)g.$$

Таким образом,

$$a(g) = \frac{1}{n} \sum_\mu \bar{a}(\mu)\mu(g). \quad (2.16)$$

Следовательно, преобразование Фурье есть вычисление коэффициентов представления в базисе t_{μ_i} , как вычисление значений многочлена заменяя переменную $g \rightarrow \mu(g^{-1})$ на соответствующий корень из 1. Обратное преобразование Фурье получается вычислением значений многочлена $\sum_\mu \bar{a}(\mu)t_\mu$ заменяя переменные t_μ на $\mu(g)$ и поделив результат на n .

При умножении больших чисел меньше M^l , $M = 2^d$ большие числа и их произведение представляются многочленами с коэффициентами из Z_+ :

$$f = \sum_{i=0}^{l-1} a_i * M^i, \phi = \sum_{i=0}^{l-1} b_i M^i, \psi = \sum_{k=0}^{2l-2} c_k M^k, 0 \leq a_i, b_i \leq M-1, c_k = \sum_{i+j=k} a_i b_j, 0 \leq c_k \leq l(M-1)^2.$$

В полукольце Z_+ нет корней степени n . Поэтому коэффициенты вычисляем по модулю нескольких простых чисел p_1, p_2, p_3 , таких, что $n|p_i - 1$. Эти условия гарантируют существование циклической группы корней из 1 порядка n в кольцах (полях) Z_{p_i} . Чтобы однозначно восстановить коэффициенты c_k по их образам (вычетам) в указанных кольцах, должно быть $l(M-1)^2 < \prod_i p_i$. Среди простых чисел меньше 2^{32} имеется два числа (Ноден. Алгебраическая арифметика) для которых $2^{26}|p_i - 1$. Однако, если брать $n = 2^{26}$, $l = 2^{25}$

получаем ограничение на $M = 2^d \leq \lceil \sqrt{\frac{p_1 p_2}{2^{25}}} \rceil + 1$. Из-за этого ограничения используют три простых числа, взяв $n = 2^{25} |p_i - 1|$. Так можно вычислить произведение чисел представляемых более чем 3 миллиардами двоичных цифр (битов). Представление коэффициентов в таких кольцах проводится модулярное вычисление как при преобразовании Фурье множителей, так и при обратном преобразовании Фурье для произведения.

При вычислении значений используются автоморфизмы групповой алгебры, что позволяет сократить количество операций. Покажем, как уменьшается количество операций вычисления n значений многочлена $f(x_i)$, когда x_i являются различными корнями из 1 в случае, когда n разлагается на малые множители $n = p_1 p_2 \dots p_k$.

Пусть

$$X = (x_{i,j}), Y = (y_{i,j}), x_{i,j} = x_i^j, y_{i,j} = x_j^{-i}, i = 0, 1, \dots, n-1, j = 0, 1, \dots, n-1$$

матрицы. Между коэффициентами многочлена и значениями имеется связь:

$$X * (a_0, \dots, a_{n-1})^T = (P(x_0), P(x_1), \dots, P(x_{n-1}))^T. \quad (2.17)$$

Обозначим через $Z = (z_{i,j})$ произведение матриц XY . По построению диагональные элементы равны n и

$$z_{i,j} = \frac{1 - (\frac{x_i}{x_j})^n}{1 - \frac{x_i}{x_j}}, i \neq j.$$

Если отношение корней является корнем степени n и все они различны, то Y является обратной матрицей с точностью до множителя n . Если $x_k = x_1^k = \exp(\frac{2\pi i k}{n})$, то матрицы X, Y симметричны и комплексно сопряжены. Матрица Y соответствует матрице X , где корни расставлены в другом порядке $y_k x_k = 1$. Это сводит задачу обратного преобразования Фурье к преобразованию Фурье с обратным x_1^{-1} вместо x_1 и нормировке с множителем $\frac{1}{n}$. Остается эффективно вычислить значения многочлена в корнях из 1. Пусть $n = pm$. Тогда вычисление n значений на корнях степени n для многочлена:

$$f(x) = a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$$

сводится к вычислению m значений для p многочленов степени $m = \frac{n}{p}$:

$$f_0(x^p) = a_0 + a_p x^p + \dots + a_{(m-1)p} x^{(m-1)p},$$

$$f_1(x^p) = a_1 + a_{p+1} x^p + \dots + a_{(m-1)p+1} x^{(m-1)p},$$

.....,

$$f_{p-1}(x^p) = a_{p-1} + a_{2p-1} x^p + \dots + a_{(m-1)p+p-1} x^{(m-1)p}.$$

Значения нашего многочлена выражаются через эти вычисленные значения формулой:

$$f(x) = f_0(x^p) + x f_1(x^p) + \dots + x^{p-1} f_{p-1}(x^p).$$

Поэтому нам потребуется дополнительно складывать $(p-1)n$ раз и умножать примерно столько же раз (на самом деле чуть меньше, при вычислении случая $x = 1$ обходимся без

умножения). Таким образом, количество операций для преобразования Фурье на n точках удовлетворяет рекуррентному соотношению

$$T(n) = pT\left(\frac{n}{p}\right) + (p-1)n. \quad (2.18)$$

Когда все множители малые это приводит к $T(n) = O(n \log n)$ операций. Здесь опять не учли множитель $\log \log n$, связанный с увеличением разрядности коэффициентов.

Если принять, формулу (2.9) точным, как для операции сложения, так и для операций произведения, то для преобразования Фурье с $n = p_1 p_2 \dots p_k$ точками требуется выполнить N умножений и N сложений, где

$$N = p_1 p_2 \dots p_k (p_1 + p_2 + \dots + p_k - k) = n(p_1 + p_2 + \dots + p_k - k).$$

Как уже было сказано, количество произведений несколько меньше этого. Для достижения минимального значения $N(n)$ надо найти минимум этого выражения при условии $p_1 p_2 \dots p_k \geq n$. Например для $n = 45$ число $N = 360$ меньше, чем в случае $p_i, k = 6, n = 64, N(64) = 384$. Если $n = 81$ число $N = 648$ меньше чем $N(96) = 672 < 896 = N(128)$.

Заметим, что метод Карацуба эффективнее табличного умножения уже для чисел, когда множители имеют более 64 бит. В то же время, метод Кули-Тьюки эффективнее метода Карацуба только для множителей начиная с чисел, когда произведение в своей записи превышает 10000 битов. Это происходит из-за увеличения коэффициентов произведения многочленов, представление которых требует увеличение как количества простых чисел p_i для модулярного вычисления, так и за счет их роста. Поэтому количество операций для вычисления произведения n битных чисел растет примерно как $O(n \log n \log \log n (\log \log \log n)^2)$. В дальнейшем, в 2007 г. алгоритм умножения улучшался в направлении более быстрого расчета за $O(n \log n \log_*(n))$, где $\log_*(n)$ очень медленно растущая функция, определяемая как минимальное значение k , такого, что башня степеней из k двоек

$$n < 2^{2^{\dots 2^2}}$$

станет больше n . В 2018г. найден алгоритм умножения n битных чисел, вычисляющий произведение за $O(n \log n)$ операций. В этом же году, при достаточно разумных предположениях другие авторы доказали, что нельзя вычислить произведение произвольных n битных чисел быстрее, чем за $O(n \log n)$ операций.

2.4 Бигрупповая алгебра

Пусть G конечная абелева группа порядка n , K коммутативное кольцо с единицей, где n обратимо и имеются корни из 1 соответствующей степени, $K(G)$ групповая алгебра. Элементы групповой алгебры можно рассматривать как линейные операторы, действующее в $V = K(G)$ через умножение. Отметим, что все эти операторы в базисе t_μ приводятся к диагональному виду

$$gt_\mu = \mu^{-1}(g)t_\mu.$$

Введем еще операторы-характеры, действующие диагонально в базисе g :

$$\mu : g \rightarrow \mu(g)g.$$

Полученная алгебра операторов на $V = K(G)$, называемая в дальнейшем бигрупповой алгеброй группы G , состоит из формальных линейных сумм:

$$a(\mu, g)\mu g.$$

Образующие этой алгебры μ, g удовлетворяют коммутационным соотношениям:

$$(\mu g)(g_1) = (\mu)(gg_1) = \mu(gg_1)gg_1 = \mu(g)g\mu(g_1)g_1 = (\mu(g)g\mu)(g_1).$$

Следовательно

$$\mu g = \mu(g)g\mu. \quad (2.19)$$

Это означает, что бигрупповая алгебра является некоммутативной цветной градуированной алгеброй, с группой градуировки (цветов) изоморфной $G^* \oplus G$.

Выберем образующие в циклическом разложении на подгруппы

$$G = Z_{n_1} \oplus Z_{n_2} \oplus \dots \oplus Z_{n_k}.$$

Сопоставим им переменные $y_1, \dots, y_k$, аналогичному разложению в группе характеров сопоставим переменные $x_1, \dots, x_k$. Все переменные между собой коммутируют за исключением взаимных и их степеней:

$$x_i y_i = \theta_i y_i x_i,$$

где θ_i примитивный корень из 1 степени n_i . Заметим, что переменные $x_i^{n_i} = 1, y_i^{n_i} = 1$ коммутируют со всеми многочленами от этих переменных, поэтому можно считать, что они принадлежат кольцу K .

Так получаются квазикоммутативные многочлены, соответствующие бигрупповой алгебре с образующими $x_i, y_i, x_i^{n_i} = 1 = y_i^{n_i}$ и с коммутационными соотношениями

$$x_i y_j = \theta_i^{\delta_i^j} y_j x_i.$$

Когда $k > 1$ удобно пользоваться мультииндексами $i = (i_1, i_2, \dots, i_k), \theta = (\theta_1, \theta_2, \dots, \theta_k), x = (x_1, x_2, \dots, x_k), y = (y_1, y_2, \dots, y_k)$. При этом операции с мультииндексными величинами вычисляются покомпонентно:

$$i + j = (i_1 + j_1, \dots, i_k + j_k), \quad ij = (i_1 j_1, \dots, i_k j_k), \quad x^i = x_1^{i_1} x_2^{i_2} \dots x_k^{i_k}.$$

Тогда многочлены можно рассматривать как многочлены от двух (мультииндексных) переменных с коммутационным соотношением:

$$x^i y^j = \theta^{ij} y^j x^i.$$

Эти обозначения удобны и представляют простые обозначения для тензорных произведений k объектов. Докажем теперь следующую теорему.

Теорема 4. Пусть G коммутативная группа порядка n . Характеристика поля не делит n и группа характеров со значениями в поле имеет порядок n . Тогда бигрупповая алгебра изоморфна алгебре матриц порядка $n \times n$

Доказательство. Бигрупповая алгебра имеет размерность n^2 как и алгебра матриц. Поэтому, нам достаточно показать, что матрицам e_l^g состоящий из единицы на пересечении строки с номером соответствующим элементу g и столбца с номером l и нулями в остальных позициях соответствует некоторый элемент бигрупповой алгебры. Для этого покажем, что многочлену

$$a = \frac{1}{|G|} g \sum_{\mu} \mu l^{-1} = \frac{1}{n} \sum_{\mu} \mu g l^{-1}$$

соответствует матрица e_l^g в базисе e_g . Вычислим действие элемента a на один из векторов e_h :

$$a(e_h) = \frac{1}{n} \sum_{\mu} \mu(l^{-1}h) e_{gl^{-1}h}.$$

В соответствии с леммой 1 сумма равна нулю при $h \neq l$, а при $h = l$ получаем, что вектор переходит в e_g . А это означает, что элементу a бигрупповой алгебры соответствует матрица e_l^g . Так как любой матрице $A = \sum_{gl} a_{gl} e_l^g$ мы можем сопоставить многочлен (элемент бигрупповой алгебры) по формуле:

$$\sum_{\mu g} \bar{a}_{\mu g} \mu g, \quad \bar{a}_{\mu g} = \frac{1}{n} \sum_l \mu(g^{-1}l^{-1}) a_{gl,l}.$$

Умножим слева и справа на множитель $\mu(s)$ и просуммируем по всем характерам. Это даст формулу обратного перехода:

$$a_{sl} = \sum_{\mu} \mu(s) \bar{a}_{\mu,l}.$$

Этим теорема доказана. □

Если матрица единичная, то $\bar{a}_{\mu g} = 0$ за исключением случая, когда $\mu = e = g$. Этому случаю соответствует свободный коэффициент многочлена и у единичной матрицы он равен $\bar{a}_{ee} = 1$. Для некоторых алгебр свободный член еще называют действительной частью. Для матриц он равен среднему от собственных значений $\frac{1}{n} \text{Tr}(A)$. Если найдется такой многочлен, который в произведении с нашим многочленом дает нули всюду за исключением свободного члена, то соответствующая матрица в произведении с нашей матрицей даст единичную матрицу, умноженный на этот коэффициент. Это позволяет свести задачу нахождения обратной матрицы к задаче нахождения обратного в многочленах. Аналогично, многочленам, не содержащим переменных y_i соответствуют диагональные матрицы. Для приведения матрицы к диагональной форме, соответствующей многочлену f , следует найти многочлен g , что gfg^{-1} не содержит переменных y_i . Таким образом задачи линейной алгебры сводятся к задачам не коммутативных многочленов. Последние более близки к комбинаторным задачам.

В дальнейшем ограничимся рассмотрением только случая $G = (Z_p)^k$. В этом случае, группа цветов является $2k$ мерным симплектическим пространством над конечным полем Z_p (p - простое число). Единичной матрице соответствует нулевой цвет (произведению однородных-цветных элементов соответствует сложение цветов по модулю p). Симплектическая структура однородных элементов $\prod_l x_l^{i_l} = x^I, I = (i_1, \dots, i_{2k}, \prod_l x_l^{j_l} = x^J$ определяется

формулой:

$$[I, J] = \sum_{l=1}^k i_l j_{l+k} - i_{l+k} j_l.$$

Это кососимметрическое скалярное произведение определяет коммутационные соотношения двух цветных элементов a, b со цветами I, J :

$$ab = \theta^{[I, J]} ba.$$

В обозначениях учитывая кососимметричность использовали квадратные скобки как в случае кососимметричного произведения в алгебрах Ли.

Сопоставим образующим бигрупповой алгебры представляющие их матрицы для группы $G = Z_{n_1} \oplus Z_{n_2} \oplus \dots \oplus Z_{n_k}$. Рассмотрим вначале случай $n_1 = n$ ($k = 1$). Пусть $\theta = \theta_1$ примитивный корень степени n_1 . Характерам сопоставим диагональные матрицы:

$$x = \begin{pmatrix} 1 & 0 & 0 & \dots & \dots \\ 0 & \theta & 0 & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & 0 & \theta^{i-1} & 0 & \dots \\ \dots & \dots & 0 & 0 & \theta^i & \dots \\ \dots & \dots & \dots & \dots & \dots & \theta^{n-1} \end{pmatrix}$$

В качестве матрицы y можно взять:

$$y = \begin{pmatrix} 0 & 0 & 0 & \dots & \dots & \dots & 0 & 1 \\ 1 & 0 & 0 & \dots & \dots & \dots & \dots & \dots \\ 0 & 1 & 0 & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 1 & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & 1 & 0 & 0 & \dots & \dots \\ \dots & \dots & \dots & 0 & 1 & 0 & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & 1 & 0 \end{pmatrix}$$

Легко проверить, что

$$x_1 y_1 = \theta_1 y_1 x_1, \quad x_1^{n_1} = E = y_1^{n_1}.$$

Здесь E - единичная матрица размерности n_1 . Базис матриц $x_1^i y_1^j, 0 \leq i < n_1, 0 \leq j < n_1$ называется базисом Сильвестра, а базис $\theta_1^{-ij/2} x_1^i y_1^j$ называется базисом Вейля-Швингера. Для случая $k > 1$ вводим числа $N_0 = 1, N_i = n_i N_{i-1}$ и определим матрицы

$$x_i = (E_{N_{i-1}} \otimes x_i) \otimes E_{n/N_i}, y_i = (E_{N_{i-1}} \otimes y_i) \otimes E_{n/N_i}.$$

Это означает, что в соответствующей матрице x_i с $\theta_i, \theta_i^{n_i} = 1$ или y_i заменяем нулевые элементы на нулевые матрицы порядка N_{i-1} , числа $1, \theta^i$ на диагональные матрицы, являющиеся произведением единичной матрицы порядка N_i на эти числа, и продолжаем вдоль диагонали n/N_i раз. Тогда все матрицы будут иметь порядок n и будут выполняться соотношения нормировки $x_i^{n_i} = E = y_i^{n_i}$ и коммутационные соотношения

$$x_i y_i = \theta_i y_i x_i, \quad x_i y_j = y_j x_i, i \neq j, \quad x_i x_j = x_j x_i, y_i y_j = y_j y_i. \quad (2.20)$$

Заметим, что все элементы базиса, соответствующие не нулевому цвету (нулевому цвету соответствует единичная матрица) имеют нулевой след.

Соответствующие однородные элементы образуют (обобщенный) базис Сильвестра и обобщенный базис Вейля-Швингера:

$$W_{i_1, j_1, i_2, j_2, \dots, i_k, j_k} = \prod_{l=1}^k \theta_l^{-i_l j_l / 2} x_l^{i_l} y_l^{j_l}.$$

Если n_l нечетное, то число $i_l j_l / 2$ всегда можно заменить на целое число, вычисленное по модулю n_l . Базис Вейля-Швингера удобен тем, что если автоморфизм переводит образующие x_i, y_i в элементы базиса Вейля-Швингера (без множителей, являющиеся корнями из 1), то любой элемент базиса Вейля-Швингера переходит в элементы этого базиса. Такие автоморфизмы назовем положительными. Любой цветной автоморфизм является произведением знакового автоморфизма, меняющей определенным образом только знаки множители однородных элементов, и положительного автоморфизма.

Ясно, что алгебраическая структура бигрупповой алгебры однозначно определяет эту симплектическую структуру. Ответим на вопрос, нельзя ли бигрупповую алгебру представить как групповую алгебру? Допустим, что группа содержит образующие x_i, y_i бигрупповой алгебры. Пусть $g_1 * g_2 = g_3$, $g_4 * g_5 = \theta g_3$. Так как в группе произведения не совпадают, а множители принадлежат группе, то существует элемент $g_6 = g_4 * g_5 \neq g_3$. Групповая алгебра конечной группы G состоит из элементов

$$\sum_{g \in G} a(g)g, \quad \sum_g a(g)g = 0 \rightarrow \forall g : a(g) = 0.$$

В бигрупповой алгебре $g_6 + g_3 = 0$ ($\theta = -1$). Следовательно, если она представляется как групповая алгебра должна быть $a(g_6) = 1 = 0 = 1 = a(g_3)$. Получили противоречие.

В следующей секции займемся тем, определяется ли алгебраическая структура симплектической структурой на группе цветов.

2.5 Автоморфизмы бигрупповой алгебры

Бигрупповые алгебры группы Z_{p^k} и $(Z_p)^k$ изоморфны как алгебры, но не изоморфны как цветные алгебры. Под цветными изоморфизмами мы подразумеваем изоморфизмы, переводящие однородные элементы одной алгебры в однородные элементы другой алгебры. Множество цветных автоморфизмов у второго побольше и их структура проще. Структура бигрупповых алгебр проще и автоморфизмы полностью определяются автоморфизмами группы цветов, являющихся симплектическим пространством размерности $2k$ над полем Z_p .

Любой гомоморфизм (ассоциативных алгебр над коммутативным кольцом) из одной алгебры в другую полностью определяется значениями на образующих алгебр. Если элемент x является корнем полинома $P(x) = a_0 + a_1 x + \dots + a_k x^k = 0$, то образ $f(x)$ будет корнем такого же многочлена $a_0 + a_1 f(x) + \dots + a_k f(x)^k = 0$. В этом смысле, группа автоморфизмов алгебр играют роль группы Галуа для алгебраических чисел. Интересны так же полиномы от нескольких переменных;

$$xy - 1 = 0 \Rightarrow f(x)f(y) = 1, xyx^{-1}y^{-1} = a \rightarrow f(x)f(y)f(x)^{-1}f(y)^{-1} = a.$$

Наиболее интересными образующими являются корни из 1 $x^p - 1 = 0$, нильпотенты $x^p = 0$ и проекторы $x^2 - x = 0$. Наиболее удобными являются образующие Дарбу. Для бигрупповой алгебры группы $(Z_p)^k$ это соответствует базису Дарбу в группе цветов $x_1, \dots, x_k, x_{k+1} = y_1, \dots, x_{2k} = y_k$, удовлетворяющих коммутационным соотношениям:

$$x_i x_j x_i^{-1} x_j^{-1} = \theta^{\delta_k^{j-i} - \delta_k^{i-j}}, \quad \theta^p = 1, \quad x_i^p = 1.$$

Здесь θ примитивный корень степени p . Число p может быть и не простым числом. Только в этом случае группа цветов будет симплектическим пространством не над полем Z_p , а над кольцом с делителями нуля. При автоморфизме образующие Дарбу перейдут в переменные удовлетворяющие таким же коммутационным соотношениям. Иногда удобнее пользоваться образующими Клиффорда, удовлетворяющие коммутационным соотношениям:

$$x_i x_j x_i^{-1} x_j^{-1} = \theta_i, \quad i < j, \quad \theta_i^p = 1.$$

Здесь θ_i примитивные корни степени p (p - не обязательно простое). Имея образующие Дарбу, легко строит образующие Клиффорда, а из образующих Клиффорда строит образующие Дарбу. Если x_i образующие Дарбу, то

$$x_1, x_{k+1}, x_1 x_{k+1} x_2, x_1 x_{k+1} x_{k+2}, \dots, \left(\prod_{i=1}^{k-1} x_i x_{i+k} \right) * x_k, \left(\prod_{i=1}^{k-1} x_i x_{i+k} \right) x_{2k}, \prod_{i=1}^k x_i x_{i+k}$$

являются образующими Клиффорда. Правда здесь последний член лишний как для базиса. Для образующих Клиффорда имеет место обобщение теоремы Поттера:

$$(a_1 x_1 + \dots + a_m x_m)^p = \sum_i a_i^p, \quad (x_i^p) = 1, \quad m \leq 2k.$$

При $m = 2k + 1$ (с лишним членом) сумма справа будет иметь член $\prod_i a_i \prod_i x_i$, если $\prod_i x_i$ число. Для степеней с образующими Клиффорда легко выражаются квантовые биномиальные коэффициенты.

Мы будем использовать только образующие Дарбу с простым числом $p \geq 2$. Пусть f цветной автоморфизм. Тогда образующие x_i переходят в

$$f(x_i) = \theta^{c_i} \prod_{j=1}^{2k} x_j^{a_{ij}} = \theta^{c'_i} W(a_{i1}, \dots, a_{i2k}), \quad c'_i = c_i + \sum_{j=1}^k a_{ij} a_{i(j+k)}.$$

Здесь $\bar{a}_i = (a_{i1}, a_{i2}, \dots, a_{i(2k)})$ цвет однородного элемента, а $W(a_{i1}, \dots, a_{i2k})$ элемент базиса Вейля-Швингера соответствующего цвета. Автоморфизм можно разложить на знаковый и положительный. Под знаковым подразумеваем автоморфизм, у которой матрица

$$A = \begin{pmatrix} 1 & 0 & 0 & \dots & \dots \\ 0 & 1 & 0 & \dots & \dots \\ 0 & 0 & 1 & \dots & \dots \\ \dots & \dots & \dots & 1 & 0 & 0 & \dots \\ \dots & \dots & \dots & 0 & 1 & 0 & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & 1 \end{pmatrix} = E$$

единичная. При этом меняются только знаки перед представителями однородных (цветных) элементов, а цвета не меняются. Под положительным автоморфизмом понимаем автоморфизм, у которой все знаки $c'_i = 0$ - положительные. При этом однородный элемент цвета a переходит в однородный элемент цвета b :

$$x^a \rightarrow \theta^{f(a)} x^b, \quad b = Aa, \quad A = (a_{ij}), \quad i, j = 1, 2, \dots, 2k. \quad (2.21)$$

Группа цветов C является $2k$ мерным пространством над полем Z_p . Обратным к однородному элементу $x^a = x_1^{a_1} x_2^{a_2} \dots x_{2k}^{a_{2k}}$ является $x^{-a} = x_{2k}^{-a_{2k}} \dots x_1^{-a_1} = \theta^{\phi(a)} x_1^{-a_1} \dots x_{2k}^{-a_{2k}}$, где $\phi(a) = -\sum_{i=1}^k a_i a_{i+k}$. Возьмем два однородных элемента $\alpha = A_0 x^a = A_0 x_1^{a_1} \dots x_{2k}^{a_{2k}}$, $\beta = B_0 x^b = B_0 x_1^{b_1} \dots x_{2k}^{b_{2k}}$. Произведения $\alpha\beta, \beta\alpha$ отличаются на знак $\alpha\beta = \theta^{[a,b]} \beta\alpha$. Вычислим этот знак, учитывая коммутационные соотношения, меняющие знак при перестановке сомножителей:

$$[a, b] = \sum_{i=1}^k a_i b_{i+k} - a_{i+k} b_i.$$

Здесь $a_i b_{i+k}$ появляется при переносе $x_i^{a_i} y_i^{b_{i+k}} = \theta^{a_i b_{i+k}} y_i^{b_{i+k}} x_i^{a_i}$ и $a_{i+k} b_i$ при переносе $y_i^{a_{i+k}} x_i^{b_i} = \theta^{-a_{i+k} b_i} x_i^{b_i} y_i^{a_{i+k}}$. Эта операция $C \times C \rightarrow Z_p$ билинейна и кососимметрична. Она не вырождена. Следовательно, группа цветов с такой кососимметричной операцией (скалярным произведением) становится симплектическим пространством над полем Z_p .

К кососимметричному скалярному произведению можно прийти еще так. Отображение f переводящее $x_i \rightarrow x_{i+k}, x_{i+k} \rightarrow x_i^{-1}, i \leq k$ является автоморфизмом, который при повторном применении переводит образующие в свои обратные. Определим на группе цветов сопряжение $c = (c_1, \dots, c_k, c_{k+1}, \dots, c_{2k}) \rightarrow \bar{c} = (c_{k+1}, \dots, c_{2k}, -c_1, \dots, -c_k)$ - соответствующее изменению цвета при таком автоморфизме. Правда под сопряжением обычно понимают операцию, которая при повторном применении приводит к тождественному преобразованию, т.е. равна своему обратному. В нашем случае повторное сопряжение приводит тождественному преобразованию с точностью до знака $\bar{\bar{c}} = -c$. Такое сопряжение приводит к ранее полученному антисимметричному скалярному произведению:

$$[a, b] = (a, \bar{b}) = \sum_{i=1}^k (a_i b_{i+k} - a_{i+k} b_i) = -[b, a].$$

Имеет место:

Теорема 5. Матрица преобразования группы цветов $A = (a_{ij})$ (размерности $2k \times 2k$ над полем Z_p) цветного автоморфизма симплектична. Произведение элементов базиса Вейля-Швингера определяется по формуле:

$$W(a)W(b) = \theta^{[a,b]/2} W(a+b).$$

Положительный автоморфизм переставляет элементы базиса Вейля-Швингера не меняя знаки перед ними.

Доказательство. Коммутатор двух однородных элементов является числом - корнем p -ой степени от 1. Следовательно, при автоморфизме переходит в саму себя. Пусть при автоморфизме однородные элементы x^a, x^b переходят в $\theta^{\phi(a)} x^{Aa}, \theta^{\phi(b)} x^{Ab}$. Тогда $x^a x^b = \theta^{[a,b]} x^b x^a$ переходит в

$$\theta^{\phi(a)+\phi(b)} x^{Aa} x^{Ab} = \theta^{\phi(a)+\phi(b)+[Aa, Ab]} x^{Ab} x^{Aa} = \theta^{\phi(a)+\phi(b)+[a,b]} x^{Ab} x^{Aa}.$$

Следовательно матрица A , соответствующая автоморфизму сохраняет кососимметричное скалярное произведение:

$$[Aa, Ab] = [a, b], \quad (A, \overline{Ab}) = (a, \bar{b}).$$

Переход к сопряженному в группе цветов осуществляется умножением на матрицу:

$$\bar{b} = Jb, \quad J = \begin{pmatrix} \dots & \dots & \dots & 1 & 0 & \dots \\ \dots & \dots & \dots & 0 & 1 & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & 1 \\ -1 & 0 & \dots & \dots & \dots & \dots \\ 0 & -1 & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & -1 & 0 & \dots & 0 \end{pmatrix} = \begin{pmatrix} 0 & E \\ -E & 0 \end{pmatrix}$$

Следовательно, для любых цветов a, b) выполняется:

$$(a, Jb) = (Aa, JAb) = (a, A^T JAb) \Rightarrow J = A^T JA.$$

Это означает симплектичность матрицы A .

Выразим $x^a x^b$ через $W(a+b) = \theta^{-\frac{1}{2} \sum_{i=1}^k (a_i + b_i)(a_{i+k} + b_{i+k})}$:

$$x^a x^b = \prod_{i=1}^{2k} x_i^{a_i} \prod_{i=1}^{2k} x_i^{b_i} = \theta^{-\sum_{i=1}^k a_{i+k} b_i} \prod_{i=1}^{2k} x_i^{a_i + b_i}.$$

Следовательно

$$x^a x^b = \theta^\alpha W(a+b), \quad \alpha = \frac{1}{2} \sum_{i=1}^k (a_i + b_i)(a_{i+k} + b_{i+k}) - \sum_{i=1}^k a_{i+k} b_i.$$

Вычислим произведение двух элементов базиса Вейля-Швингера:

$$W(a)W(b) = \theta^{-\frac{1}{2} \sum_{i=1}^k (a_i a_{i+k} + b_i b_{i+k})} x^a x^b = \theta^\beta W(a+b),$$

где

$$\beta = \alpha - \frac{1}{2} \sum_{i=1}^k (a_i a_{i+k} + b_i b_{i+k}) = \frac{1}{2} \sum_{i=1}^k (a_i b_{i+k} - a_{i+k} b_i) = \frac{1}{2} [a, b].$$

Таким образом, $W(a)W(b) = \theta^{\frac{1}{2}[a,b]} W(a+b)$. Если $[a, b] = 0$, то $W(a)W(b) = W(a+b)$.

Представим цвет $a = (a_1, \dots, a_k, a_{k+1}, \dots, a_{2k}) = (a_1, \dots, a_k, 0, \dots, 0) + (0, \dots, 0, a_{k+1}, \dots, a_{2k}) = a' + a''$ в виде суммы. При этом $Aa = Aa' + Aa'' = b' + b'' = b$ и

$$W(a) = x^{a'} x^{a''} = \theta^{\frac{1}{2}[a', a'']} W(a') W(a'') = \theta^{\frac{1}{2}[a', a'']} x^a.$$

Пусть автоморфизм положительный, т.е. для любого i автоморфизм переводит $x_i \rightarrow W(d_i)$, $d_i = (a_{i1}, a_{i2}, \dots, a_{i(2k)})$. Представляя a' в виде суммы элементарных косоортонормальных векторов $(1, 0, \dots, 0) + \dots = a_1(1, 0, \dots, 0) + a_2(0, 1, \dots, 0) + \dots + a_k(0, 0, \dots, 0, 1, 0, \dots)$,

получаем, что $b' = Aa'$ представляется в виде суммы косоортогональных (в силу сохранения кососимметрических скалярных произведений симплектическим преобразованием A). Следовательно,

$$W(b') = \prod_{i=1}^k W(d_i)^{a_i},$$

как произведение элементов базиса Вейля-Швингера, соответствующих косоортогональным цветам. Тогда

$$W(a) = \theta^{-\frac{1}{2}[a', a'']} W(a') W(a'') \rightarrow \theta^{-\frac{1}{2}[a', a'']} W(b') W(b'') = \theta^{-\frac{1}{2}[a', a''] + \frac{1}{2}[b', b'']} W(b' + b'') = W(b).$$

Здесь учли, что $[b', b''] = [Aa', Aa''] = [a', a'']$. Это означает, что положительный автоморфизм любой элемент базиса Вейля-Швингера $W(a)$ переводит в другой элемент базиса $W(Aa)$ без изменения знака. $\square$

Эта теорема позволяет вычислить знаки перед однородными элементами для произведения двух элементов бигрупповой алгебры. Это важно, так как у произведения коэффициент перед одночленом образуется как сумма произведений многих однородных членов сомножителей с разными знаками. Так как цветной автоморфизм определяется $2k$ цветами и симплектической матрицей A порядка $2k \times 2k$ над полем Z_p , можно посчитать общее количество цветных автоморфизмов, которое равно количеству возможных знаков (знаковых автоморфизмов) p^{2k} , умноженное на количество симплектических матриц порядка p^{2k^2-k} . Для любого элемента $x^a y^b$, $(a, b) \neq (0, 0)$ $(x^a y^b)^p = \theta^{-ab(p-1+p-2+\dots+1)} x^{ap} y^{bp} = \theta^{-abp(p-1)/2} = 1, p \text{ odd}$. Поэтому при нечетном p количество симплектических отображений группы цветов равно

$$\prod_{i=1}^k (p^{2i} - 1) p^{2i-1} = p^{k^2} \prod_{i=1}^k (p^{2i} - 1).$$

Таким образом, общее количество цветных изоморфизмов равно

$$p^{k^2+2k} \prod_{i=1}^k (p^{2i} - 1) = O(p^{2k^2+3k}).$$

Такое большое количество автоморфизмов делает возможным использовать их в гомоморфной криптографии (автоморфизмы сохраняют обе операции).

Приведем еще несколько теорем о симплектических матрицах. Так как они не будут использоваться в дальнейшем, приведем их без доказательства.

Теорема 6. *Характеристическое уравнение симплектического отображения симметрично относительно замены $\lambda \rightarrow \lambda^{-1}$, т.е. может быть записано в виде уравнения*

$$\eta^k + a_1 \eta^{k-1} + \dots + a_k = 0, \quad \eta = \lambda + \lambda^{-1}.$$

При этом собственные вектора (определены с точностью до ненулевого множителя из поля $Z_{p^{2k}}$) можно выбрать так, чтобы они образовали базис Дарбу x_i, y_i .

Теорема 7. *Если характеристическое уравнение $\det(A - \lambda E) = 0$ симплектического отображения A не приводимо, то порядок отображения равен $q + 1$ ($A^{q+1} = E$). Ненулевые цвета (за исключением $x^0 = 1$) распадаются на $q - 1 = p^k - 1$ орбит этого автоморфизма длиной $q + 1$.*

2.6 Вычисление коэффициентов разложения матрицы и значений

Вычислим коэффициентов разложения в базисе Сильвестра бигрупповой алгебры группы

$$G = Z_{n_1} \bigoplus Z_{n_2} \bigoplus \dots \bigoplus Z_{n_k}.$$

Первоначально представим номера строк и столбцов в системе исчисления $n_1, n_2, \dots, n_k$:

$$i = i_1 N_0 + i_2 N_1 + \dots + i_k N_{k-1} = (i_1, i_2, \dots, i_k), \quad N_0 = 1, N_i = n_i * N_{i-1}, \quad 0 \leq i_j < n_j.$$

Элементу базиса Сильвестра

$$x^I y^J = \prod_{l=1}^k x_l^{i_l} y_l^{j_l}$$

соответствует матрица с элементами:

$$X_{I,J} = (x_{ij}), \quad x_{ij} = \theta^{(i,I)} \delta_i^j \oplus^J = \prod_{l=1}^k \theta_l^{i'_l i_l} \delta_{i'_l}^{j'_l} \oplus^{j_l}.$$

Здесь $i = (i'_1, \dots, i'_k)$, $j = (j'_1, \dots, j'_k)$ номера строк и столбцов представляются в системе исчисления $n_1, n_2, \dots, n_k$, знак $\oplus$ означает сумму по модулю n_l в каждой компоненте с номером l .

Матричное представление элемента бигрупповой алгебры $\sum_{I,J} x_{I,J} X_{I,J}$ имеет характер k кратного преобразования Фурье

$$a_{i,j} = \sum_I x_{I,i} \oplus_j \theta^{(i,I)}.$$

Переход из матричного представления к представлению в виде элемента бигрупповой алгебры имеет характер обратного преобразования Фурье:

$$x_{I,J} = \frac{1}{n} \sum_i \theta^{-(i,I)} a_{i,i} \oplus^J.$$

При малых n_i на эти и им подобные операции потребуется $O(n^2 k) = O(n^2 \ln n)$ операций. К таким операциям относится вычисление всех n^2 значений:

$$A = f(x, y), A^T = f^T(x, y), B = \phi(x, y), B^T = \phi^T(x, y).$$

Это вычисление и есть традиционное преобразование Фурье. Эти значения могут быть вычислены исходя как из координат в базисе Сильвестра $x_{I,J}$, так и непосредственно из коэффициентов матриц $a_{i,j}$. Количество выполняемых операций для этого вычисления $O(n^2 k) = O(n^2 \ln n)$.

Вычисление значений для произведения матриц получим исходя из попарных произведений элементов

$$C = AB = \sum_{I_1, J_1, I_2, J_2} a(I_1, J_1) x^{I_1} y^{J_1} b^T(I_2, J_2) y^{J_2} x^{I_2} = \sum_{I, J} c(I, J) x^I y^J.$$

Отсюда

$$c(I, J) = \sum_{I_1, J_1} a(I_1, J_1) b^T(I - I_1, J - J_1) \theta^{-(J, I - I_1)}, \quad x^{I_1} y^{J_1} y^{J_2} x^{I_2} = x^I y^J \theta^{-(J, I_2)}.$$

Аналогично получается $c^T(I, J) = c(I, J) \theta^{(J, I)}$.

Для выражения не коммутативности умножения через значения в произведении $\psi(x, y)$ докажем следующую лемму:

Лемма 5. *Значения произведения многочленов с не коммутирующими переменными выражаются формулой:*

$$\psi(x, y) = \frac{1}{n} \sum_{T_1, T_2} f(x, \theta^{T_1} y) \phi(\theta^{T_2} x, y) \theta^{(T_1, T_2)}. \quad (2.22)$$

Доказательство. Отметим, что сокращенная запись в (2.22) означает

$$f(x, \theta^{T_1} y) = \sum_{I, J} a(I, J) x^I y^J \theta^{(T, J)} = \sum_{I=(i_1, \dots, i_k), J=(j_1, \dots, j_k)} a(I, J) \prod_{l=1}^k x_l^{i_l} y_l^{j_l} \theta_l^{t_l j_l}.$$

Достаточно доказать лемму в случае одной компоненты ($k = 1$). Распишем подробно правую часть (2.22)

$$\begin{aligned} & \frac{1}{n} \sum_{t_1, t_2, i_1, j_1, i_2, j_2} a(i_1, j_1) b(i_2, j_2) x^{i=i_1+i_2} y^{j=j_1+j_2} \theta^{t_1 j_1 + t_2 i_2 + t_1 t_2} = \\ & = \frac{1}{n} \sum_{i, j} x^i y^j \sum_{i_1, j_1} a(i_1, j_1) b(i - i_1, j - j_1) \sum_{t_1} \theta^{t_1 j_1} \sum_{t_2} \theta^{t_2 (i_2 + t_1)}. \end{aligned}$$

Последняя (внутренняя) сумма равна 0 за исключением случая $i_2 + t_1 = 0 \pmod n$. В этом случае сумма равна n . Соответственно,

$$\sum_{t_1, t_2} \theta^{t_1 j_1 + t_2 i_2 + t_1 t_2} = n \theta^{-j_1 i_2}.$$

Это и есть коэффициент переноса переменной y^{j_1} :

$$x^{i_1} y^{j_1} x^{i_2} y^{j_2} = x^i y^j \theta^{-j_1 i_2}.$$

Следовательно, коэффициент перед $x^i y^j$ произведения есть

$$\sum_{j_1, i_1} a(i_1, j_1) b(i - i_1, j - j_1) \theta^{-j_1 i_2} = c(i, j).$$

Это доказывает лемму. □

Подставив $y \rightarrow \theta^{-T_1}y$ получим другой вид соотношений

$$\psi(x, \theta^{-T_1}y) = \frac{1}{n} f(x, y) \sum_{T_1, T_2} \phi(\theta^{T_2}x, \theta^{-T_1}y) \theta^{(T_1, T_2)}.$$

Умножение элементов бигрупповой алгебры так же выполняется по компонентам через вычисления произведений значений и выделения коэффициентов перед элементами базиса Сильвестра. Формулы (2.22) позволяют вычислить коэффициенты $c(I, J)$ в общем случае обходясь всего $2n^2 - 1$ произведениями значений сомножителей при любом n . При $G = (Z_2)^k$ вычисления проще и можно обойтись без комплексных умножений для вычисления произведения действительных матриц.

Вычисление произведения опирается на автоморфизмах, меняющих только знаки перед базисными элементами. Переходя в нумерации строк и столбцов к $(n_1, n_2, \dots, n_k)$ исчислению, произведение элементов выразится по формуле:

$$c_{I,K} = \sum_L a_{I,L} b_{L,K}.$$

Произведение симметрично относительно перестановок элементов матриц:

$$c_{I \oplus J, K \oplus J} = \sum_L a_{I \oplus J, L \oplus J} b_{L \oplus J, K \oplus J}.$$

Это соответствует автоморфизму матриц

$$X \rightarrow y^J X y^{-J}.$$

В частности, при $G = (Z_2)^k, n = 2^k, y^J = y_i$ этот автоморфизм действует в каждом блоке порядка 2^i , разбитой на подблоки порядка 2^{i-1} так:

$$S_{iy} : \begin{pmatrix} A_1 & A_2 \\ A_3 & A_4 \end{pmatrix} \rightarrow \begin{pmatrix} A_4 & A_3 \\ A_2 & A_1 \end{pmatrix}, \quad \begin{pmatrix} C_4 & C_3 \\ C_2 & C_1 \end{pmatrix} = \begin{pmatrix} A_4 & A_3 \\ A_2 & A_1 \end{pmatrix} * \begin{pmatrix} B_4 & B_3 \\ B_2 & B_1 \end{pmatrix}.$$

Произведение симметрично так же относительно умножения строк и столбцов на:

$$\theta^{(I,J)-(J,K)} c_{I,K} = \sum_L \theta^{(I,J)-(J,L)} a_{I,L} \theta^{(L,J)-(J,K)} b_{L,K}.$$

Это соответствует автоморфизму матриц

$$X \rightarrow x^J X x^{-J}.$$

В частности, при $G = (Z_2)^k, n = 2^k, x^J = x_i$ этот автоморфизм действует в каждом блоке порядка 2^i , разбитой на подблоки порядка 2^{i-1} так:

$$S_{ix} : \begin{pmatrix} A_1 & A_2 \\ A_3 & A_4 \end{pmatrix} \rightarrow \begin{pmatrix} A_1 & -A_2 \\ -A_3 & A_4 \end{pmatrix}, \quad \begin{pmatrix} C_1 & -C_2 \\ -C_3 & C_4 \end{pmatrix} = \begin{pmatrix} A_1 & -A_2 \\ -A_3 & A_4 \end{pmatrix} * \begin{pmatrix} B_1 & -B_2 \\ -B_3 & B_4 \end{pmatrix}.$$

Порожденные ими n^2 симметрий и n^2 антиизоморфизмов в сочетании с транспонированием образуют достаточное множество функционалов для вычисления произведения. Автоморфизмы меняют знаки перед половиной элементов базиса Сильвестра, являющихся

собственными "векторами" относительно таких автоморфизмов с собственными значениями ± 1 . Соответственно, они действуют на значениях меняя знаки на противоположные у этой половины. Транспонирование меняет значения перед нечетными элементами базиса, их $\frac{n(n-1)}{2}$.

На самом деле последние симметрии можно обобщить:

$$c_{I,K} \rightarrow \theta^{(I,J_1)-(J_3,K)} c_{I,K}, \quad a_{I,K} \rightarrow \theta^{(I,J_1)-(J_2,K)} a_{I,K}, \quad b_{I,K} \rightarrow \theta^{(I,J_2)-(J_3,K)} b_{I,K}.$$

Впрочем перестановки по строкам и столбцам можно сделать разными по этой аналогии.

Предварительно рассмотрим умножение матриц порядка 2×2 используя значения как коэффициенты разложения по проекторам.

2.7 Умножение через значения при $n = 2^k$

Выбор билинейных функционалов для вычисления произведения не однозначен. Если заданы некоторые функционалы, являющиеся значениями, то любой автоморфизм определяет другой набор значений. В алгебре матриц автоморфизмы алгебры внутренние $A(a) = CaC^{-1}$. Градуированные внутренние автоморфизмы в бигрупповой алгебре являются знаковыми и меняют знаки, за исключением знака перед единицей. Пусть t некоторый элемент базиса Сильвестра. Сопоставим этому элементу автоморфизм T и характер t

$$T : T(g) = tgt^{-1} = \varepsilon_g g, \quad \varepsilon_g = tgt^{-1}g^{-1}, \quad t(g) = tgt^{-1}, \quad t(g_1g_2) = t(g_1)t(g_2).$$

Переход к транспонированию осуществляется заменой базиса Сильвестра на обратные элементы $g \rightarrow g^{-1}$ и является антиизоморфизмом или нечетным автоморфизмом алгебры. Переход к транспонированию и автоморфизм является нечетным автоморфизмом. Они определяют нечетные характеры. Мы здесь рассматриваем только бигрупповые алгебры группы $(Z_2)^k$. В этом случае можно обойтись без комплексных значений. Элементы базиса g разделяются на четные $g^2 = 1$ и нечетные $g^2 = -1$. При транспонировании матрицы меняются знаки коэффициентов перед нечетными элементами.

Пусть матрицы A, B представляются в базисе Сильвестра в виде

$$A = \sum_g a(g)g = \sum_{I,J} a(I,J)x^I y^J, \quad B = \sum_g b(g) = \sum_{I,J} b(I,J)x^I y^J.$$

Их произведение представляется в виде:

$$C = \sum_{g_1, g_2} a(g_1)b(g_2)g_1g_2 = \sum_{I_1, J_1} x^{I_1} y^{J_1} \sum_{I_2, J_2} a(I_1, J_1)b(I_2, J_2)(-1)^{(J_1, I_2 \oplus I_1)}.$$

Обозначим $I_2 = I \oplus I_1, J_2 = J \oplus J_1$. Каждое произведение $a(I_1, J_1)b(I_2, J_2)$ входит в произведение перед $x^I y^J$ один раз с коэффициентом $(-1)^{(J_1, I_2)}$. В каждое произведение $t = (T_x, T_y)$ значений сомножителей произведение $a(I_1, J_1)b(I_2, J_2)$ входит один раз со знаком $(-1)^{(T_x, J_1 \oplus J_2) + (T_y, I_1 \oplus I_2)}$. Если удастся выбрать знаки $(-1)^{s(T_x, T_y, I, J)}$ перед произведениями t значений так, чтобы сумма была

$$\sum_t (-1)^{s(t, g') + [t, g']} = R(g) \delta_g^{g'} (-1)^{(J_1, I_2)},$$

то с помощью таких сумм произведений значений мы выделим коэффициенты перед $g = x^I y^J$ поделив на $R(g)$. Если функцию $s(t, g')$ искать в виде $s(t, g) = \sum_{l=1}^k s(t_l, g_l)$ (покомпонентной), то достаточно удовлетворять этому условию для одной компоненты. Поэтому, в дальнейшем в этом параграфе рассмотрим только случай $k = 1$ ($n = 2$) одной компоненты.

Представим матрицы в виде не коммутативных многочленов $A = f(x, y), B = \phi(x, y)$:

$$A = a(0, 0) + a(1, 0)x + a(0, 1)y + a(1, 1)xy, B = b(0, 0) + b(1, 0)x + b(0, 1)y + b(1, 1)xy.$$

Вычислим их произведение $C = \psi(x, y) = f(x, y) * \phi(x, y)$, учитывая не коммутативность умножения.

Если считать значения подставляя в многочлен значения $x = \pm 1, y = \pm 1, x^2 = 1, y^2 = 1$ и обозначить значения первого множителя $f(x, y)$ и второго $\phi(x, y)$, то значения произведения $\psi(x, y)$ должны менять знак перед членом, содержащим переменную y в первом множителе в сочетании с множителем x во втором множителе. Здесь мы приняли правило, что все значения многочленов, как у множителей, так и у произведения, считаются в записи, когда переменная x стоит слева от переменной y . Таким образом, значение $\psi(x, y) = f(x, y) * \phi(x, y), f(x, y) = a_0 + a_1x + a_2y + a_3xy, \phi(x, y) = b_0 + b_1x + b_2y + b_3xy$ вычисляется так:

$$\psi(x, y) = f(x, y) \cdot \phi(x, y) - 2(a_2y + a_3xy) \cdot (b_1x + b_3xy).$$

Здесь приняли обозначение $*$ означает умножение некоммутативных многочленов, а $\cdot$ умножение (коммутативное) значений многочленов. Величина $a_2y + a_3xy$ вычисляется через значения многочлена $a_2y + a_3xy = \frac{1}{2}(f(x, y) - f(x, -y))$, аналогично $b_1x + b_3xy = \frac{1}{2}(\phi(x, y) - \phi(-x, y))$. Это означает

$$\begin{aligned} f(x, y) * \phi(x, y) &= f(x, y) \cdot \phi(x, y) - \frac{1}{2}(f(x, y) - f(x, -y)) \cdot (\phi(x, y) - \phi(-x, y)) = \\ &= \frac{1}{2}[f(x, y) \cdot \phi(x, y) + f(x, y) \cdot \phi(-x, y)) + f(x, -y) \cdot \phi(-x, y) - f(x, -y) \cdot \phi(-x, y)]. \end{aligned}$$

По другому, значения произведения многочленов выражается через произведения значений сомножителей по формуле:

$$f(x, y) * \phi(x, y) = \frac{1}{n} \sum_{t_1=0}^{n-1} \sum_{t_2=0}^{n-1} (-1)^{t_1 t_2} f(x, (-1)^{t_1} y) \cdot \phi((-1)^{t_2} x, y). \quad (2.23)$$

При $n = 2^k$ аналог формулы (2.23) будет содержать множество изменений знаков:

$$\psi(x, y) = f(x, y) * \phi(x, y) = \frac{1}{n} \sum_{T_1=0}^{n-1} \sum_{T_2=0}^{n-1} (-1)^{(T_1, T_2)} f(x, (-1)^{T_1} y) \cdot \phi((-1)^{T_2} x, y). \quad (2.24)$$

Эта формула является частным случаем формулы (2.22) для группы $G = (Z_2)^k$. Аналогично получается транспонированный вид:

$$\psi^T(x, y) = \frac{1}{n} \sum_{T_1=0}^{n-1} \sum_{T_2=0}^{n-1} (-1)^{(T_1, T_2)} f((-1)^{T_1} x, y) \cdot \phi(x, (-1)^{T_2} y). \quad (2.25)$$

Достаточно вычислить значения функционалов при двух различных вариантах, отличающихся транспонированием. Удобнее работать значениями -функционалами одной матрицы f , а у другой с транспонированным значением ϕ^T и наоборот. Удобство заключается в том, что коэффициент при единице выражается только с положительными знаками:

$$c(1) = \sum_{j_1, I_1} a(I_1, J_1) b^T(I_1, J_1) = \sum_{I_1, J_1} a^T(I_1, J_1) b(I_1, J_1).$$

Формулы (2.24), (2.25) остаются в силе, только надо учесть, что у транспонированного значения коэффициент соответствует транспонированному элементу базиса:

$$b^T(I_1, J_1) y^{J_1} x^{I_1} = b(I_1, J_1) x^{I_1} y^{J_1}.$$

Перенос y в конец соответствует переносу суммарной степени:

$$a(I_1, J_1) x^{I_1} y^{J_1} b^T(I_2, J_2) y^{J_2} x^{I_2} = (-1)^{(J, I_2)} a(I_1, J_1) x^{I_1} y^{J_1} b^T(I_2, J_2) x^I y^J, \quad I = I_1 + I_2, J = J_1 + J_2.$$

Аналогично с переносом переменной x .

Рассмотрим следующие произведения:

$$\psi_{T_1, T_2}(x, y) = f(x, (-1)^{T_1} y) \phi^T((-1)^{T_2} x, y),$$

$$\psi_{T_1, T_2}^T(x, y) = f^T((-1)^{T_2} x, y) \phi(x, (-1)^{T_1} y).$$

Точнее мы вычисляем произведения, соответствующие значениям аргументов $x_i = y_i = 1$. В каждое из этих произведений входит член

$$a(I_1, J_1) b(I_2, J_2) = a(I_1, J_1) b^T(I_2, J_2) (-1)^{I_2 J_2} = a^T(I_1, J_1) b(I_2, J_2) (-1)^{I_1 J_1}$$

с некоторым знаком, зависящим так же от T_1, T_2 . В первом случае знак равен

$$(-1)^{(T_1, J_1) + (T_2, J, I_2)},$$

и он соответствует перебросу члена x^{I_2} вперед в произведении $x^{I_1} y^{J_1} y^{J_2} x^{I_2}$. В член $C(I, J)$ произведения матриц они входят со знаком $(-1)^{(J, I_2)}$. Соответственно, от реального знака отличается на $(-1)^{(T_1, J_1) + (T_2, I_2)}$. Аналогично, второй член отличается от реального на множитель-знак $(-1)^{(T_1, J_2) + (T_2, I_1) + (I, J)}$.

Коэффициент перед нулевой степени в произведении есть

$$c(0, 0) = \sum_{i_1, j_1} a(i_1, j_1) b^T(i_1, j_1) = \sum_{i_1, j_1} a^T(i_1, j_1) b(i_1, j_1).$$

Несложно убедиться, что эта сумма содержится в каждом из произведений.

Для нахождения коэффициента перед $c(I, J)$ отделим суммы знаков в выражениях ψ_{T_1, T_2} , ψ_{T_1, T_2}^T на те, для которых $(T_1, J) + (T_2, I) = 0 \pmod{2}$ ($x^I y^J$ коммутирует с $x^{T_1} y^{T_2}$) и $(T_1, J) + (T_2, I) = 1 \pmod{2}$ (не коммутирует). Отметим, что

Рассмотрим вначале случай $k = 1$. Коэффициент перед $a(i_1, j_1) b^T(i_2, j_2)$ входит в член $c(i, j)$ со знаком $(-1)^{j i_2}$. С таким же знаком входит член $(-1)^{ij} a^T(i_1, j_1) b(i_2, j_2)$. Рассмотрим их знаки в выражениях

$$(-1)^{t_1 \alpha + t_2 \beta} \psi_{t_1, t_2}, \quad (-1)^{t_1 \alpha + t_2 \beta + \alpha \beta} \psi_{t_1, t_2}^T$$

как в случае $t_1\alpha + t_2\beta = 0 \pmod 2$, так и в случае $t_1\alpha + t_2\beta = 1 \pmod 2$. Будем суммировать по множеству $\{(t_1, t_2) | t_1\alpha + t_2\beta = 0\}$ в первом случае и по множеству $\{(t_1, t_2) | t_1\alpha + t_2\beta = 1\}$ во втором случае. Если $\alpha = 0 = \beta$ первое множество состоит всем множеством $\{(t_1, t_2)\}$, а второе множество пустое. В противном случае множество разделяется на две половины одинаковой мощности. В нашем случае $(t_1, t_2) = (0, 0), (\beta, \alpha)$. Вычисляя сумму при $\alpha = 0 = \beta$ по первому и второму множеству, т.е. по всем (t_1, t_2) в нашем случае.

$$\sum_{i_1, j_1, i_2, j_2} a(i_1, j_1) b^T(i_2, j_2) \sum_{t_1, t_2} (-1)^{t_1 j_1 + t_2 i_2}$$

получим $2n^2 c(0, 0)$. Когда $(\alpha, \beta) \neq (0, 0)$ коэффициент при $x^\alpha y^\beta$ у произведения вычисляется по формуле:

$$c(\alpha, \beta) = \frac{1}{n^2} \left(\sum_{t_1\alpha + t_2\beta = 0} \psi_{t_1, t_2} + (-1)^{\alpha\beta+1} \sum_{t_1\alpha + t_2\beta = 1} \psi_{t_1, t_2}^T \right). \quad (2.26)$$

Проверим формулу при $k = 1$:

$$\begin{aligned} \psi_{00} &= (a_{00} + a_{01})(b_{00} + b_{10}), & \psi_{00}^T &= (a_{00} + a_{10})(b_{00} + b_{01}), \\ \psi_{01} &= (a_{00} + a_{01})(b_{11} + b_{01}), & \psi_{01}^T &= (a_{11} + a_{01})(b_{00} + b_{01}), \\ \psi_{10} &= (a_{00} - a_{01})(b_{00} + b_{10}), & \psi_{10}^T &= (a_{00} + a_{10})(b_{00} - b_{01}), \\ \psi_{11} &= (a_{00} - a_{01})(b_{11} + b_{01}), & \psi_{01}^T &= (a_{11} + a_{01})(b_{00} - b_{01}), \end{aligned}$$

Эта же формула остается в силе и в многокомпонентном случае $k > 1$, когда произведения в показателе имеют характер скалярного произведения - сумма произведений компонент.

2.8 Заключительные замечания

Последние достижения по умножению больших чисел отражены в

David Harvey, Joris Van Der Hoeven. Integer multiplication in time $O(n \log n)$. Hal <https://hal.archives-ouvertes.fr/hal-02070778> (18 Mar 2019).

Доказательство невозможности более быстрого, чем $O(n \log n)$ операций приведено в

Reyman Afshani, Casper Freksen, Lior Kamma, Kasper Green Larsen. Lower Bounds for Multiplication via Network Coding. arXiv: 1902.10935v1 [cs.DS] 28 Feb 2019

По быстрому умножению матриц первый алгоритм в Strassen V. Gaussian elimination is not optimal. Numer. Math. -1969. – Vol. 13, - P. 354-356.

Обзоры по таким работам имеется в Olga Holtz. Fast and stable matrix multiplication. (доклад)

и (обзор и теория) в

Жданович Д.В. Экспонента сложности матричного умножения. Фундаментальная и прикладная математика, 2011/2012, том 17, №2, с. 107-166.

Основной метод описан в

Coppersmith D., Winograd S. Matrix multiplication via arithmetic progressions. J. Symbol. Comput. -1990. – Vol. 9. P. 251-280.

Последнее уточнение экспоненты умножения до $\alpha = 2.3728639$ в

Francois Le Gall. Powers of Tensors and Fast Matrix Multiplication. 2014 (архив).

Приведенный алгоритм, основанный на методе градуированных вычислений с градуировкой бигрупповой алгебры еще не опубликован. Опубликованы подготовительные материалы. Бигрупповые алгебры в

Айдагулов Р.Р. Бигрупповые алгебры и их автоморфизмы. В эл. журнале Дневник науки №1 2019г. (20стр.)

Айдагулов Р.Р. Бигрупповые алгебры и квантовая комбинаторика. В эл. журнале Дневник науки №1 2019г. (7 стр.)

О градуированных вычислениях опубликовано в

Айдагулов Р.Р. Эффективность вычисления при умножении матриц. Материалы V Международной научно-технической конференции "Современные информационные технологии в образовании и научных исследованиях" (статья в сборнике) 2017г.

Айдагулов Р.Р. Главацкий С.Т. Градуированные вычисления. Современные информационные технологии и ИТ образование, издательство Фонд содействия развитию интернет-медиа, ИТ-образования, человеческого потенциала Лига интернет-медиа (Москва), том 15, №2, стр. 283-289 (2019г.).

Глава 3

Тесты на простоту

Суть RSA состоит в том, что мы не в состоянии вычислить за приемлемое время порядок мультипликативной группы Z_n^* , когда n большое составное число вида $n = pq$ не зная само разложение на простые. В этом случае структура мультипликативной группы имеет вид прямой суммы двух циклических групп:

$$Z_n^* = Z_L \oplus Z_l, \quad L = \text{lcm}(p-1, q-1), \quad l = \text{gcd}(p-1, q-1).$$

Чтобы найти операцию, обратную к $x \rightarrow x^e \bmod n$, надо вычислить обратную $d = e^{-1} \bmod L$. Последнее удастся вычислить только разложив n на множители. Это не под силу и суперкомпьютерам, когда число n порядка 2^{1000} и более. Люди для общения выкладывают в открытом доступе свои числа (n, e) . Другой человек, не желающий открыть тайну переписки с ним свое сообщение зашифровывает по схеме $x \rightarrow x^e \bmod n$, получив сообщение человек, знающий свое секретное число d расшифровывает по схеме $y \rightarrow y^d = x^{de} = x \bmod n$.

Человек может поставить свою электронную подпись в виде $(x_0^d \bmod n, x_0)$, где x_0 открытое имя. Каждый зная открытые числа (n, e) возводя первое число в степень e , по модулю n может удостовериться в подлинности автора подписи (другому не под силу подделать подпись). Такая криптография называется криптографией с открытым ключом и позволяет очень многое. Она базируется на трудной вычислимости d , по открытым данным (n, e) . В математическом плане это связано с алгоритмами проверки на простоту и факторизации больших чисел.

Такая криптография называется криптографией с открытым ключом и позволяет очень многое. Она базируется на трудной вычислимости d , по открытым данным (n, e) . В математическом плане это связано с алгоритмами проверки на простоту и факторизации больших чисел.

Такие приложения в криптографии дали толчок в алгоритмах тестирования на простоту и разложения на простые больших чисел.

3.1 Решето Эратосфена

Решето Эратосфена является простым эффективным методом вычисляющим все простые числа до N . Для этого вначале определяют несколько первых простых чисел $p_1 =$

$2, p_2 = 3, \dots, p_l, M = p_1 p_2 \dots p_l$. Пусть S множество вычетов по модулю M взаимно простых с M . Порождая множество чисел $mM + k, 0 \leq m \leq [N/M], k \in S$ сокращаем тестируемое на простоту множество чисел примерно $a_l = \prod_{1 \leq i \leq l} \frac{p_i}{p_i - 1}$ раз. В частности $a_1 = 2, a_2 = 3, a_3 = \frac{15}{4} = 3.75, a_4 = \frac{35}{8} = 4.375$. Такая предварительная подготовка к рассеиванию уменьшает как количество операций, так и используемую память для нахождения простых. Предварительно всем проверяемым числам дадим оценку $t[m, k] = 1$ простоты со значением 1. Далее по простым $p_i, i > l$ (с оценкой простоты 1) до $\sqrt{N}$ проходим по вычetaм

$$\text{for}(k=0; k < |S|; k++) \{ c = s[k];$$

$$m_0 = -cM^{-1} \mod P_i; a = [N/Mp_i];$$

$$\text{for}(j = 0; j \leq a; j++) \{ t(m_0, k) = 0; m_0 + = p_i; \}$$

}

Этот алгоритм можно слегка модернизировать для подсчета простых чисел из интервала (L, R) . Однако, для больших интервалов этот способ далек от лучших по эффективности. Когда надо только вычислить количество, а не сами простые из этого интервала, существуют более эффективные алгоритмы.

Этот алгоритм легко модернизируется для вычисления B гладких чисел и их подсчета из интервала (L, R) .

Часто нужно не только порождать простые числа, но так же вычислить некоторые характеристики полученных простых чисел. Например, требуется выдать вместо с простыми числами p так же $ord_p(2) = k$ - наименьшее натуральное k , что $p | 2^k - 1$. В таких случаях лучше по ходу тестирования делимости чисел $p_i | n$ при рассеивании проверить (рассеивая) и свойство $n = 1 \mod p_i$. Тогда вместе с простыми q мы получаем так же все простые делители $p | q - 1$, т.е факторизуем все числа $q - 1$ при нахождении простых q . Это существенно сокращает количество операций по сравнению с тем, когда мы каждый раз отдельно факторизовали бы числа $q - 1$.

Алгоритм использует пробное деление по всем простым, не превосходящим $\sqrt{N}$. Такое использование пробных делений эффективно так же для факторизации одного числа $n < 10^8$.

Алгоритм можно использовать так же для тестирования на простоту. Однако, последнее эффективно разве что для чисел меньше 100.

3.2 Тесты Ферма, Эйлера и Рабина-Миллера.

Вероятностные тесты на простоту используют то или иное свойство простых чисел. Четные числа сразу узнаются. Среди них простое число только 2. Поэтому, тесты рассчитаны на проверку нечетного числа на простоту (не все правильно работают при подаче на вход четного числа).

Тест Вильсона $n! = -1 \mod n$.

Это не только необходимое, но и достаточное условие простоты. Однако это требует больших вычислений и не применяется даже для доказательства простоты.

Тест Ферма по основанию a .

$$a^{n-1} = 1 \pmod n.$$

Это полиномиальный (от количества цифр числа n) тест. Если это соотношение выполняется, то число n называется псевдопростым Ферма для основания a . Доказано, что для каждого основания $a \geq 2$ составных чисел $n \leq x$, проходящих тест $o(\pi(x))$. Это означает, что при любом основании отсеиваются очень много составных, не проходя тест на простоту. Тем не менее этот тест считается малоэффективным, так как часто (по сравнению с другими тестами) дает подтверждение простоты (много псевдопростых чисел). Доказано, что существует бесконечно много составных чисел n , которые являются псевдопростыми для любого взаимно простого с n модуля a . Такие числа называются числами Кармайкла. Их еще определяют как числа, для которых выполняется:

$$a^n = a \pmod n, \forall a.$$

Кармайкл определил функцию Кармайкла $c(n) = lcm(p_i^{k_i-1}(p_i - 1))$, $n = \prod_i p_i^{k_i}$ максимальный порядок в мультипликативной группе Z_n^* . Соответственно, n число Кармайкла, если и только если $c(n) | n - 1$.

Имеется теорема о том, что чисел Кармайкла меньше n оценивается снизу формулой:

$$C(x) = \#\{n < x | n - Carmicl's\ number\} > C_0 x^{2/7}.$$

Есть и более сильные оценки. Имеется гипотеза, что их очень много:

$$C(x) > C_\varepsilon x^{1-\varepsilon}.$$

Поэтому этот тест почти не используется.

Тест Эйлера по основанию a .

$$a^{(n-1)/2} = \left(\frac{a}{n}\right) \pmod n.$$

Здесь $\left(\frac{a}{n}\right)$ - символ Якоби. Он мультипликативен по каждому аргументу и

$$\left(\frac{2}{n}\right) = (-1)^{(n^2-1)/8}.$$

Если m, n - нечетные, то

$$\left(\frac{m}{n}\right) = (-1)^{(m-1)(n-1)/4} \left(\frac{n}{m}\right).$$

Эти формулы позволяют считать символ Якоби за полиномиальное время. Обычно проверка производится не очень большим основанием a и тест Эйлера легко проверяется.

Для этого теста нет чисел на подобии чисел Кармайкла. Для каждого основания, псевдопростых Эйлера меньше, чем псевдопростых Ферма. Тем не менее, этот тест так же мало используется. Причиной тому является тест Рабина-Миллера, который несколько проще для проверки и обычно строже.

Тест Рабина-Миллера.

Теорема. Пусть n - простое и $n - 1 = 2^s t$, где число t нечетно. Если число a не делится на n , то

$$\text{or } a^t = 1 \pmod n,$$

$$\text{or exist } i \quad a^{2^i t} = -1 \pmod n, \quad 0 \leq i < s.$$

Такой тест легко проверяется вычислением $b = a^t \pmod n$. Если это число равно $\pm 1 \pmod n$, то n проходит тест на псевдопростоту. Иначе $i++$, пока $i < s$ вычисляем $b = b^2 \pmod n, i++$. Если появляется -1 выходим из цикла с заключением n прошел тест на псевдопростоту. В противном случае заключаем n составное. Псевдопростые (прошедшие тест, но не являющиеся простыми) этого теста называются сильно псевдопростыми по основанию a .

Рабин доказал, что для нечетного $n > 9$ количество оснований сильной псевдопростоты числа n удовлетворяет неравенству:

$$S(n) \leq \frac{1}{4} \phi(n),$$

где $S(n) = \#\{a < n | a - \text{сильно псевдопростое}\}$.

Относительно наименьшего свидетеля $W(n)$ не простоты составного числа n известно следующее:

Теорема. Существует бесконечно много составных чисел n , для которых выполняется:

$$W(n) > (\ln n)^{1/(3 \ln \ln n)}.$$

Таких чисел, не превосходящих x больше $x^{1/(35 \ln \ln x)}$ при достаточно больших x .

Тем не менее доказано, что при справедливости расширенной гипотезы Римана верна

$$W(n) < 2 \ln^2 n.$$

Соответственно, при справедливости этой гипотезы, этот тест позволяет доказать простоту (или составность) числа n за полиномиальное время.

3.3 Круговые многочлены и специальные числа.

Пусть ζ примитивный корень из 1 степени m . Многочлен

$$\Phi_m(x) = \prod_{(i,m)=1} (x - \zeta^i)$$

называется круговым. Степень многочлена $\phi(m)$. Все корни многочлена $\Phi_m(x)$ являются примитивными корнями степени m из 1. Так как

$$x^m - 1 = \prod_{i=1}^m (x - \zeta^i) = \prod_{d|m} \Phi_d(x)$$

многочлен $x^m - 1$ раскладывается в произведение круговых многочленов.

Многочлен $f(x_1, \dots, x_k)$ степени l от k переменных называется однородным, если степень каждого одночлена l . Тогда $f(\lambda x_1, \lambda x_2, \dots, \lambda x_k) = \lambda^l f(x_1, x_2, \dots, x_k)$. С любым многочленом степени l от k переменных (если он неоднороден) можно построить однородный многочлен от $k+1$ переменных $x_1, x_2, \dots, x_k, t$

$$F(x_1, x_2, \dots, x_k, t) = t^l f\left(\frac{x_1}{t}, \frac{x_2}{t}, \dots, \frac{x_k}{t}\right).$$

Так строят (проективное замыкание) и для кругового многочлена

$$\Phi_m(x, y) = \prod_{(i, m)=1} (x - \zeta^i y).$$

Соответственно

$$x^m - y^m = \prod_{d|m} \Phi_d(x, y).$$

Воспользовавшись формулой обращения:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right)$$

в мультипликативной форме:

$$g(n) = \prod_{d|n} f(d) \Leftrightarrow f(n) = \prod_{d|n} g\left(\frac{n}{d}\right)^{\mu(d)}$$

получаем:

$$\Phi_m(x, y) = \prod_{d|m} (x^{m/d} - y^{m/d})^{\mu(d)}.$$

Отсюда, для четного индекса $2m$ получаем:

$$\Phi_{2m}(x, y) = \prod_{d|m, d-\text{odd}} \left(\frac{x^{2m/d} - y^{2m/d}}{x^{m/d} - y^{m/d}} \right)^{\mu(d)} = \prod_{d|m, d-\text{odd}} (x^{m/d} + y^{m/d})^{\mu(d)}.$$

Это позволяет выразить круговые многочлены, имеющие не большое количество простых делителей:

$$\Phi_1(x, y) = x - y, \quad \Phi_2(x, y) = x + y, \quad \Phi_p(x, y) = x^{p-1} + x^{p-2}y + \dots + y^{p-1},$$

$$\Phi_{2^{k+1}}(x, y) = x^{2^k} + y^{2^k}, \quad \Phi_6(x, y) = x^2 - xy + y^2.$$

Ясно, что $\gcd(x, y)^{\phi(m)} | \Phi_m(x, y)$. Поэтому интересны только делители $\Phi_m(x, y)$ для случая взаимной простоты чисел $(x, y) = 1$. Тогда для любого простого делителя p числа $\Phi_m(x, y)$ справедливо $p \nmid x$, $p \nmid y$, и $(x/y) \bmod p$ имеет точный порядок m , т.е. $m|p-1$. Это позволяет обобщить доказательство Эвклида бесконечности простых чисел до бесконечности простых чисел, дающих остаток 1 при делении на m . Достаточно взять $\Phi_m(P, 1)$, $P = p_1 \dots p_k$.

Значения $\Phi_m(x, y)$, $(x, y) = 1$ являются кандидатами на простые множители числа $x^m - y^m$. В частности, при простом $m = p$ такими кандидатами являются числа

$$\Phi_p(x, y) = \frac{x^p - y^p}{x - y}, \quad (x, y) = 1.$$

Их называют обобщенными числами Мерсена. При $x = 2, y = 1$ получаются обычные числа Мерсена, при $x = 2, y = -1$ - числа Вагстафа.

Когда $m = 2^{k+1}$ получаем

$$\Phi_{2^{k+1}}(x, y) = x^{2^k} + y^{2^k}$$

получаем обобщенные числа Ферма. Обычные числа Ферма при $x = 2, y = 1$. Одна теорема Гауса гласит, что круг можно разделить на нечетное простое число частей (или построит правильный многоугольник такого порядка) тогда и только тогда, когда это простое число Ферма. Такими являются 3, 5, 17, 257, 65537. По завещанию Гаусса его могила имеет форму правильного 17 угольника.

3.4 Последовательности Люка.

Для некоторых специальных чисел имеется более эффективный тест точно определяющий (не вероятностный) простое число или нет. Для чисел Мерсена это тест Люка. Тест Люка основан на числах Люка, которые определяются рекуррентной последовательностью второго порядка

$$L_{k+1} = aL_k - bL_{k-1}, \quad a, b \in Z.$$

Эта последовательность часто рассматривается над кольцом Z_n или над кольцом многочленов по модулю некоторого многочлена $Z[x]/(f(x))$. Любое решение с начальными условиями:

$$L_0 = C_0, L_1 = C_1$$

может быть представлено в виде линейной комбинации двух фундаментальных решений рекуррентного соотношения второго порядка:

$$U_k = U_k(a, b) = \frac{x^k - (a - x)^k}{x - (a - x)}, \quad U_0 = 0, U_1 = 1.$$

$$V_k = V_k(a, b) = x^k + (a - x)^k, \quad V_0 = 2, V_1 = a.$$

Здесь x корень уравнения $x^2 - ax + b = 0$, $(x(a - x) = b)$.

Тогда $L_k = \alpha U_k + \beta V_k$, $\beta = C_0/2$, $\alpha = C_1 - a\beta$.

Когда $\Delta = a^2 - 4b$ не является квадратом в Z (или Z_n), члены последовательности удобнее рассматривать как элементы квадратичного расширения. Так получают периоды последовательности над конечным кольцом.

Часто используемым случаем является случай $a = 1, b = -1$. В этом случае последовательность U_k определяет числа Фибоначчи. Последовательность Люка с $a, b \in Z$ остается ограниченным только в случае $a = \pm 1, b = 1$ или в случае $ab = 0, a + b = \pm 1, 0$.

Последовательности вычисляются почти так же (полиномиально) быстро, как и степени через удвоения индекса:

$$U_{2k} = U_k V_k, \quad V_{2k} = V_k^2 - 2b^k,$$

$$U_{2k+1} = U_k V_{k+1} + b^k, \quad V_{2k+1} = V_k V_{k+1} - ab^k.$$

Можно выразить и общие последовательности Люка через удвоение индекса. По сути эти формулы являются следствием вычисления степени матрицы A через удвоения индекса:

$$\begin{pmatrix} L_{k+1} \\ L_k \end{pmatrix} = A \begin{pmatrix} L_k \\ L_{k-1} \end{pmatrix}, \quad A = \begin{pmatrix} a & -b \\ 1 & 0 \end{pmatrix}.$$

3.5 Псевдопростые числа Фибоначчи и Люка.

Последовательность Фибоначчи является частным случаем последовательности Люка. Поэтому в дальнейшем эта последовательность будет рассматриваться как важный пример последовательности Люка.

Имеет место теорема. Если p простое число, не делящее $2b\Delta$, то

$$p \mid U_{p - (\frac{\Delta}{p})}.$$

Это необходимое условие простоты для чисел $(n, 2b\Delta) = 1$ лежит в основе теста Люка на простоту. Нечетное число n взаимно простое с $2b\Delta$ проходит тест Люка на простоту с данными (a, b) , если

$$n | U_{n-(\frac{\Delta}{n})}.$$

Составные числа, проходившие тест на простоту называются псевдопростыми по основанию (a, b) .

Обычно, в случае $(\frac{\Delta}{n}) = -1$ отсеиваются больше составных чисел. Тест Люка и псевдопростоты Люка для случая $a = 1, b = -1, \Delta = 5$ называются тестом Фибоначчи и псевдопростотами Фибоначчи. Пока не найдено ни одного нечетного числа $n = \pm 2 \pmod{5}$, для которого проходит тест по основанию 2 и тест Фибоначчи.

3.6 Тест Грехэма-Фробениуса

Тест Люка, определенный на рекуррентных последовательностях второго порядка, соответствует вычислению мультипликативной группы квадратичного расширения. По аналогии можно рассмотреть последовательности более высокого порядка или конечные расширения кольца Z_n и считать порядок мультипликативной группы, когда n простое.

3.6.1 Тест Фробениуса

В тесте Люка используется не вся информация от

$$x^n = \begin{cases} a - x \pmod{(f(x), n)}, & (\frac{\Delta}{n}) = -1 \\ x, & \pmod{(f(x), n)}, (\frac{\Delta}{n}) = 1. \end{cases}$$

В тесте Люка использовали только делимость $n | U_{n-(\frac{\Delta}{n})}$. Псевдопростота по Фробениусу определяется обеими последовательностями Люка:

$$U_{n-(\frac{\Delta}{n})} = 0 \pmod{n}, \quad V_{n-(\frac{\Delta}{n})} = \begin{cases} 2b, & (\frac{\Delta}{n}) = -1 \\ 2, & (\frac{\Delta}{n}) = 1. \end{cases}$$

Этот тест более строгий, чем тест Люка. Наименьшее псевдопростое число Фробениуса по отношению к $x^2 - x - 1$ равно 4181 (девятнадцатое число Фибоначчи), а первым псевдопростым с условием $(\frac{\Delta}{n}) = -1$, $\Delta = 5$ является 5777. Для многочлена $x^2 + 5x + 5$ не известно ни одного примера псевдопростого числа Фробениуса с условием $(\frac{\Delta}{n}) = -1$.

С вычислительной точки зрения тест можно реализовать так, чтобы он был только (примерно) в 3 раза сложнее, чем тест псевдопростоты, а тест Люка в 2 раза сложнее теста Ферма. Суть сводится к вычислению степени матрицы. Одно из реализаций использование формул удвоения через функцию V , выразив U через него:

$$U_m = \Delta^{-1}(2V_{m+1} - aV_m),$$

$$V_{j+k} = V_j V_k - b^j V_{k-j}.$$

При $b = 1$ отсюда получаются простые формулы удвоения:

$$V_{2j} = V_j^2 - 2, \quad V_{2j+1} = V_j V_{j+1} - a.$$

Общий случай сводится к этому следующим образом. Последовательность с четными индексами соответствует

$$V_{2m}(a, b) = V_m(a^2 - 2b, b^2) = b^m V_m(A, 1), \quad A = a^2 b^{-1} - 2.$$

Отсюда получается тест для $(n, 2ab\Delta) = 1$ в виде:

$$AV_m(A, 1) \equiv 2V_{m+1}(A, 1) \pmod{n}, \quad m = \frac{1}{2}(n - (\frac{\Delta}{n})),$$

$$b^{(n-1)/2} V_m(A, 1) \equiv 2 \pmod{n}.$$

Первая часть тест Люка, вместе тест Фробениуса.

3.6.2 Сильно псевдопростые Люка, Фробениуса

Псевдопростота рассматривается только в случае, когда Δ не является квадратом целого числа и многочлен отличен от $x^2 \pm x + 1$. Доказано, что псевдопростых чисел много как Люка, так и Фробениуса. Правда для теста Фробениуса доказано только для случая $(\frac{\Delta}{n}) = 1$ и в частном случае $(\frac{\Delta}{n}) = -1, f(x) = x^2 - x - 1$.

По аналогии с сильной псевдопростотой, соответствующей тесту Рабина-Миллера для теста Ферма, определяется сильная псевдопростота Люка и Фробениуса, являющиеся аналогом теста Рабина-Миллера по отношению к этим тестам.

Из автоморфизма Фробениуса получается, что для простого n в кольце $R = Z_n[x]/(f(x))$ выполняется:

$$(x(a-x)^{-1})^{n-(\frac{\Delta}{n})} = 1.$$

Из $(x(a-x)^{-1})^{2m} = 1$ следует $(x(a-x)^{-1})^m = \pm 1$ (доказательство получается из рассмотрения двух случаев $(\frac{\Delta}{n}) = \pm 1$, учитывая, что в поле не более двух корней у уравнения $x^2 = 1$).

Записав $n - (\frac{\Delta}{n}) = 2^s t$, $t - odd$ получаем, что для простого n не делящего $2b\Delta$ или

$$(x(a-x)^{-1})^t = 1$$

или существует $0 \leq i < s$, что выполняется

$$(x(a-x)^{-1})^{2^i t} = -1.$$

Это означает, что

$$U_t \equiv 0 \pmod{n}$$

или

$$V_{2^i t} \equiv 0 \pmod{n}.$$

Выполнение этого условия для нечетного не простого n , такого, что $(n, b\Delta) = 1$, определяет сильную псевдопростоту Люка. Из сильной псевдопростоты Люка следует псевдопростота Люка (так же очевидно, как из сильной псевдопростоты следует псевдопростота).

Для квадратичного расширения Z_n (n - простое) для любого отличного от нуля элемента

$$x^{n^2-1} = 1.$$

Пусть $n^2 - 1 = 2^S T$, T — нечетное. Тогда или

$$x^T = 1$$

или

$$x^{2^i T} = -1, \quad 0 \leq i < S.$$

Если этот тест выполняется для псевдопростого числа Фробениуса, то такое число называется сильно псевдопростым числом Фробениуса. Показано, что таких чисел встречается еще меньше.

Алгоритм проверки на сильную псевдопростоту Люка (и Фробениуса) сложнее проверки сильной псевдопростоты примерно в два (и три) раза.

Глава 4

Доказательство простоты.

4.1 $n - 1$ метод.

Этот метод основан на проверке того, что мультипликативная группа Z_n^* циклическая и имеет порядок $n - 1$. Люка доказал следующую теорему:

Теорема Люка. Пусть a, n натуральные числа больше 1. Если

$$a^{n-1} \equiv 1 \pmod{n},$$

но

$$a^{(n-1)/q} \not\equiv 1 \pmod{n}$$

для любого простого делителя q числа $n - 1$, то число n простое.

Из этих условий следует, что (мультипликативный) порядок числа a равен $n - 1$, т.е. $\phi(n) \geq n - 1$. Так как для составного числа n функция Эйлера $\phi(n) < n - 1$, число n не составное, т.е. простое.

Число a , для которого выполняется условия теоремы будет образующим в циклической группе Z_n^* . При $n > 31\# + 1$ (через $x\#$ обозначают произведение всех простых, не превосходящих x) для простого n имеется не менее $n/(2 \ln \ln n)$ образующих.

Нахождение образующей и факторизация числа $n - 1$ может быть очень сложной. В некоторых специальных случаях это приводит к простым алгоритмам, доказывающую или опровергающую простоту за малое количество операций. Такой ситуацией является проверка чисел Ферма $n = 2^{2^k} + 1$ на простоту. В этом случае $n - 1$ является степенью 2, т.е. достаточно найти хотя бы одно число a , такое что

$$a^{(n-1)/2} \not\equiv 1 \pmod{n}.$$

Таким числом может быть любое, удовлетворяющее условию:

$$\left(\frac{a}{n}\right) = \left(\frac{2^{2^k} + 1}{a}\right) = -1.$$

Последнее выполняется для $a = 3(k > 0), 5(k > 1), 7(k - \text{even}, k > 0), \dots$ В частности, в тесте Пепена берется $a = 3$ и вычисляется квадраты:

```
X=a;i=1;
while(i<k){ X*=X;X%=n;}
```

```

if(X+1==n) cout<<"n - is prime";
else cout<<"n- is not prime";

```

Однако, числа Ферма быстро растут и таким образом не удалось доказать простоту не для одного из них, кроме известных еще со времен Ферма $k = 0, 1, 2, 3, 4$. А при не больших $k > 4$ оказалось проще доказать, что они не простые, найдя их делители, за исключением некоторых (составность F_{24} было доказано тестом Пепена). В частности Эйлер нашел делитель 641 у пятого числа Ферма $2^{2^5} + 1$. А при $k > 30$ даже эффективный тест Пепена становится ресурсоемкой (требует слишком большого машинного времени) даже для самых быстрых суперкомпьютеров.

4.1.1 Частичное разложение $n - 1$.

Часто удается найти только частичное разложение числа $n - 1 = FR$, где для F известны все простые делители, а для R не известны. Справедлива следующая теорема:

Пусть

$$a^{n-1} \equiv 1 \pmod{n}.$$

Если при этом для любого простого делителя $q|F$ верно:

$$a^{(n-1)/q} \not\equiv 1 \pmod{n}$$

то любой простой делитель p числа n удовлетворяет условию $F|p-1$. Таким образом, n в F -ичной системе исчисления имеет разложение $n = 1 + c_1F + c_2F^2 + \dots, 0 \leq c_i < F$.

Доказательство следует из того, что для числа $b = a^R \pmod{n}$ мультипликативный порядок равен F для любого простого делителя $p|n$.

В частности, отсюда вытекает следствие:

если при этом $F \geq \sqrt{n}$, то n простое (не может иметь два простых делителя больше $\sqrt{n}$).

Если $n^{1/3} \leq F < \sqrt{n}$, то или n простое, или $n = pq, p = aF + 1, q = bF + 1 \Rightarrow n = abF^2 + (a + b)F + 1$. Легко оценивается $ab < n/F^2 \rightarrow ab < F \rightarrow a + b \leq F$. Исключая возможность $p = F + 1, q = F^2 - F + 1$ получаем $c_2 = ab, c_1 = a + b, c_1^2 - 4c_2 = (a - b)^2$. Соответственно n - простое, если $n = 1 + c_1F + c_2F^2$ и $c_1^2 - 4c_2$ не квадрат.

Конягин и Померанс нашли условие (необходимое и достаточное) простоты числа n и для случая $n^{3/10} \leq F < n^{1/3}$.

4.2 $n + 1$ метод.

В основе $n + 1$ метода используется факт, что у квадратичного расширения Z_n при простом n мультипликативная группа имеет порядок $n^2 - 1 = (n - 1)(n + 1)$. Для расширений k -ой степени $M_k = \frac{n^k - 1}{n - 1} = n^{k-1} + \dots + n + 1$. Соответственно можно исследовать и такие обобщения. Однако, тут появляется трудно разложимое (как правило) еще большее чем n число M_k . Поэтому такой подход носит скорее теоретический характер и после создания полиномиального теста, доказывающего простоту, интерес к таким тестам с $k > 2$ ослаб.

Такой тест удобен, когда хорошо разлагается число $n + 1$. Например, при проверке чисел Мерсенна $n = 2^p - 1$, число $n + 1$ является степенью двойки.

4.2.1 Тест Люка-Лемера.

Пусть $a, b \in Z$, рассмотрим

$$f(x) = x^2 - ax + b, \Delta = a^2 - 4b$$

и последовательности Люка

$$U_k = \frac{x^k - (a - x)^k}{x - (a - x)} \mod f(x), \quad V_k = x^k + (a - x)^k \mod f(x).$$

Пусть натуральное число n удовлетворяет условию $(n, 2b\Delta) = 1$. Рангом появления числа n называется наименьшее натуральное число r такое, что $n|U_r$. Обоснованием такого определения является то, что при выполнении приведенного выше условия на n такие r , что $n|U_r$ всегда существуют и из $i|j \rightarrow U_i|U_j$ следовательно для всех кратных kr $n|U_{kr}$. Мало того, из $n|U_m \rightarrow r|m$. Действительно, пусть $m = kr + s, s < r$, тогда

$$2U_m = \frac{(x^{kr} + (a - x)^{kr})(x^s - (a - x)^s)}{x - (a - x)} + \frac{(x^{kr} - (a - x)^{kr})(x^s + (a - x)^s)}{x - (a - x)} = V_{kr}U_s + U_{kr}V_s.$$

Так как при $n|U_{kr}$ число V_{kr} не делится на n и даже взаимно просто с n при указанных условиях $((n, 2b\Delta) = 1)$. Следовательно, при $0 < s < r$ число U_m не делится на n .

Ранее показали, что для простого числа p выполняется $p|U_{p-(\frac{\Delta}{n})}$, что означает $r_p|p-(\frac{\Delta}{n})$.

По аналогии с $n - 1$ методом, имеется теорема:

Теорема 8. Пусть n - натуральное число, удовлетворяющее условиям $(n, 2b\Delta) = 1, (\frac{\Delta}{n}) = -1$ и $n + 1 = FR$. Если

$$n|U_{n+1}, (U_{(n+1)/q}, n) = 1 \quad \forall q|F$$

для всех простых делителей q числа $F|n + 1$, то для любого простого делителя $p|n$ выполняется соотношение $F|p-(\frac{\Delta}{n})$. Если при этом $F > \sqrt{n} + 1$, то n простое.

Если F является четным делителем числа $n + 1$, и

$$V_{F/2} \equiv 0 \mod n, \gcd(V_{F/2q}, n) = 1 \quad \forall q|F$$

q нечетный простой делитель F . Если $F > \sqrt{n} + 1$, тогда n простое.

Доказательство. По аналогии с $n - 1$ методом, для любого простого делителя $p|n$ ранг r_p делится на F , следовательно $F|p-(\frac{\Delta}{p})$. Когда $F > \sqrt{n} + 1$, у n не может быть два простых делителя.

Рассмотрим теперь вторую часть с четным F . Из $V_{F/2} \equiv 0 \mod n$ следует, что для любого простого делителя $p|n$ ранг r_p делит F . Если $r_p|F/2$, то $p|U_{F/2}, p|V_{F/2}$, что приводит к соотношению $p|b^{F/2} \rightarrow p|b$, что противоречит условию $(n, 2b\Delta) = 1$. Аналогично из $p|U_{F/2q} \rightarrow p|U_{F/2q}|U_{F/2}$ получаем противоречие. Таким образом, $\forall p|n \quad F|r_p$, что приводит к доказанной первой части. □

Имеются так же аналоги теорем с ограничениями $n^{1/3} + 1 < F \leq \sqrt{n}$ и $n^{3/10} + 1 < F \leq n^{1/3}$. Мы здесь рассмотрим только как приложение тест для чисел Мерсенна.

Теорема 9. Пусть p – нечетное простое число, $v_0 = 4$, $v_{k+1} = v_k^2 - 2$. Число $M_p = 2^p - 1$ простое тогда и только тогда, когда $v_{p-2} \equiv 0 \pmod{M_p}$.

Доказательство. Пусть $f(x) = x^2 - 4x + 1$, тогда $\Delta = 12$. Так как $M_p \equiv 3 \pmod{4}$, $M_p \equiv 1 \pmod{3}$, то $(\frac{\Delta}{M_p}) = -1$. Заметим, что $v_i = V_{2^i} \pmod{n = M_p}$. Учитывая, что $(\frac{\Delta}{M_p}) = -1$ получаем, что в случае простоты числа Мерсена выполняется:

$$x_1^{M_p} = x_2 \pmod{M_p}, x_1 x_2 = 1 \rightarrow x_1^{(M_p+1)/2} = x_2^{(M_p+1)/2} \pmod{M_p}.$$

Т.е. $(\frac{x_1}{x_2})^{2^{p-1}} = 1$.

Применим предыдущую теорему с числом $F = 2^{p-1}$ и получим простоту n в случае $v_{p-2} \equiv 0 \pmod{M_p}$. Если при простом M_p $(\frac{x_1}{x_2})^{2^{p-2}} = 1 \pmod{M_p}$, то приходим к противоречию с условием $(\frac{\Delta}{M_p}) = -1$. Это доказывает необходимость. $\square$

4.3 Тест на простоту Агравала, Кайала и Саксены (AKS - тест).

Первый доказанный полиномиальный тест на простоту для произвольных чисел n создан этими авторами в 2002 г. В 2004 г. они опубликовали несколько усовершенствованный вариант.

4.3.1 Проверка на простоту с помощью корней из единицы.

Если n - простое, то

$$g(x)^n \equiv g(x^n) \pmod{n}$$

для любого многочлена $g(x) \in Z[x]$. В частности,

$$(x + a)^n \equiv x^n + a \pmod{n} \quad \forall a \in Z. \quad (4.1)$$

Если это соотношение выполняется хотя бы для одного a взаимно простого с n , то n простое (если n не простое, то найдется биномиальный коэффициент с положительным номером $k < n$, что $n \nmid C_n^k$).

Однако, проверять выполнение (4.1) сложно из-за высокой степени многочлена. Проще проверять следствие:

$$(x + a)^n \equiv x^n + a \pmod{(f(x), n)} \quad \forall a \in Z \quad (4.2)$$

когда степень многочлена $f(x)$ не большая. Например, при $f(x) = x - 1$ это соотношение эквивалентно

$$(a + 1)^n \equiv (a + 1) \pmod{n}$$

сравнению Ферма. При $f(x) = x^r - 1$ это соотношение более строгое, для $r = 2$ это примерно соответствует тесту Люка. Приведем соответствующий алгоритм AKS:

Алгоритм AKS. 1. Проверка на степень.

Если n является квадратом или более высокой степенью, выдать **n- составное**.

2. Начальная установка.

Найти наименьшее целое число r , такое, что порядок числа n в группе Z_r^* превосходит $\log^2 n$.

Если n имеет собственный делитель в промежутке $[2, \sqrt{\phi(r)} \log n]$, выдать **n- составное**.

3. Биномиальное сравнение.

$$\text{for}(1 \leq a \leq \sqrt{\phi(r)} \log n) \{ \\ if(x+a)^n \not\equiv x^n + a \pmod{(x^r - 1, n)}$$

выдать **n- составное**. }

выдать **n- простое**.

Алгоритм базируется на следующей теореме:

Теорема 10. Пусть $n > 2$ - целое число, r - натуральное число, взаимно простое с n , такое, что порядок числа n в группе Z_r^* больше, чем $\log^2 n$ и сравнение

$$(x+a)^n \equiv x^n + a \pmod{(x^r - 1, n)} \quad (4.3)$$

выполнено для каждого целого числа a , такого, что $0 < a \leq \sqrt{\phi(r)} \log n$. Если n имеет простой делитель $p > \sqrt{\phi(r)} \log n$, то $n = p^m$ при некотором натуральном m . В частности, если n не имеет простых делителей в промежутке $[1, \sqrt{\phi(r)} \log n]$, и n не является точной степенью, то n простое.

Доказательство. Пусть $p|n$ простой делитель и $p > A = \sqrt{\phi(r)} \log n$. В теореме AKS в качестве многочлена $f(x)$ взят многочлен $f(x) = x^r - 1$. Однако, доказательство в дальнейшем обобщено и для произвольных многочленов $f(x) \in Z_n(x)$ удовлетворяющих условию $f(x^n) \equiv 0 \pmod{f(x)}$. Для неприводимого многочлена ее корни расширяют основное поле до поля, являющейся алгебраическим расширением степени d , где выполняется соотношение $x^{n^d} \equiv x \pmod{f(x)}$. Здесь это свойство считается выполненным по определению $f(x)$. К тому же, считаем, что $x^{n^{d/q}} - x$ взаимно прост с $f(x)$ при любом простом делителе $q|d$, d — степень многочлена $f(x)$.

Доказательство разобьем на несколько логических частей.

Множество многочленов, для которых выполняется сравнение Определим множество многочленов G (для заданного многочлена $f(x) \in Z_n[x]$):

$$G = \{g(x) \in Z_p[x] : g(x)^n \equiv g(x^n) \pmod{(f(x))}\}.$$

Здесь p - простой делитель числа n (возможно $p = n$). 1. Множество G замкнуто относительно умножения:

$$g_1(x) \in G, g_2(x) \in G \Rightarrow g_1(x)g_2(x) \in G.$$

2. Если $f(x^n) \equiv 0 \pmod{f(x)}$, то G состоит из классов вычетов по модулю $f(x)$:

$$g_1(x) \in G, g_2(x) \equiv g_1(x) \pmod{f(x)} \Rightarrow g_2(x) \in G.$$

Действительно

$$\begin{aligned} g_2(x^n) &\equiv g_1(x) \pmod{f(x^n)} \Rightarrow g_2(x^n) \equiv g_1(x^n) \pmod{f(x)} \\ &\equiv g_1(x)^n \pmod{f(x)} \Rightarrow g_2(x)^n \pmod{f(x)} \Rightarrow g_2(x) \in G. \end{aligned}$$

Множество степеней, для которых выполняется соотношение

Определим множество степеней:

$$J = \{j \in \mathbb{Z} : j > 0, g(x)^j \equiv g(x^j) \pmod{f(x)} \forall g(x) \in G\}.$$

Множество J замкнуто относительно умножения и содержит $1, p, n$.

Отображение множество многочленов в поле

Пусть $h(x)$ неприводимый делитель многочлена $f(x)$ как многочлена над Z_p . Пусть ζ корень $h(x)$ в поле разложения K . Это поле является расширением Z_p и является образом кольца $Z_p[x]/(f(x))$, $x \rightarrow \zeta$. Из условия взаимной простоты $(x^{n^{d/q}} - x, f(x))$ следует $\zeta \neq 0$ и $\zeta^{n^d-1} = 1$. Следовательно порядок элемента n в мультипликативной группе Z_r^* в точности равен d - степени многочлена $f(x)$. В исходном алгоритме AKS число r задано через многочлен $f(x) = x^r - 1$ и число d получается как $d = \phi(r)$. А в обобщенной, задано число d - степень многочлена, а r произвольное число относительно которого число n имеет точный порядок r , т.е. $n^k = 1 \pmod{r} \Rightarrow d|k$.

Обозначим через $\bar{G}$ образ множество G в поле K :

$$\bar{G} = \{\gamma \in K : \gamma = g(\zeta), g \in G\}.$$

Для любого $j \in J$ выполняется $f(\zeta^j) \equiv f(\zeta)^j = 0$. Следовательно ζ^j так же корень многочлена. Так как количество корней не превосходит степени многочлена, то любой элемент $j \in J$ представляется в виде $j = n^i \pmod{r}$, где r (мультипликативный) порядок элемента ζ в конечном поле $F_p(\zeta)$. В частности $p = n^i \pmod{r}$.

Подмножество G , для которого ограничение отображения в поле - инъективно

Положим

$$G_d = \{g(x) \in G : \deg(g(x)) < d \text{ } (\deg(0) = -\infty)\}.$$

Покажем, что сужение отображения из $G \rightarrow K$ на G_d является инъекцией. Пусть $g_1(x), g_2(x) \in G_d$ и $g_1(\zeta) = g_2(\zeta)$. Отсюда следует, что $g_1(\zeta^j) = g_1(\zeta)^j = g_2(\zeta)^j = g_2(\zeta^j)$. Так как $g_1(x) - g_2(x)$ имеющий степень меньше d имеет не меньше d корней, то $g_1(x) \equiv g_2(x)$, т.е. многочлены совпадают и отображение $G_d \rightarrow K$ биекция. Различным многочленам соответствуют различные элементы поля.

Рассмотрим многочлены

$$g(x) = \prod_{0 \leq a \leq \sqrt{d} \log n} (x + a)^{\epsilon_a},$$

где каждое из ϵ_a равно 0 или 1. Многочлен, соответствующий случаю $\epsilon_a = 1 \forall a$ заменим на нулевой, имеющий меньшую степень. **Тогда количество различных многочленов равно**

$$2^{[\sqrt{d}\log n]+1} > 2^{\sqrt{d}\log n} = n^{\sqrt{d}}.$$

Таким образом, можно подвести (предварительный) итог:

$$\#\bar{G} \geq \#G_d > n^{\sqrt{d}}.$$

Вспомним, что

Модулярность множество степеней, для которых выполняется сравнение в $Z_p[x]$

Пусть

$$J' = \{0 \leq j < p^k - 1 : \text{exist } j_0 \in J : j \equiv j_0 \pmod{p^k - 1}\}$$

множество, состоящее из вычетов элементов J по модулю $p^k - 1$. Множество J' так же замкнуто относительно умножений (модулярных), содержит n, p , следовательно и $np^{k-1} \equiv n/p \pmod{p^k - 1}$. Для любого элемента $j \in J'$ выполняется $g(\zeta)^j = g(\zeta^j)$.

Получение уравнения $y^{j_1} = y^{j_2}$, имеющего в поле K решений больше, чем степень. Рассмотрим целые числа $p^a(n/p)^b$, где a, b целые числа из промежутка $[0, \sqrt{d}]$. Поскольку имеется более чем d вариантов выбора таких пар (a, b) , то должны существовать две различные пары $j_1 = p^{a_1}(n/p)^{b_1}, j_2 = p^{a_2}(n/p)^{b_2}$ сравнимые по модулю r . Следовательно

$$g(\zeta)^{j_1} = g(\zeta^{j_1}) = g(\zeta^{j_2}) = g(\zeta)^{j_2} \quad \forall g(x) \in G.$$

Другими словами

$$y^{j_1} - y^{j_2} = 0$$

имеет корней больше чем $n^{\sqrt{d}}$.

Однако $j_1, j_2 \leq p^{\sqrt{d}}(n/p)^{\sqrt{d}} = n^{\sqrt{d}}$, что возможно только при тождественно нулевом многочлене, т.е. при $j_1 = j_2$. Соответственно из $p^{a_1}(n/p)^{b_1} = j_1 = j_2 = p^{a_2}(n/p)^{b_2}$ получаем, что

$$n^{b_1-b_2} = p^{b_1-b_2-a_1+a_2}.$$

Итоговое заключение

Из однозначности разложения на простые и различности пар получаем, что n является степенью простого числа p .

□

4.3.2 Сложность алгоритма

Теорема 11. Пусть дано целое число $n \geq 3$ и пусть r -наименьшее целое число, такое, что порядок элемента n в подгруппе Z_r^* превосходит $\log^2 n$. Тогда $r \leq \log^5 n$.

Доказательство. Пусть r_0 наименьшее простое число, которое не делит число

$$N = n(n-1)(n^2-1)\dots(n^{\lfloor \log^2 n \rfloor} - 1).$$

Тогда порядок n по модулю r_0 превосходит $\log^2 n$, следовательно $r \leq r_0$. Из неравенства Чебышева

$$\prod_{p \leq x} p > 2^x, \text{ for } x \geq 31/$$

Оцениваем N :

$$N < n^{1+1+2+\dots+\lfloor \log^2 n \rfloor} < n^{\log^4 n} = 2^{\log^5 n}.$$

Значит при $\log^5 n \geq 31$ существует такой r_0 . Последнее неравенство выполняется при $n \geq 4$. А для $n = 3$ легко находится $r = 5$. $\square$

Основываясь на этой теореме вначале сложность алгоритма AKS оценивалось как $O(\log^{16.5})$. Далее было показано, что для большинства целых сложность оценивается как $O(\log^6 n)$. Экспериментальная (эмпирическая) оценка $\approx 1000 \log^6 n$.

4.4 Квартичный тест на простоту

В этом тесте рассматривается одно фиксированное сравнение по множеству различных многочленов:

Теорема 12. Пусть n, r, b -натуральные числа, такие, что $n > 1, r|n-1, r > \log^2 n, b^{n-1} \equiv 1 \pmod n$ и $\gcd(b^{(n-1)/q} - 1, n) = 1 \forall q|r$. Если

$$(x-1)^n \equiv x^n - 1 \pmod{x^r - b, n},$$

то n является простым или степенью простого числа.

Доказательство. Пусть простое $p|n$ и пусть $A = b^{(n-1)/r} \pmod p$. Тогда A имеет порядок r в Z_p^* , так что, в частности $r|p-1$. Заметим, что

$$x^n = x(x^r)^{(n-1)/r} \equiv Ax \pmod{x^r - b, p}.$$

Следовательно

$$(x-1)^n \equiv x^n - 1 \equiv Ax - 1 \pmod{x^r - b, p}.$$

С учетом этого

$$(x-1)^{n^2} \equiv (Ax-1)^n \equiv A^2x - 1 \pmod{x^r - b, p}.$$

По индукции

$$(x-1)^{n^j} \equiv A^j x - 1 \pmod{x^r - b, p}.$$

По условиям теоремы A имеет точный порядок r как по модулю числа n , так и по модулю его простых делителей p . Следовательно $r|p-1$. Поэтому

$$x^p = x(x^r)^{(p-1)/r} \equiv b^{(p-1)/r} x \equiv A^k \pmod{(x^r - b, p)}.$$

Соответственно

$$(x-1)^{p^i n^j} \equiv (A^j x - 1)^{p^i} \equiv A^j x^{p^i} - 1 \equiv A^{j+ik} x - 1 \pmod{(x^r - b, p)}.$$

Для всех натуральных i, j получаем

$$(x-1)^{p^i(n/p)^j} \equiv A^{j(1-k)+ik} x - 1 \pmod{(x^r - b, p)}.$$

Заметим, что $x-1$ является единицей в кольце $Z_p[x]/(x^r - b)$. Действительно $\gcd(x-1, x^r - b) = \gcd(x-1, 1-b)$. Это верно, если $\gcd(x-1, 1-b) = 1$. Поскольку порядок b по модулю p больше 1, то

Обозначим через E (мультипликативный) порядок элемента $x-1$ в кольце $Z_p[x]/(x^r - b)$. Заметим, что

$$E \geq 2^r - 1$$

поскольку многочлены

$$\prod_{j \in S} (A^j x - 1),$$

где S пробегает собственные подмножества чисел $\{0, 1, \dots, r-1\}$ различны и каждый из них является степенью многочлена $x-1$.

Далее как прежде должны существовать две различные пары $(i_1, j_1), (i_2, j_2), 0 \leq i_l, j_l \leq \sqrt{r}$, такие, что

$$j_1(1-k) + i_1 k \equiv j_2(1-k) + i_2 k \pmod{(r)},$$

так что если $u_1 = p^{i_1}(n/p)^{j_1}, u_2 = p^{i_2}(n/p)^{j_2}$, то

$$(x-1)^{u_1} \equiv (x-1)^{u_2} \pmod{(x^r - b, p)}.$$

Следовательно $u_1 \equiv u_2 \pmod{(E)}$. Но $u_1, u_2 \leq n^{\sqrt{r}}$ и $E > 2^r - 1 > n^{\sqrt{r}} - 1$ ($r > \log^2 n$). Следовательно $u_1 = u_2$ и $n = p^c$. $\square$

У этой теоремы есть слабая сторона. Даже если n простое, в большинстве случаев у $n-1$ простые делители меньше $\log^2 n$. Поэтому задачу свели относительно простых делителей числа $n^d - 1, d < (2 \log \log n)^{c \log \log(\log^2 n)}$ с ограничениями $d^2 \log^2 n < r \leq (d+1)d^2 \log^2 n$. Следующий результат (Bernstein, Mihailescu, Avanzi) позволяет построить быстро проверяемое условие простоты.

Теорема 13. Пусть n, r, d - целые числа, $n > 1, r|n^d - 1, r > d^2 \log^2 n$. Пусть также $f(t)$ - нормированный многочлен из $Z_n[t]$ степени d , R есть кольцо $Z_n[t]/(f(t))$, и пусть элемент $b = b(t) \in R$ такое, что $b^{n^d-1} = 1$, и $(b^{(n^d-1)/q} - 1, f(t)) = 1$ ¹ для каждого простого $q|r$. Если

$$(x-1)^{n^d} \equiv x^{n^d} - 1 \pmod{(x^r - b)}$$

в кольце $R[x]$, то n - простое или степень простого числа.

¹В формулировке книги ошибка в условии (совпадает с 0 в кольце R)

Доказательство. Доказательство схоже с доказательством предыдущей теоремы. Пусть p - простой делитель числа n и $h(t)$ - неприводимый делитель многочлена $f(t)$ по модулю p . Пусть K есть конечное поле $Z_p[t]/(h(t))$, так что K есть гомоморфный образ кольца R . Положим $N = n^d$, $P = p^{\deg(h(t))}$. Тогда $P|p^d|N$. отождествим b с его образом в K и положим $A = b^{(N-1)/r}$, так что по условию теоремы A имеет точный порядок r . Тогда существует некоторое целое число k , такое, что для всех неотрицательных i, j выполняется:

$$(x-1)^{N^j} \equiv A^j x - 1 \pmod{(x^r - b)}, \quad (x-1)^{P^i} \equiv A^{ik} x - 1 \pmod{(x^r - b)},$$

где многочлены рассматриваются как элементы кольца $K[x]$. Далее доказательство полностью совпадает с доказательством предыдущей теоремы. $\square$

Алгоритм (кватричный вариант AKS- теста). Нам задано целое число $n > 1$. Этот вероятностный алгоритм позволяет решить, является число n простым или составным, и в случае завершения выдает правильный ответ.

1. Начальная установка.

Если n есть квадрат или более высокая степень, выдать n — **составное**.

Найти пару r, d натуральных чисел с минимальным значением rd^2 , такую, что $r|n^d - 1$ и $d^2 \log^2 n < r \leq (d+1)d^2 \log^2 n$;

Выбирать случайные нормированные многочлены $f(t) \in Z_n[t]$ степени d до тех пор, пока либо n не будет объявлено составным, либо не найдется такого многочлена $f(t)$, что $t^{n^d} \equiv t \pmod{(f(t))}$, и многочлен $t^{n^{d/q}} - t$ взаимно прост с $f(t)$ для каждого простого $q|d$;

Выбирать случайные многочлены $b(t) \in Z_n[t]$ степени d до тех пор, пока либо n не будет объявлено составным, либо не найдется такого многочлена $b(t)$, что $b(t)^{n^d-1} \equiv 1 \pmod{(f(t))}$, и многочлен $b(t)^{(n^d-1)/q} - 1$ взаимно прост с $f(t)$ для каждого простого делителя $q|r$.

2. Биномиальное сравнение.

Если

$$(x-1)^{n^d} \not\equiv x^{n^d} - 1 \pmod{(x^r - b(t), f(t), n)},$$

возвратить n — **составное**.

Возвратить n — **простое**.

Глава 5

Экспоненциальные алгоритмы факторизации

5.1 Метод пробного деления

Этот алгоритм просто делит n на все простые числа, пока $p^2 \leq n_0$, где n_0 не разложенная часть.

1. $n_0 = n$.

```
p = 2; while(p2 ≤ n0) {  
  while(p|n0) { n0 / = p; }  
  p = next prime; }
```

5.2 Метод квадратов

5.2.1 Метод Ферма

Этот метод применял еще Ферма для нахождения простых делителей нечетного числа n , представляя это число в виде разности квадратов двух чисел $n = a^2 - b^2 = (a - b)(a + b)$. Он хорошо работает, когда число n представляется в виде произведения близких чисел $n = uv$. Тогда $a = \frac{u+v}{2}$, $b = \frac{u-v}{2}$. При этом $a > \sqrt{n}$ (немногим больше), а $b^2 = a^2 - n$ мало. Худший случай, когда $v = 3$, $u = p$ и в этом случае $a = (n + 9)/6$.

Алгоритм Ферма. Дано нечетное число $n > 1$. Алгоритм либо выдает нетривиальный делитель, либо доказывает, что n - простое число.

```
for(⌈√n⌉ ≤ a ≤ (n + 9)/6) {  
  if(b = √(a2 - n) является целым) return a - b}
```

return n — составное.

Первое замечание. Этот алгоритм медленный, когда один из простых делителей мал. Поэтому лучше сочетать этот алгоритм с алгоритмом пробного деления до простых чисел меньше некоторого.

Второе замечание. Для худшего случая число a надо искать не только около $\sqrt{n}$ но сочетать еще с поиском таких чисел около $\sqrt{kn}$.

5.2.2 Метод Лемана

Метод Лемана учитывает указанные недостатки алгоритма Ферма и начинает искать малые делители пробным делением.

Алгоритм Лемана. Дано нечетное число $n > 21$. Алгоритм либо выдает нетривиальный делитель числа n , либо доказывает, что n - простое число.

1. Пробное деление.

Проверяем, имеет ли число n нетривиальный делитель $d \leq n^{1/3}$. Если имеет, то вернуть делитель d .

2. Цикл $\text{for}(1 \leq k \leq \lceil n^{1/3} \rceil)$

$\text{for}(\lceil 2\sqrt{kn} \rceil \leq a \leq \lceil 2\sqrt{kn} + n^{1/6}/(4\sqrt{k}) \rceil)$

$\text{if}(b = \sqrt{a^2 - 4kn} \text{ является целым}) \text{return } \gcd(a + b, n);$

return число n является простым.

Если алгоритм корректен, то количество операций

$$\sum_{k=1}^{\lceil n^{1/3} \rceil} \left(\frac{n^{1/6}}{4\sqrt{k}} + 1 \right) = O(n^{1/3}).$$

Докажем корректность.

Теорема 14. Метод Лемана корректен.

Доказательство. Будем считать, что не найден простой делитель в первом этапе. Тогда n простое, или $n = pq, p \leq q$ произведение двух простых. Нам надо показать, что в случае, когда n не простое, мы найдем простой делитель.

Пусть $B = \sqrt{n^{1/3}q/p}$. При любом фиксированном B и приближаемом числе $\alpha = \frac{p}{q}$ найдутся такие числа $u, v \leq B$, что

$$\left| \frac{u}{v} - \frac{p}{q} \right| < \frac{1}{vB}.$$

Здесь достаточно взять дробь $\frac{u}{v} = \frac{p_l}{q_l}$ от разложения в непрерывную числа $\frac{p}{q}$, что следующее $Q_{l+1} > B$. Тогда

$$b = |uq - vp| < \frac{q}{n^{1/6}\sqrt{q/p}} = n^{1/3} \quad (qp = n).$$

Примем за k число $k = uv$. Осталось показать, что $k = uv \leq n^{1/3}$. Так как $\frac{u}{v} < \frac{p}{q} + \frac{1}{vB}$ и $v \leq B$, то $k = uv = \frac{u}{v}v^2 < \frac{p}{q}v^2 + \frac{v}{B} \leq \frac{p}{q}n^{1/3} + 1 = n^{1/3} + 1$.

Положим $a = uq + vp$. Тогда с учетом того, что $k = uv, n = pq$ получаем $4kn = a^2 - b^2$ ($b = |uq - vp|$). Покажем, что $2\sqrt{kn} \leq a < 2\sqrt{kn} + \frac{n^{1/6}}{4\sqrt{k}}$ (ограничение поиска по a

сверху в алгоритме). Так как $uqv = kn$, получаем $a = uq + vp \geq 2\sqrt{kn}$ (арифметическое среднее не меньше геометрического). Пусть $a = 2\sqrt{kn} + E$. Тогда

$$4kn + 4E\sqrt{kn} \leq (2\sqrt{kn} + E)^2 = a^2 = 4kn + b^2 < 4kn + n^{2/3},$$

следовательно $E < \frac{n^{1/6}}{4\sqrt{k}}$.

Осталось показать, что $\gcd(a+b, n)$ является нетривиальным делителем. Так как $n|(a+b)(a-b)$, достаточно показать, что $a+b < n$. Если $\gcd(a+b, n) = 1$, то получили бы противоречие $n|(a-b) < n$. Имеем

$$a+b < 2\sqrt{kn} + \frac{n^{1/6}}{4\sqrt{k}} + n^{1/3} < 2\sqrt{(n^{1/3}+1)n} + \frac{n^{1/6}}{4\sqrt{n^{1/3}+1}} + n^{1/3} < n,$$

последнее неравенство выполняется при $n \geq 21$. □

5.3 Методы Монте-Карло

Здесь рассматриваются эвристические методы нахождения делителя числа n . Для них нет хорошей оценки количества операций до нахождения простого делителя. Трудоемкость зависит от везения в некотором (начальном) выборе (функции).

5.3.1 Ро-метод Полларда для разложения на множители.

Пусть S конечное множество и $f : S \rightarrow S$ отображение.

Рассмотрим последовательность $s_0, s_1 = f(s_0), \dots, s_{i+1} = f(s_i), \dots$. Такая последовательность всегда содержит некоторый хвост $s_0, \dots, s_{m-1}$ длины $m \geq 0$ и период $s_{m+l} = s_m$ длины l (овальная часть), т.е. последовательность напоминает букву ρ , что заключено в названии метода (Ро= ρ , а не сокращение от фамилии разработчика Pollard). Если рассмотреть эту последовательность по модулю простого числа p , то эта последовательность по эвристическим соображениям имеет длину хвоста и овала порядка $O(\sqrt{p})$.

Например, взяв функцию $f(x) = x^2 + 1 \pmod{n}$ построим последовательность F_i . Если p простой делитель n , то в последовательности $F_i - F_{2i}$ появится хотя бы одно значение, делящееся на p в промежутке $m \leq i \leq m+l$, где m длина хвоста, l - длина овала по модулю p . Соответственно алгоритм имеет вид:

Алгоритм разложения по ро-методу Полларда.

1. Инициализация.

Выбираем случайное $a \in [1, n-3]$;

Выбираем случайное $s \in [0, n-1]$;

$$U = V = s;$$

Определяем функцию $F(x) = x^2 + a \pmod{n}$.

2. Поиск делителя.

$$U = F(U);$$

$$V = F(V);$$

$$V = F(V);$$

$$g = \gcd(U - V, n);$$

if($g == 1$) go to Поиск делителя;
 3. Неудачная Инициализация
if($g == n$) go to Инициализация.
 4. Успех

return g ;

Период по модулю числа называют эпактом. Вообще говоря эпакт может зависеть и от начального значения, т.е. могут быть и разные периоды. Неудачные инициализации появляются только в случае, когда эпакты по модулю любого простого делителя равны эпакту самого числа n . Для простых чисел p эпакты функции $f(x) = x^2 + 1$ оцениваются величиной $O(\sqrt{p \log p})$.

5.4 Детские шаги, гигантские шаги

Этот метод часто используется и имеет уровень сложности порядка $O(\sqrt{m})$, где m длина поиска. Допустим задана последовательность $a_0, a_{i+1} = f(a_i)$ принимающее значение b при некотором $0 < i \leq m$ и требуется найти такое t , что $a_t = b$. Пусть $l = O(\sqrt{m})$ и так же легко вычислить $g(y) = f^{(-l)}(y)$ обратную к $f^{(l)}(x) = f(f(\dots f(x)))$. Тогда значение $t = cl + d$ может быть получено сравнением двух последовательностей $a_i, i < l$ и $b_0 = b, b_{j+1} = g(b_j)$ длинами порядка $O(\sqrt{m})$ и нахождением одинаковых среди них. Последняя операция осуществляется упорядочением по величине (хотя бы одну из последовательностей) и выполняется примерно за $O(\sqrt{m} \log m)$ операций. Причем эта оценка является не эвристической оценкой, выполняющейся часто только в среднем, а есть конкретная оценка сверху.

Этот метод часто используется для нахождения дискретного логарифма в циклической группе с образующей $g = a_0$ от заданного элемента $b = g^t$, $t = ?$, используя для этого функцию $f(x) = gx$.

5.5 Метод вычисления значений многочлена

Положим $B = \lceil n^{1/4} \rceil$, $f(x) = x(x-1)\dots(x-B+1)$. Вычисляя значения $y_j = \gcd(f(jB), n)$, $j = 1, 2, \dots, B+1$ получаем, что для простого делителя $\sqrt{n} > p|n$, хотя бы одно из чисел y_j должен делиться на p . Таким образом найдется делитель, если указанное число y_j отлично от нуля. В последнем случае само y_j можно разделить на части и считать их. Повторим такую процедуру пока не получится собственный делитель.

Для эффективной работы этой процедуры надо вычислить B значений многочлена степени B . Есть алгоритмы, позволяющие вычислить их за $O(B \log^2 B)$ операций. Алгоритм, приведенный в книге (алгоритм 9.6.5 в разделе 9.6.3) на мой взгляд требует порядка как минимум B^2 операций уже в первом этапе.

Один из способов вычисления учет разностей на одинаковых расстояниях. Например $f_1(x^2) = f(x+1) + f(x-1) - 2f(x)$ является многочленом от x^2 степени в два раза меньше исходной. Для самой функции находим $f_2(x^4)$ и т.д. Расчеты с учетом этого обстоятельства позволяют вычислить за $O(n^{1/4} \log^n)$ операций и найти делитель примерно за то же количество операций.

5.6 Основы теории квадратичных форм

Рассмотрим квадратичные формы

$$ax^2 + bxy + cy^2, a, b, c \in Z, \Delta = b^2 - 4ac \neq 0.$$

Если сделать замену:

$$x = \alpha X + \beta Y, y = \gamma X + \delta Y, \begin{pmatrix} \alpha & \beta \\ \gamma & \delta \end{pmatrix} \in Mat_2(Z)$$

то квадратичная форма перейдет в другую:

$$a'X^2 + b'XY + c'Y^2, \quad a' = a\alpha^2 + b\alpha\gamma + c\gamma^2, \\ b' = 2a\alpha\beta + b(\alpha\delta + \beta\gamma) + 2c\gamma\delta, c' = a\beta^2 + b\beta\delta + c\delta^2.$$

Две такие формы считаются эквивалентными, если определитель матрицы перехода равен ± 1 , т.е.

$$\begin{pmatrix} \alpha & \beta \\ \gamma & \delta \end{pmatrix} \in GL_2(Z).$$

В этом случае множество принимаемых формами (целых) значений совпадает. Здесь рассмотрим только формы с отрицательным дискриминантом принимающие положительные значения. Таковую форму называют приведенной, если

$$-a < b \leq a < c \quad \text{or} \quad 0 \leq b \leq a = c.$$

Теорема 15. (Гаусс)

Никакие две приведенные формы с отрицательным дискриминантом не эквивалентны. И каждая форма с отрицательным дискриминантом и с $a > 0$ эквивалентна некоторой приведенной форме.

Доказательство. Доказательство сводится к выделению квадрата $a(x+dy)^2 + (b-2ad)xy + (c-ad^2)y^2$, где d ближайшее к $b/(2a)$ целое. Если при этом новое $c' = c - ad^2 < a$, то заменим a на это число, а c заменяем на a . Если при этом нарушилось условие приведенности, повторяем процедуру. Это является так же алгоритмом приведения. $\square$

5.6.1 Композиция и группа классов

Пусть D - целое число, не являющееся полным квадратом, $(a_1, b, c_1), (a_2, b, c_2)$ - квадратичные формы дискриминанта D , и пусть c_1/a_2 является целым числом. Так как средние коэффициенты совпадают $a_1c_1 = a_2c_2$, т.е. $c_1/a_2 = c_2/a_1$.

Покажем, что произведение чисел n_1n_2 , представимых первой и второй формами, представляется формой $(a_1a_2, b, c_1/a_2)$ того же дискриминанта D . Действительно, легко проверяется, что

$$(a_1x_1^2 + bx_1y_1 + c_1y_1^2)(a_2x_2^2 + bx_2y_2 + c_2y_2^2) = a_1a_2x_3^2 + bx_3y_3 + (c_1/a_2)y_3^2,$$

где

$$x_3 = x_1x_2 - (c_1/a_2)y_1y_2, y_3 = a_1x_1y_2 + a_2x_2y_1 + by_1y_2.$$

Будем говорить, что квадратичная форма (a, b, c) является примитивной, если $\gcd(a, b, c) = 1$. Дискриминант квадратичной формы $D = b^2 - 4ac$ или делится на 4 или дает остаток 1 при делении на 4. Если дано целое число D , делящее на 4, то оно является дискриминантом формы $(1, 0, -D/4)$, если $D \equiv 1 \pmod{4}$, то дискриминантом формы $(1, 1, (1-D)/4)$. Пусть $C(D)$ обозначает множество классов эквивалентности примитивных бинарных квадратичных форм дискриминанта D . Каждый класс эквивалентности $\langle a, b, c \rangle$ - это множество форм, эквивалентных данной (a, b, c) .

Имеет место

Лемма 6. Пусть $\langle a_1, b, c_1 \rangle = \langle A_1, B_1, C_1 \rangle \in C(D)$, $\langle a_2, b, c_2 \rangle = \langle A_2, B, C_2 \rangle \in C(D)$, и числа $c_1/a_2, C_1/A_2$ являются целыми. Тогда $\langle a_1 a_2, b, c_1/a_2 \rangle = \langle A_1 A_2, B, C_1/A_2 \rangle$

Доказательство. Эквивалентным квадратичным формам соответствует один и тот же идеал дискриминанта D . А приведенному произведению соответствует произведение в классе идеалов. $\square$

Лемма 7. Пусть $(a_1, b_1, c_1), (a_2, b_2, c_2)$ - примитивные квадратичные формы дискриминанта D . Тогда существуют форма (A_1, B, C_1) , эквивалентная (a_1, b_1, c_1) и форма (A_2, B, C_2) , эквивалентная (a_2, b_2, c_2) , такие, что $\gcd(A_1, A_2) = 1$

Доказательство. Доказательство оставляется читателю в качестве упражнения. Можно найти в книге. $\square$

Эта лемма позволяет определить операцию умножения в классе эквивалентных примитивных форм $C(D)$ по формуле:

$$\langle a_1, b_1, c_1 \rangle * \langle a_2, b_2, c_2 \rangle = \langle A_1 A_2, B, C_1/A_2 \rangle.$$

Заметим, что из условия $\gcd(A_1, A_2) = 1, A_1 C_1 = A_2 C_2$ следует C_1/A_2 - целое и операция определено корректно. Коммутативность операции очевидна, ассоциативность проверяется не сложно. Если D четно, то форма $\langle 1, 0, -D/4 \rangle$ является единицей 1_D операции $*$, если D нечетно, то единицей 1_D является $\langle 1, 1, (1-D)/4 \rangle$. Если $(a, b, c) \in C(D)$, то форма (c, b, a) является обратным. Следовательно $C(D)$ коммутативная группа порядка $h(D)$, называемая группой классов примитивных бинарных квадратичных форм дискриминанта D .

Алгоритм умножения форм выглядит так:

1. $g = \gcd(a_1, a_2, (b_1 + b_2)/2)$;

Найдем u, v, w , такие, что $ua_1 + va_2 + w(b_1 + b_2)/2 = g$;

2. Возвратить ответ:

$$a_3 = \frac{a_1 a_2}{g^2}, b_3 = b_2 + 2 \frac{a_2}{g} \left(\frac{b_1 - b_2}{2} v - c_2 w \right), c_3 = \frac{b_3^2 - g}{4a_3}.$$

Для $h(D), D < 0$ имеется знаменитая формула Дирихле: где χ_D квадратичный характер по модулю D (символ Кронекера)- $\lambda_D(p) = \left(\frac{D}{p}\right)$ - символ Лежандра, в остальных точках определяется из мультипликативности, В 1918г. Шур показал, что

$$L(1, \chi_D) < \frac{1}{2} \ln |D| + \ln \ln |D| + 1,$$

что дает хорошую оценку сверху для числа классов $h(D)$. К. Зигель показал, что $h(D) = |D|^{1/2+o(1)}$ при $D \rightarrow -\infty$. Однако, его оценка неэффективна и не позволяет оценить $h(D)$ снизу.

5.6.2 Амбиговы формы и разложение на множители.

Нетрудно выписать все элементы группы $C(D)$ имеющие порядок 1 или 2 относительно умножения классов (т.е. обратных самому себе). Их называют амбиговыми формами. Имеются три вида таких форм: $(a, 0, c)$, (a, a, c) , (a, b, a) , которые приводят к разложению дискриминанта $D = -4ac$, $a(a - 4c)$, $(b - 2a)(b + 2a)$. Классификация амбиговых форм дается леммой:

Лемма 8. Пусть $D < 0$ отрицательный дискриминант. Если D четно, то амбиговы формы дискриминанта D содержат формы $(u, 0, v)$, где $0 < u \leq v$, $\gcd(u, v) = 1$, $uv = -D/4$. Кроме того, если $uv = -D/4$, причем $\gcd(u, v) = 1$ or 2 и число $(u+v)/2$ нечетно, то мы получим формы $((u+v)/2, v-u, (u+v)/2)$, когда $v/3 \leq u < v$ и формы $(2u, 2u, (u+v)/2)$, когда $0 < u < v/3$. Если D нечетно, то амбиговыми формами дискриминанта D являются формы $((u+v)/4, (v-u)/4, (u+v)/4)$, когда $-D = uv$, $0 < v/3 \leq u \leq v$, $\gcd(u, v) = 1$ и формы $(u, u, (u+v)/4)$, когда $-D = uv$, $0 < u \leq v/3$, $\gcd(u, v) = 1$.

Лемма доказывается простой проверкой.

Очевидно, форма 1_D амбигова и она соответствует тривиальному разложению на множители чисел $D/4$ или D . Тривиальному разложению соответствуют формы:

$$(2, 2, (4 - D)/8), \quad D \equiv 12 \pmod{16}, \quad D \leq -20,$$

$$(4, 4, 1 - D/16), \quad D \equiv 0 \pmod{32}, \quad D \leq -64$$

$$(3, 2, 3), \quad D = -32.$$

Все остальные формы приводят к нетривиальному разложению D или $D/4$.

Предположим, что число D имеет k различных нечетных простых делителей. Из последней леммы следует, что существует 2^{k-1} амбиговых форм дискриминанта D . Если же $D \equiv 12 \pmod{16}$ или $D \equiv 0 \pmod{32}$, то существует 2^k и 2^{k+1} амбиговых форм соответственно.

Предположим, что n нечетное число, имеющее как минимум два различных простых делителя. Если $n \equiv 3 \pmod{4}$, то $D = -n$ является дискриминантом, если же $n \equiv 1 \pmod{4}$, то $D = -4n$ является дискриминантом. В первом случае, если мы найдем амбигову форму, отличную от $(1, 1, (1+n)/4)$, то мы получим нетривиальное разложение числа n на множители. Во втором случае, если мы найдем амбигову форму, отличную от $(1, 0, n)$ и $(2, 2, (1+n)/2)$, то мы получим нетривиальное разложение числа n . Таким образом, задача разложения свелась к нахождению не тривиальных амбиговых форм.

Покажем, как можно найти амбиговы формы с дискриминантом $D < 0$. Пусть $h = h(D) = 2^l h_0$ (h_0 - нечетно) обозначает порядок группы $C(D)$. Если $f = \langle a, b, c \rangle \in C(D)$, то положим $F = f^{h_0}$. Тогда либо $F = 1_D$, либо одна из форм $F, F^2, \dots, F^{2^{l-1}}$ имеет порядок 2 в этой группе, т.е. является амбиговой формой. Если соответствующая амбигова форма отлична от тривиальной, то мы получим нетривиальное разложение.

Для такого способа разложения требуется знать число классов h . В предположении справедливости расширенной гипотезы Римана доказано, что величина

$$\bar{L} = \prod_{p \leq n^{1/5}} \left(1 - \frac{\chi_D(p)}{p}\right)^{-1}, \quad \bar{h} = (w/\pi) \sqrt{|D|} \bar{L}$$

мало отличается от числа классов

$$|h - \bar{h}| < cn^{2/5} \ln^2 n,$$

где c эффективно вычисляемая постоянная. Далее, для нахождения величины, кратной порядку любой заданной $f \in C(D)$, лежащей в интервале $(\bar{h} - cn^{2/5} \ln^2 n, \bar{h} + cn^{2/5} \ln^2 n)$ за время $O(n^{1/5} \ln n)$ может быть использован метод Шенкса "детские шаги, гигантские шаги". Поскольку вычисление $\bar{L}$ может быть выполнено за $O(n^{1/5})$ шагов, мы можем добиться разложения числа n при подходящем выборе f за $O(n^{1/5} \ln n)$ операций с целыми числами порядка n . В предположении расширенной гипотезы Римана, существует эффективная постоянная c' , такая, что классы примитивных приведенных форм (a, b, c) дискриминанта D при $a \leq c' \ln^2 |D|$ порождают всю группу классов $C(D)$. Это приводит к детерминированному алгоритму разложения со сложностью $O(n^{1/5} \ln^3 n)$ с целыми числами порядка n при предположении ERH.

Имеется вероятностный метод приближенного вычисления за ожидаемое время $O(|D|^{1/5+\epsilon})$ с погрешностью $O(|D|^{2/5+\epsilon})$, не зависящее от ERH. Соответственно этот алгоритм может разложить число n примерно за то же время в среднем (без твердой оценки сверху).

Глава 6

Субэкспоненциальные алгоритмы разложения на множители.

6.1 Базовый метод QS

Метод квадратичного решета основан так же в нахождении двух квадратов a^2, b^2 , разность которых делится на n . Только в отличии от метода Лемана, здесь квадраты строятся $a = a_1 * a_2 * ... a_k \bmod n$, $b = \sqrt{c_1 * ... * c_i} \bmod n$. Предварительно находятся множество различных a_i , таких, что

$$a_i^2 = k_i n + c_i.$$

Далее из таких представлений выбираются такие номера i , чтобы произведение выбранных c_i было квадратом $\prod_{i \in I} c_i = b^2$.

Рассмотрим это на простом примере $n = 1649$. Возьмем в качестве a_i , числа начиная с $\sqrt{n}$:

$$41^2 = 1681 \equiv 32 \bmod n,$$

$$42^2 = 1764 \equiv 115 \bmod n,$$

$$43^2 = 1849 \equiv 200 \bmod n.$$

Произведение первого и второго остатка является квадратом:

$$(41 * 43)^2 = 80^2 \bmod n \rightarrow n | (114^2 - 80^2).$$

Следовательно

$$\gcd(114 - 80, n) = 17 | n.$$

При больших n основная трудность заключается в формировании из правых частей квадрата числа. С этой целью, мы можем сеять такие значения a_i , чтобы

$$c_i = a_i^2 \bmod n$$

разлагались в произведение простых чисел из не очень большого набора. Например, если мы нашли набор из более чем k чисел a_i , таких, что соответствующие c_i разлагаются в произведение простых чисел из некоторого заданного набора k простых, то из представлений $c_i = \prod_{j \in J} p_j^{r_{ij}}$ можем выбрать такие, произведение которых будет квадратом. Для

этого в k - мерном пространстве векторов $\bar{c}_i = \{c_{ij}\}$, $c_{ij} = r_{ij} \bmod 2$ над полем Z_2 находим представление нулевого вектора в виде суммы. Это можно сделать вследствие того, что в k -мерном пространстве $k + 1$ векторов линейно зависимы.

Пусть $K = \#P$ количество простых чисел, которые могут встретиться (допускаются) в разложении чисел c_i . Остается выбрать достаточное количество квадратов (как минимум $K + 1$), остатки по модулю n которых дают разложение из указанного множества простых P (достаточно, чтобы это были простые, которые входят в нечетной степени в разложении чисел c_i). Удобно включить в множество P элемент -1 , взяв числа a_i немногим меньше $\sqrt{n}$ в разложении $a_i^2 - n$ получаем, что в разложение -1 входит в нечетной степени.

В традиционном алгоритме квадратичного решета QS множество P состоит из элемента -1 и простых чисел, не превосходящих ограничения B , так чтобы c_i образовали B -гладкие числа. На самом деле для простых чисел $p \in P$ должна выполняться $\left(\frac{n}{p}\right) = 1$, так как $p|x^2 - n \rightarrow \left(\frac{n}{p}\right) = 1$. Вероятность того, что $x^2 - n$ при $|x - \sqrt{n}| < n^\epsilon$ будет B -гладким оценивается как u^{-u} , $u = \frac{\log n}{2 \log B}$. На этом основывается вычисление минимального достаточного для нахождения разложения значения B :

$$\ln B \sim \frac{1}{2} \sqrt{\ln n \ln \ln n}, \quad u \sim \sqrt{\frac{\ln n}{\ln \ln n}}.$$

При этом количество операций, требуемых для разложения оценивается величиной:

$$L(n)^{1+o(1)}, \quad L(n) = \exp(\sqrt{\ln n \ln \ln n}).$$

Базовый алгоритм QS выглядит так:

1. Инициализация.

$$B = \lceil \sqrt{L(n)} \rceil;$$

Установить $p_1 = 2, a_1 = 1$;

Искать нечетные простые $p \leq B$, для которых $\left(\frac{n}{p}\right) = 1$, и присвоить им номера $p_2, \dots, p_K$; для $(2 \leq i \leq K)$ искать корни $\pm a_i$, такие, что $a_i^2 \equiv n \bmod (p_i)$;

2. Просеивание

Просеять последовательность $(x^2 - n), x = \lceil \sqrt{n} \rceil, \lceil \sqrt{n} \rceil + 1, \dots$ на предмет B -гладких чисел, пока мы не добавим в множество S $K + 1$ подходящую пару $(x, x^2 - n)$;

Линейная алгебра

$for((x, x^2 - n) \in S)\{$

Установить разложение на простые сомножители

$$x^2 - n = \prod_{i=1}^K p_i^{e_i}, \quad \bar{v}(x^2 - n) = (e_1, e_2, \dots, e_K);$$

}

Составить $(K + 1) \times K$ матрицу, строками которой являются всевозможные векторы $\bar{v}(x^2 - n)$, взятые по модулю 2.

Использовать алгоритмы линейной алгебры для нахождения нетривиального подмножества строк этой матрицы, дающих в сумме 0-вектор (по модулю 2), скажем $\bar{v}(x_1) + \bar{v}(x_2) + \dots + \bar{v}(x_k) = \bar{0}$.

4. Разложение на множители.

$x = x_1 x_2 \dots x_k \pmod{n};$
 $y = \sqrt{(x_1^2 - n)(x_2^2 - n) \dots (x_k^2 - n)} \pmod{n};$
//Извлечь этот корень из непосредственного разложения на множители
Вычислить

$$d = \gcd(x - y, n)$$

и выдать как результат.

По мнению автора выбор B гладких не оптимально. В качестве допускаемых простых P мы можем взять простые числа по другому распределению, включающие так же не только малые простые.

6.2 Решето числового поля

В алгоритме числового решета происходит просеивание в числовом поле и в Z_n , n разлагаемое на множители число. Основная идея числового поля заключается в следующем.

Из гладких целых чисел числового поля строится такое произведение

$$\prod_{i=1}^k \theta_i = \gamma^2.$$

Пусть $\phi : Z[\alpha] \rightarrow Z_n$ гомоморфизм. Добиваемся гладкости чисел $\varphi(\theta_i)$ и их квадратности $\prod_i \varphi(\theta_i) = v^2 \pmod{n}$ путем просеивания и методом линейной алгебры. Так получаем два квадрата:

$$u^2 \equiv \varphi(\gamma)^2 \equiv \varphi(\gamma^2) \equiv \varphi(\theta_1)\varphi(\theta_2)\dots\varphi(\theta_k) \equiv v^2 \pmod{n}.$$

В случае нетривиальности разложения $\gcd(u - v, n)$ даст делитель n .

Тут требуется множество пояснений. Построение гомоморфизма ϕ осуществляется таким образом. Выбирается степень многочлена $d = \lceil \frac{3 \ln n}{\ln \ln n} \rceil$ и число $m = \lceil n^{1/d} \rceil$. Представляя число n в m -ичной системе исчисления получаем многочлен:

$$n = m^d + c_{d-1}m^{d-1} + \dots + c_0 = f(m), \quad f(x) = x^d + c_{d-1}x^{d-1} + \dots + c_0.$$

Для этого представления показывается, что старший коэффициент равен 1. Это означает, что кольцо $Z[\alpha]$ состоит из целых чисел поля. В поле алгебраических чисел (вообще в алгебраическом расширении) целые числа определяются как числа из поля, удовлетворяющие полиномиальному уравнению с целыми коэффициентами (все числа из поля алгебраических чисел удовлетворяют этому условию), старший коэффициент которой равен 1. Целые числа поля образуют кольцо. Например в поле $Q[\sqrt{5}]$ кольцо $Z[\sqrt{5}]$ не исчерпывает все целые числа и кольцом целых чисел является $Z[\frac{1+\sqrt{5}}{2}]$.

Кольцо целых чисел поля является дедекендивым кольцом, где имеется однозначное разложение на простые идеалы. Именно этим пользуется при просеивании в поле алгебраических чисел и вычислении показателей по модулю 2. Показатели считаются не для простых чисел, а для простых идеалов, делящих простые числа.

Если многочлен $f(x) = g_1(x)g_2(x)$ получился приводимым (над Z , эквивалентно над Z), то мы сразу получаем разложение $n = g_1(m)g_2(m)$. Поэтому, в дальнейшем считается, что $f(x)$ неприводимым. Пусть α корень неприводимого многочлена $f(x)$. Просеиваемые

элементы θ из кольца $Z[\alpha]$ выбираются вида $a - b\alpha$ с целыми $a, b, |a|, b < M$. Среди них выбирается множество S (из k пар (a, b)) так, чтобы

$$\prod_{(a,b) \in S} (a - b\alpha) = \gamma^2, \quad \gamma \in Z[\alpha],$$

$$\prod_{(a,b) \in S} (a - bm) = v^2, \quad v \in Z.$$

Тогда в качестве делителя предлагается $(\varphi(\gamma) - v, n)$.

В реализации этой идеи возникает множество трудностей. В кольце $Z[\alpha]$ нет однозначности разложения, что затрудняет получение квадрата из произведений алгебраических чисел. Необходимым условием квадратности произведения является условие квадратности нормы. Норма определяет гомоморфизм относительно произведения из поля алгебраических чисел в поле Q и норма целых алгебраических чисел целое $N(\theta_1\theta_2) = N(\theta_1)N(\theta_2)$. Норма является произведением всех сопряженных элементов и в нашем случае совпадает с

$$N(a - b\alpha) = b^d f\left(\frac{a}{b}\right) = F(a, b).$$

Здесь $F(x, y)$ однородный вид исходного многочлена $f(x)$. Соответственно для получения гладкости мы просеиваем на значения в точках (a, b) многочлена $F(a, b)G(a, b)$, $G(x, y) = x - my$. Так как нам нужны квадраты для значений обоих многочленов, простым числам p по которым просеиваем делимости отводим два поля для F и для G . Однако, квадратность нормы не позволяет нам утверждать существование алгебраического числа γ , квадратом которой является $F(a, b)$. Поэтому, нам надо сеять делимости на идеалы, делящее простое число p .

Для простого число p обозначим через $R(p)$ множество вычетов $r \in [0, p - 1]$, таких, что $f(r) \equiv 0 \pmod{p}$. Они определяют разные идеалы, делящие простое число p , точнее главные идеалы порожденные этими числами разлагаются в произведение разных простых идеалов, делящих данное простое число p . Соответственно мы увеличиваем длину вектора показателей так, чтобы каждому такому простому идеалу соответствовало свое поле. Имеет место следующая лемма:

Лемма 9. Если S есть множество пар взаимно простых целых чисел a, b , такое, что каждое $a - b\alpha$ является в $Z[\alpha]$ гладким, и если $\prod_{(a,b) \in S} (a - b\alpha)$ является квадратом некоторого элемента из T , кольца целых алгебраических чисел в $Q[\alpha]$, то $\sum_{(a,b) \in S} \bar{v}(a - b\alpha) \equiv 0 \pmod{2}$.

Квадратность проверяется не в кольце $Z[\alpha]$ а в кольце целых алгебраических чисел T , где имеется однозначное разложение на простые идеалы. Остается еще проверить, является ли главным идеалом произведение идеалов, квадрат которой главный. Решение этого вопроса решается следующей леммой:

Лемма 10. Пусть $f(x)$ - неприводимый многочлен из $Z[x]$ со старшим коэффициентом единица, и пусть α - корень многочлена $f(x)$ в поле комплексных чисел. Положим d равным нечетному простому числу, и s - равным целому числу, такому, что $f(s) \equiv 0$

$\text{mod } (q)$ и $f'(s) \not\equiv 0 \pmod{(q)}$. Пусть S есть множество пар взаимно простых целых чисел (a, b) , таких, что q не делит ни одно из чисел $a - bs$ при $(a, b) \in S$, и $f'(\alpha)^2 \prod_{(a,b) \in S} (a - b\alpha)$ является квадратом в $Z[\alpha]$. Тогда

$$\prod_{(a,b) \in S} \left(\frac{a - bs}{q} \right) = 1.$$

Считается, что проверка по указанным ниже k показателям q_i, s_i при $k = \lceil 3 \log n \rceil$ достаточно для установления квадратности в элементах поля (не только квадратность идеала).

Сам алгоритм NFS выглядит так:

1. Начальная установка.

$$\begin{aligned} d &= \left\lceil \left(\frac{3 \ln n}{\ln \ln n} \right)^{1/3} \right\rceil, \\ B &= \left\lceil \exp \left(\sqrt[3]{\frac{8}{9} \ln n (\ln \ln n)^2} \right) \right\rceil, \\ m &= n^{1/d}, \end{aligned}$$

Записать n по основанию m :

$$n = m^d + c_{d-1}m^{d-1} + \dots + c_0 = f(m),$$

Попытаться разложить $f(x)$ на неприводимые многочлены в $Z[x]$ с использованием алгоритма разложения многочленов.

Если $f(x) = g(x)h(x)$ допускает нетривиальное разложение, то вывести в качестве делителя $g(m)$.

Пусть $F(x, y) = x^d + c_{d-1}x^{d-1}y + \dots + c_0y^d$ однородный вид многочлена $f(x)$ и $G(x, y) = x - my$.

for(простое $p \leq B$) найти множество

$$R(p) = \{r \in [0, p-1] : f(r) \equiv 0 \pmod{(p)}\},$$

$$k = \lceil 3 \log n \rceil,$$

Вычислить k простых чисел $q_1, q_2, \dots, q_k > B$, таких, что $R(q_j)$ содержит некоторый элемент s_j , такой что $f'(s_j) \not\equiv 0 \pmod{(q_j)}$, и сохранить при этом k пар q_j, s_j ,

$$B' = \sum_{p < B} \#R(p),$$

$$V = 1 + \pi(B) + B' + k,$$

$$M = B,$$

2. Решето

Использовать решето для нахождения множество S' пар взаимно простых целых чисел (a, b) , таких, что $0 < |a|, b \leq M$, и произведение $F(a, b)G(a, b)$ является B -гладким, пока

мы не получим $\#S' > V$, а если это не удастся, увеличить M и попробовать еще раз, или перейти к шагу - [Начальная установка] и там увеличить B .

3. Матрица.

// построим двоичную матрицу размером $V \times V$ for $((a, b) \in S' \{ //$ вычислим $\bar{v}(a - b\alpha)$, двоичный вектор показателей для числа $a - b\alpha$, имеющий V битов.

Установить первый бит вектора показателей 1, если $G(a, b) < 0$, иначе установить этот бит 0.

Следующие $\pi(B)$ битов зависит от простых чисел $p \leq B$, определим p^γ как степень числа p в разложении $|G(a, b)|$ на простые сомножители.

Установить бит для p в 1, если γ нечетно, иначе установить этот бит 0.

Следующие B' битов будут соответствовать парам p, r , где p — простое, не превосходящее B , и $r \in R(p)$. Здесь используется обозначение $v_{p,r}(a - b\alpha)$.

Установить бит для пары (p, r) в 1, если $v_{p,r}(a - b\alpha)$ нечетно, иначе установить 0,

// последние k битов соответствуют парам q_j, s_j для установления квадратности числа (а не только идеалов).

Установить бит для пары q_j, s_j в 1, если $(\frac{a - bs_j}{q_j}) = -1$, иначе установить 0.

Добавить вектор показателей $\bar{v}(a - b\alpha)$ в качестве очередной строки нашей матрицы.

}.

4. Линейная алгебра.

При помощи методов линейной алгебры найти непустое подмножество S множества S' , такое, что

$$\sum_{(a,b) \in S} \bar{v}(a - b\alpha) = \bar{0}.$$

5. Квадратные корни.

Использовать известное разложение квадрата $\prod_{(a,b) \in S} (a - bm)$ на простые сомножители для нахождения вычета $v \bmod (n)$, такого, что $\prod_{(a,b) \in S} (a - bm) \equiv v^2 \bmod (n)$.

Используя метод извлечения квадратного корня из алгебраического числа найти алгебраическое число γ , принадлежащее $Z[\alpha]$, являющееся квадратным корнем из

$$f'(\alpha)^2 \prod_{(a,b) \in S} (a - b\alpha),$$

и путем простой подстановки $\alpha \rightarrow m$ вычислить $u = \varphi(\gamma)$.

6. Разложение.

$$\text{return gcd}(u - f'(m)v, n).$$

Если алгоритм дает тривиальный делитель числа n , то надо искать новые зависимости, если их нет, то просеиваем дальше для нахождения новых зависимостей.